



DIPARTIMENTO DI INGEGNERIA ELETTRICA, ELETTRONICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN: INGEGNERIA INFORMATICA

ADVANCED PROGRAMMING LANGUAGES

Healthcare Facility Operations and Clinical Management Platform

Documentazione

STUDENTI:

STEFANO CARAMAGNO

FEDERICO CALABRESE

DOCENTI:

PROF.SSA VINCENZA CARCHIOLO

PROF. MARCO GRASSIA

ANNO ACCADEMICO 2025-2026

SOMMARIO

1. INTRODUZIONE.....	1
1.1. CONTESTO APPLICATIVO	1
1.2. OBIETTIVI DELLA PIATTAFORMA.....	2
1.3. AMBITO COPERTO E CONFINI DEL SISTEMA.....	3
1.4. STRUTTURA DEL DOCUMENTO	4
2. VISIONE GENERALE DEL SISTEMA.....	6
2.1. PANORAMICA DELLA SOLUZIONE	6
2.2. ATTORI APPLICATIVI	7
2.2.1. ADMIN	7
2.2.2. CLINICIAN	8
2.2.3. PATIENT.....	8
2.2.4. DELEGATE	8
2.3. RESPONSABILITA' OPERATIVE PER RUOLO.....	9
2.4. SEPARAZIONE DEI COMPITI E VINCOLI DI ACCESSO	10
3. REQUISITI DELLA SOLUZIONE	12
3.1. REQUISITI FUNZIONALI PRINCIPALI.....	12
3.2. REQUISITI NON FUNZIONALI.....	13
3.3. REQUISITI DI SICUREZZA.....	14
3.4. REQUISITI DI TRACCIABILITA' E AUDITABILITA'	16
3.5. REQUISITI DI MODULARITA' E PORTABILITA'	17
3.6. REQUISITI DEL MODULO AI CONSULTATIVO	18
4. FLUSSI OPERATIVI PRINCIPALI	20
4.1. GESTIONE DELL'ACCESSO E DEL CICLO DI VITA DELL'ACCOUNT	20
4.1.1. REGISTRAZIONE	20
4.1.2. ATTIVAZIONE ACCOUNT	20
4.1.3. LOGIN E LOGOUT	21
4.1.4. RESET PASSWORD.....	21
4.2. GESTIONE ANAGRAFICHE, PROFILI, DELEGHE E CONSENSI	22
4.3. GESTIONE DEL CATALOGO PRESTAZIONI.....	22
4.4. GESTIONE DISPONIBILITA', SLOT E APPUNTAMENTI.....	23
4.4.1. RICERCA DISPONIBILITA'	23
4.4.2. PRENOTAZIONE.....	23
4.4.3. RIPIANIFICAZIONE	24
4.4.4. ANNULLAMENTO	24
4.5. CHECK-IN AMMINISTRATIVO E PRESA IN CARICO	24

4.6.	PERCORSO CLINICO	25
4.6.1.	ACCESSO ALL'ENCOUNTER.....	25
4.6.2.	REGISTRAZIONE ANAMNESI.....	25
4.6.3.	REGISTRAZIONE PARAMETRI VITALI	26
4.6.4.	GESTIONE ORDINI CLINICI	26
4.6.5.	REGISTRAZIONE ESECUZIONI.....	26
4.6.6.	REDAZIONE, FIRMA E PUBBLICAZIONE DEL REFERTO.....	26
4.7.	PRE-TRIAGE LATO PATIENT E DELEGATE.....	27
4.8.	GESTIONE DEI PAGAMENTI.....	27
4.8.1.	PAGAMENTO IN APP	27
4.8.2.	PAGAMENTO IN SEDE.....	28
4.8.3.	RICONCILIAZIONE ED ESITI PROVIDER	28
4.9.	NOTIFICHE, PROMEMORIA E AUDIT LOG	28
4.10.	CONSULTAZIONE DELL'ASSISTENTE AI DA PARTE DEL CLINICO	29
5.	ARCHITETTURA DEL SISTEMA	30
5.1.	PRINCIPI ARCHITETTURALI ADOTTATI	30
5.2.	VISTA ARCHITETTURALE COMPLESSIVA	31
5.3.	DIAGRAMMA ARCHITETTURALE DELLA PIATTAFORMA	33
5.4.	TOPOLOGIA DI RUNTIME.....	34
5.5.	COMUNICAZIONI TRA I COMPONENTI.....	35
5.6.	CONFINI APPLICATIVI E OWNERSHIP DEI DATI	36
6.	TECNOLOGIE ADOTTATE E MOTIVAZIONI PROGETTUALI.....	38
6.1.	FRONTEND: HTML, CSS, TAILWIND CSS E JAVASCRIPT	38
6.2.	API GATEWAY: GO E GIN	39
6.3.	CORE SERVICE: ASP.NET CORE 8.....	40
6.4.	AI ASSISTANT: DJANGO, DJANGO REST FRAMEWORK E TRANSFORMERS	41
6.5.	PERSISTENZA DATI: MYSQL 8.0	42
6.6.	CONTAINERIZZAZIONE E ORCHESTRAZIONE: DOCKER E DOCKER COMPOSE.....	43
6.7.	SERVIZI DI SUPPORTO: MAILPIT E CACHE LOCALE HUGGING FACE	44
7.	PROGETTAZIONE DEL FRONTEND	45
7.1.	ARCHITETTURA DEL FRONTEND MULTI-PAGE.....	45
7.2.	ORGANIZZAZIONE DELLE INTERFACCE PER RUOLO	46
7.2.1.	AREA AUTH.....	46
7.2.2.	AREA ADMIN.....	46
7.2.3.	AREA CLINICIAN	47
7.2.4.	AREA PATIENT	47

7.2.5.	AREA DELEGATE	48
7.3.	COMPONENTI E SCRIPT CONDIVISI	48
7.4.	GESTIONE DELLA SESSIONE LATO CLIENT	49
7.5.	INTERAZIONE CON LE API TRAMITE FRONTEND ORIGIN	49
7.6.	GESTIONE DEGLI ERRORI E FALLBACK APPLICATIVI	50
8.	PROGETTAZIONE DELL'API GATEWAY	52
8.1.	RUOLO DEL GATEWAY NELL'ARCHITETTURA.....	52
8.2.	ROUTING E REVERSE PROXY	53
8.3.	MIDDLEWARE CROSS-ROUTING	54
8.3.1.	REQUEST ID.....	54
8.3.2.	LOGGING STRUTTURATO.....	54
8.3.3.	RECOVERY JSON.....	55
8.3.4.	CORS	55
8.3.5.	RATE LIMITING	55
8.3.6.	VERIFICA JWT	56
8.4.	TOKEN INTROSPECTION VERSO IL CORE SERVICE	56
8.5.	AGGREGAZIONE DELLO STATO DI SALUTE DEI SERVIZI	57
9.	PROGETTAZIONE DEL CORE SERVICE	58
9.1.	ORGANIZZAZIONE DELLA SOLUTION.....	58
9.1.1.	CoreService.Api	58
9.1.2.	CoreService.Application	59
9.1.3.	CoreService.Domain	59
9.1.4.	CoreService.Infrastructure	59
9.2.	LAYER API	60
9.3.	LAYER APPLICATION.....	60
9.4.	LAYER DOMAIN	61
9.5.	LAYER INFRASTRUCTURE	62
9.6.	MOTIVAZIONI DELLA SEPARAZIONE IN LAYER.....	63
10.	MODULI APPLICATIVI DEL CORE SERVICE	64
10.1.	AUTH & IDENTITY	64
10.1.1.	EMISSIONE JWT	64
10.1.2.	REVOCA TOKEN.....	65
10.1.3.	ME ENDPOINT	65
10.1.4.	ATTIVAZIONE ACCOUNT E RESET PASSWORD.....	66
10.2.	REGISTRY	66
10.2.1.	GESTIONE PAZIENTI.....	67

10.2.2.	GESTIONE DELEGATI.....	67
10.2.3.	GESTIONE CLINICI.....	67
10.2.4.	PROFILI, DELEGHE E CONSENSI	68
10.3.	SCHEDULING	68
10.3.1.	DISPONIBILITA' E SLOT	69
10.3.2.	AGENDA DEL CLINICO	69
10.3.3.	PRENOTAZIONI E STATI DEGLI APPUNTAMENTI.....	69
10.3.4.	CHECK-IN E NO-SHOW	70
10.4.	CLINICAL	70
10.4.1.	CATALOGO PRESTAZIONI.....	70
10.4.2.	PRE-TRIAGE.....	71
10.4.3.	ENCOUNTER CLINICI	71
10.4.4.	WORKFLOW DEL REFERTO	71
10.5.	PAYMENTS.....	72
10.5.1.	PAYMENT INTENTS	72
10.5.2.	TRANSAZIONI	72
10.5.3.	CHECKOUT WORKFLOW.....	73
10.5.4.	PROVIDER SIMULATO E WEBHOOK	73
10.6.	EVENTS	73
10.6.1.	NOTIFICHE APPLICATIVE	74
10.6.2.	AUDIT LOG.....	74
10.6.3.	SCHEDULER DELLE NOTIFICHE	74
10.6.4.	INVIO E-MAIL TRAMITE BACKGROUND SERVICE	75
11.	PROGETTAZIONE DEL MODULO AI ASSISTANT.....	76
11.1.	OBIETTIVO DEL SERVIZIO AI	76
11.2.	RUOLO CONSULTATIVO E LIMITI OPERATIVI.....	76
11.3.	ENDPOINT DI QUESTION ANSWERING	77
11.4.	AUTENTICAZIONE TRAMITE JWT DEL CORE SERVICE	78
11.5.	VERIFICA DEL RUOLO CLINICIAN	78
11.6.	GESTIONE DI INPUT JSON E MULTIPART	79
11.7.	ESTRAZIONE DEL TESTO DAGLI ALLEGATI	79
11.8.	RILEVAZIONE DI DATI IDENTIFICATIVI DIRETTI	80
11.9.	INFERENZA LOCALE TRAMITE MODELLO HUGGING FACE	81
11.10.	CARICAMENTO LAZY DEL MODELLO E FUNZIONAMENTO OFFLINE.....	81
11.11.	NATURA STATELESS DEL SERVIZIO	82
12.	MODELLO DEI DATI E STRATEGIA DI PERSISTENZA	84

12.1.	SCELTA ARCHITETTURALE SCHEMA-PER-DOMAIN	84
12.2.	DATABASE healthcare_registry.....	85
12.3.	DATABASE healthcare_scheduling	86
12.4.	DATABASE healthcare_clinical.....	87
12.5.	DATABASE healthcare_payments.....	89
12.6.	DATABASE healthcare_events	90
12.7.	RELAZIONI LOGICHE TRA DOMINI.....	91
12.8.	ACCESSI CROSS-DOMAIN CONTROLLATI.....	92
12.9.	SEED INIZIALE E DATI DIMOSTRATIVI	92
13.	SICUREZZA, PRIVACY E CONTROLLO DEGLI ACCESSI	95
13.1.	AUTENTICAZIONE E GESTIONE DEGLI ACCESSI	95
13.2.	AUTORIZZAZIONE BASATA SUI RUOLI	95
13.3.	SEPARAZIONE TRA AMBITO AMMINISTRATIVO E AMBITO CLINICO	96
13.4.	REVOCA DEI TOKEN E INTROSPECTION.....	97
13.5.	PROTEZIONE DEI DATI PERSONALI E CLINICI.....	98
13.6.	AUDITABILITA' DELLE OPERAZIONI SENSIBILI	99
13.7.	SALVAGUARDIE SPECIFICHE DEL MODULO AI	100
14.	INTERFACCE REST E ORGANIZZAZIONE DEGLI ENDPOINT	102
14.1.	CONVENZIONI DI PROGETTAZIONE DELLE API.....	102
14.2.	ENDPOINT DI AUTENTICAZIONE	103
14.3.	ENDPOINT DI REGISTRY	104
14.4.	ENDPOINT DI SCHEDULING	106
14.5.	ENDPOINT DI CLINICAL.....	107
14.6.	ENDPOINT DI PAYMENTS	109
14.7.	ENDPOINT DI NOTIFICATIONS E AUDIT	109
14.8.	ENDPOINT DI HEALTH	110
14.9.	ENDPOINT DEL MODULO AI.....	111
14.10.	FORMATO DELLE RISPOSTE ED ERROR HANDLING.....	111
15.	DEPLOY, CONFIGURAZIONE ED ESECUZIONE DEL SISTEMA	113
15.1.	PREREQUISITI DI ESECUZIONE	113
15.2.	VARIABILI D'AMBIENTE	114
15.3.	DOCKERFILE DEI COMPONENTI.....	115
15.4.	ORCHESTRAZIONE CON DOCKER COMPOSE	116
15.5.	SEQUENZA DI AVVIO DEI SERVIZI	117
15.6.	INIZIALIZZAZIONE DEL DATABASE	118
15.7.	MAILPIT PER LA GESTIONE SMTP LOCALE	118

15.8.	MONTAGGIO DELLA CACHE LOCALE DEL MODELLO AI	119
16.	VALIDAZIONE FUNZIONALE E SCENARI DI UTILIZZO	121
16.1.	SCENARIO AMMINISTRATIVO END-TO-END	121
16.2.	SCENARIO CLINICO END-TO-END	122
16.3.	SCENARIO PATIENT END-TO-END	124
16.4.	SCENARIO DELEGATE END-TO-END	126
16.5.	SCENARIO DI CONSULTAZIONE AI	127
16.6.	VERIFICA DEI CONTROLLI DI ACCESSO	128
16.7.	VERIFICA DI HEALTH E DISPONIBILITA' DEI SERVIZI.....	129
17.	STRUTTURA DELLA REPOSITORY.....	131
17.1.	ORGANIZZAZIONE DELLE DIRECTORY PRINCIPALI.....	131
17.2.	SEPARAZIONE TRA FRONTEND, GATEWAY, CORE SERVICE E AI ASSISTANT .	132
17.3.	CARTELLE INFRASTRUTTURALI E DI SUPPORTO	134
17.4.	COERENZA TRA REPOSITORY E ARCHITETTURA IMPLEMENTATA	135
18.	CONSIDERAZIONI FINALI	137
18.1.	SINTESI DEI RISULTATI OTTENUTI	137
18.2.	COERENZA TRA ARCHITETTURA PROGETTATA E IMPLEMENTAZIONE	137
18.3.	PUNTI DI FORZA DELLA SOLUZIONE	138
18.4.	POSSIBILI EVOLUZIONI FUTURE	139

1. INTRODUZIONE

1.1. CONTESTO APPLICATIVO

La trasformazione digitale dei processi sanitari richiede piattaforme software in grado di governare in modo integrato attività amministrative, cliniche e informative, mantenendo al tempo stesso elevati livelli di **affidabilità, tracciabilità, sicurezza e chiarezza operativa**. In un contesto nel quale interagiscono profili differenti — personale amministrativo, personale sanitario, pazienti e soggetti delegati — il valore di una soluzione applicativa non dipende soltanto dalla disponibilità di funzionalità isolate, ma soprattutto dalla capacità di orchestrare in maniera coerente l'intero ciclo di erogazione del servizio.

In tale scenario, la gestione di una struttura sanitaria privata non può essere ricondotta a una semplice raccolta di dati anagrafici o a un insieme di schermate per la prenotazione di appuntamenti. Essa implica, piuttosto, la costruzione di un ecosistema applicativo capace di supportare processi eterogenei ma strettamente interdipendenti: registrazione e attivazione degli account, gestione dei profili, amministrazione delle deleghe, consultazione delle disponibilità, prenotazione e ripianificazione degli appuntamenti, check-in, presa in carico clinica, compilazione del pre-triage, registrazione di informazioni anamnestiche e parametri vitali, redazione del referto, gestione dei pagamenti, invio di notifiche e monitoraggio delle operazioni sensibili.

L'eterogeneità di tali processi impone una progettazione che non sia soltanto funzionale, ma anche **architetturalmente disciplinata**. Un sistema di questo tipo deve infatti garantire una chiara separazione tra le responsabilità dei vari attori, evitando commistioni improprie tra ambito amministrativo e ambito clinico; deve rendere possibile il controllo puntuale degli accessi; deve assicurare la persistenza strutturata dei dati; deve rendere osservabile il comportamento dei servizi; deve infine consentire un deploy riproducibile e coerente nei diversi ambienti di esecuzione.

La piattaforma descritta in questo documento nasce per rispondere a tali esigenze attraverso una soluzione web modulare, articolata in componenti specializzati ma integrati tra loro. Il sistema è costruito intorno a un modello **multi-ruolo**, nel quale ciascun soggetto opera entro un perimetro definito e con capacità coerenti con il proprio ruolo. L'accesso alla piattaforma avviene tramite un frontend web a pagine multiple, mentre la logica applicativa è distribuita su un insieme di servizi distinti: un **API Gateway** dedicato al controllo del traffico in ingresso e alle responsabilità trasversali, un **Core Service** che governa i principali domini di business, un **AI Assistant** separato e consultivo a supporto del personale clinico, e un livello di persistenza strutturato secondo una strategia *schema-per-domain*.

L'adozione di una simile impostazione consente di affrontare in modo rigoroso sia le esigenze funzionali della piattaforma sia i vincoli di natura non funzionale. Da un lato, il sistema mette a disposizione un insieme organico di funzionalità operative; dall'altro, adotta scelte tecniche orientate alla **manutenibilità**, alla **modularità**, alla **portabilità** e alla **contenibilità della complessità**. In questo senso, l'architettura non rappresenta un elemento secondario rispetto alle funzionalità, ma costituisce il presupposto necessario affinché tali funzionalità possano essere erogate in modo coerente, controllato e sostenibile nel tempo.

1.2. OBIETTIVI DELLA PIATTAFORMA

L'obiettivo principale della piattaforma è realizzare un sistema software unificato per la gestione dei processi operativi di una struttura sanitaria privata, in grado di coprire l'intero flusso che va dall'interazione iniziale con il paziente fino alla conclusione del percorso amministrativo e clinico. Tale obiettivo generale si traduce nella necessità di predisporre una soluzione che non si limiti a informatizzare singole attività, ma che permetta di governare in maniera coordinata le interazioni tra utenti, dati, servizi applicativi e regole di accesso.

Un primo obiettivo consiste nel fornire un'esperienza d'uso coerente e specializzata per ciascun profilo applicativo. L'**Admin** deve poter operare sulle anagrafiche, sulle prenotazioni, sui check-in, sui pagamenti e sulle notifiche, senza accedere a informazioni cliniche eccedenti il proprio perimetro operativo. Il **Clinician** deve poter consultare la propria agenda, prendere in carico il paziente nel contesto corretto, registrare le informazioni cliniche e interagire con un assistente AI di natura strettamente consultiva. Il **Patient** deve poter gestire il proprio profilo, prenotare prestazioni, compilare il pre-triage, consultare la documentazione disponibile, monitorare i pagamenti e ricevere comunicazioni. Il **Delegate**, infine, deve poter operare in modo controllato per conto dell'assistito, nel rispetto dei vincoli derivanti dalle deleghe attive.

Un secondo obiettivo riguarda la strutturazione rigorosa dei processi. La piattaforma è concepita per rendere espliciti gli stati e le transizioni che caratterizzano i flussi operativi: un appuntamento viene creato, può essere ripianificato o annullato, può passare attraverso il check-in, può condurre all'apertura di un encounter clinico, può richiedere un pagamento, può generare notifiche e può produrre un referto finale. La modellazione di tali passaggi consente di evitare ambiguità operative e rende il sistema più robusto nei confronti di usi impropri, errori manuali e incoerenze di stato.

Un terzo obiettivo è la tutela della separazione tra domini e responsabilità. La piattaforma è progettata affinché il governo dei dati e delle funzionalità sia attribuito a componenti specializzati: il gateway si occupa delle responsabilità trasversali e del dispatching, il core applicativo governa i domini di business, il modulo AI è isolato come servizio separato e consultivo, mentre la persistenza è organizzata in schemi distinti per dominio funzionale. Questa separazione non ha una finalità meramente organizzativa, ma risponde all'esigenza di preservare la **coerenza del modello**, favorire l'evoluzione del sistema e ridurre l'accoppiamento tra le componenti.

Un ulteriore obiettivo fondamentale è assicurare elevati standard di **sicurezza applicativa**. La presenza di dati personali, dati sanitari, eventi di audit e informazioni di accesso impone l'adozione di meccanismi rigorosi di autenticazione, autorizzazione, revoca dei token, verifica dei ruoli e tracciamento delle operazioni sensibili. In tale prospettiva, la piattaforma non considera la sicurezza come un livello aggiuntivo successivo allo sviluppo funzionale, ma come una proprietà intrinseca della progettazione.

Rientra inoltre tra gli obiettivi della soluzione l'integrazione di un modulo di supporto basato su intelligenza artificiale, concepito non come sistema decisionale autonomo, bensì come strumento di *question answering* rivolto esclusivamente al personale clinico. Tale modulo ha una funzione di supporto informativo e opera entro vincoli precisi: validazione dell'identità dell'utente, verifica del ruolo, elaborazione di input testuali o documentali,

controlli sui dati identificativi diretti, inferenza locale tramite modello precaricato e assenza di persistenza proprietaria. L'obiettivo non è delegare alla componente AI la responsabilità clinica, ma fornire un supporto consultivo compatibile con un'architettura sicura e controllata.

Infine, la piattaforma persegue un obiettivo di natura infrastrutturale e operativa: rendere il sistema **facilmente distribuibile, riproducibile e coerente** nei diversi ambienti di esecuzione. L'impiego della containerizzazione e dell'orchestrazione tramite Docker Compose risponde a questa esigenza, consentendo di definire in modo esplicito i servizi runtime, le dipendenze, le variabili di configurazione, i volumi condivisi e le modalità di avvio dell'intero stack.

1.3. AMBITO COPERTO E CONFINI DEL SISTEMA

L'ambito coperto dalla piattaforma comprende i principali processi digitali relativi alla gestione amministrativa e clinica di una struttura sanitaria privata, con particolare attenzione alla continuità dei flussi e alla corretta delimitazione dei perimetri di responsabilità. Il sistema include funzionalità di accesso e gestione dell'identità, amministrazione di profili e deleghe, catalogazione delle prestazioni, disponibilità e prenotazione degli slot, gestione degli appuntamenti, check-in, pre-triage, percorso clinico, refertazione, pagamenti, notifiche, audit log e supporto consultivo AI per il personale sanitario.

Per quanto riguarda la sfera amministrativa, la piattaforma copre le attività necessarie alla registrazione e manutenzione delle anagrafiche, alla gestione dei clinici e dei soggetti delegati, alla consultazione e amministrazione delle prenotazioni, alla gestione delle disponibilità operative, al check-in del paziente e alla registrazione o riconciliazione dei pagamenti. Tali funzionalità sono progettate per offrire una visione completa del ciclo amministrativo, mantenendo al contempo una netta separazione dalle informazioni e dalle azioni strettamente cliniche.

Sul versante clinico, il sistema copre le attività necessarie alla presa in carico del paziente nel contesto dell'appuntamento corretto, alla consultazione dei dati utili al percorso di visita, alla registrazione di anamnesi e parametri vitali, alla gestione di ordini ed esecuzioni, nonché alla produzione e pubblicazione del referto. Il modulo AI rientra in questo ambito solo come componente di supporto informativo e non altera la centralità del flusso clinico governato dal core applicativo.

Dal punto di vista del paziente e dell'eventuale delegato, la piattaforma copre l'interazione self-service con il sistema: gestione del profilo, consultazione del catalogo, ricerca delle disponibilità, prenotazione, ripianificazione, annullamento, compilazione del pre-triage, consultazione delle informazioni rese disponibili, verifica dello stato dei pagamenti e ricezione di notifiche. Il delegato opera in un perimetro derivato e controllato, dipendente dall'esistenza di una delega valida e dalle relative abilitazioni.

I confini del sistema sono altrettanto importanti dell'ambito coperto. La piattaforma non è progettata come un *electronic health record* generalista né come un sistema di interoperabilità sanitaria su scala territoriale o nazionale. Essa non implementa integrazioni con sistemi esterni di fascicolo sanitario, non prevede comunicazioni con dispositivi biomedicali, non esegue orchestrazioni inter-organizzative complesse e non

introduce servizi di pagamento reali verso provider di terze parti: il provider di pagamento è infatti simulato internamente per consentire la modellazione e la verifica del workflow senza dipendere da integrazioni esterne effettive.

Allo stesso modo, il modulo AI non rappresenta un sistema di diagnosi automatica, di prescrizione autonoma o di supporto decisionale vincolante. La sua funzione è limitata a un uso **consultivo**, controllato e contestualizzato, senza memorizzazione locale persistente dei dati elaborati e con specifiche salvaguardie sui contenuti in ingresso.

Anche sul piano tecnico il sistema stabilisce confini chiari. Il browser interagisce esclusivamente con l'origine del frontend; il frontend statico, servito da Nginx, inoltra le richieste API al gateway; il gateway costituisce il punto di ingresso unificato verso i servizi interni; il Core Service detiene la responsabilità primaria dei domini di business; l'AI Assistant è isolato come servizio separato; la persistenza è centralizzata in una singola istanza MySQL articolata in più schemi. Questa delimitazione riduce l'accoppiamento tra componenti, chiarisce le responsabilità e rende il sistema più leggibile sia sul piano progettuale sia su quello operativo.

1.4. STRUTTURA DEL DOCUMENTO

Il presente documento è organizzato secondo una progressione che accompagna il lettore dalla comprensione del contesto e dei requisiti fino all'analisi dettagliata dell'architettura, dei moduli applicativi, delle scelte implementative e degli aspetti infrastrutturali. La struttura è stata definita con l'obiettivo di mantenere un equilibrio tra **visione d'insieme**, **rigore tecnico** e **aderenza all'implementazione effettiva** della piattaforma.

Dopo questa introduzione, il **Capitolo 2** presenta la visione generale del sistema, descrivendo gli attori applicativi, le rispettive responsabilità operative e i vincoli di accesso che governano le interazioni tra i diversi ruoli. Tale sezione ha la funzione di stabilire il quadro funzionale di riferimento entro il quale si collocano tutti i flussi successivi.

Il **Capitolo 3** raccoglie i requisiti della soluzione, distinguendo tra requisiti funzionali, non funzionali, di sicurezza, di tracciabilità, di modularità e relativi al modulo AI consultivo. In questo modo il documento esplicita non soltanto ciò che il sistema fa, ma anche le proprietà qualitative che deve garantire.

Il **Capitolo 4** analizza i principali flussi operativi, seguendo la sequenza con cui i processi si manifestano nell'uso della piattaforma: accesso, gestione del profilo, prenotazioni, check-in, percorso clinico, pagamenti, notifiche e consultazione dell'assistente AI. Tale impostazione permette di osservare il sistema non solo come insieme di moduli, ma come rete coordinata di processi.

Il **Capitolo 5** introduce l'architettura del sistema e ne presenta i principi fondamentali, la vista complessiva, la topologia di runtime, i flussi di comunicazione tra i componenti e l'ownership dei dati. All'interno di questo capitolo è inserito il **diagramma architetturale della piattaforma**, utilizzato come figura di riferimento per la comprensione della struttura complessiva del software.

Il **Capitolo 6** espone le tecnologie adottate e le motivazioni progettuali associate alle principali scelte implementative, chiarendo il ruolo del frontend web, del gateway Go/Gin,

del Core Service in ASP.NET Core, del modulo AI in Django/DRF, della persistenza MySQL e dell'orchestrazione tramite Docker Compose.

Il **Capitolo 7** è dedicato al frontend e ne descrive l'architettura multi-page, l'organizzazione per ruolo, la gestione della sessione lato client, i componenti condivisi e le modalità di interazione con le API. Il **Capitolo 8** approfondisce invece la progettazione dell'API Gateway, con particolare attenzione alle responsabilità di routing, reverse proxy, middleware trasversali, introspection dei token e health aggregata.

I **Capitoli 9 e 10** trattano il Core Service: il primo ne illustra la struttura in layer e l'organizzazione interna della solution, mentre il secondo analizza nel dettaglio i moduli applicativi che ne compongono la logica di business, ossia autenticazione, registry, scheduling, clinical, payments ed events.

Il **Capitolo 11** si concentra sul modulo AI Assistant, descrivendone finalità, vincoli, flusso di autenticazione, gestione degli input, controlli di sicurezza, inferenza locale e natura stateless. Il **Capitolo 12** affronta il modello dei dati e la strategia di persistenza, illustrando la suddivisione in schemi di dominio e le relazioni logiche tra i diversi database.

Il **Capitolo 13** approfondisce i temi di sicurezza, privacy e controllo degli accessi, mentre il **Capitolo 14** descrive l'organizzazione delle interfacce REST e le convenzioni adottate per gli endpoint e per la gestione delle risposte. Il **Capitolo 15** tratta gli aspetti di deploy, configurazione ed esecuzione dell'intero sistema, con attenzione ai prerequisiti, ai servizi containerizzati, alle variabili d'ambiente e alle dipendenze infrastrutturali.

Il **Capitolo 16** presenta gli scenari di validazione funzionale, esaminando casi d'uso end-to-end per i diversi ruoli e verificando i principali controlli di accesso e di disponibilità dei servizi. Il **Capitolo 17** descrive la struttura della repository e la corrispondenza tra organizzazione del codice e architettura implementata. Il **Capitolo 18**, infine, raccoglie le considerazioni finali sulla soluzione realizzata, sui suoi punti di forza e sulle sue possibili evoluzioni.

2. VISIONE GENERALE DEL SISTEMA

2.1. PANORAMICA DELLA SOLUZIONE

La piattaforma è concepita come un sistema web integrato per la gestione coordinata dei principali processi amministrativi, organizzativi e clinici di una struttura sanitaria privata. La soluzione adotta un modello **multi-ruolo** e si fonda su una chiara distinzione tra i soggetti che interagiscono con il sistema, le responsabilità operative a essi attribuite e i domini applicativi coinvolti nei diversi flussi.

Dal punto di vista funzionale, la piattaforma consente di governare l'intero ciclo operativo che accompagna l'erogazione di una prestazione sanitaria: accesso al sistema, gestione dell'identità e dei profili, consultazione del catalogo delle prestazioni, verifica delle disponibilità, prenotazione degli appuntamenti, eventuale ripianificazione o annullamento, check-in amministrativo, compilazione del pre-triage, presa in carico clinica, registrazione delle informazioni sanitarie rilevanti, produzione del referto, gestione dei pagamenti, invio di notifiche e tracciamento delle operazioni sensibili.

La soluzione non è costruita come un insieme indistinto di pagine o di endpoint isolati, ma come un sistema nel quale ogni funzionalità si inserisce in una **catena di processo** coerente. Un appuntamento, ad esempio, non costituisce soltanto un record schedulato nel calendario, bensì un punto di raccordo tra esigenze di disponibilità, attività amministrative, verifiche di accesso, compilazione del pre-triage, apertura dell'encounter clinico, produzione di documentazione sanitaria ed eventuale gestione economica. Questa impostazione permette al sistema di mantenere continuità tra i diversi momenti del servizio, evitando frammentazioni operative e riducendo il rischio di inconsistenze.

Dal punto di vista dell'esperienza utente, la piattaforma propone interfacce distinte per i diversi profili applicativi. Ciascun attore accede a un insieme di pagine, operazioni e dati coerente con il proprio ruolo. Tale scelta risponde a un'esigenza duplice: da un lato migliorare la chiarezza d'uso e la specializzazione delle interazioni, dall'altro garantire che le funzionalità disponibili siano effettivamente compatibili con il perimetro di responsabilità del soggetto autenticato. Il sistema non adotta quindi una visione uniforme e indifferenziata dell'utente, ma una visione **contestuale**, nella quale identità, ruolo e autorizzazioni concorrono a determinare ciò che può essere consultato o modificato.

Sul piano architetturale, la soluzione è organizzata come un insieme di componenti distinti ma integrati. Il frontend web, servito tramite Nginx, rappresenta il punto di contatto con l'utente e convoglia il traffico applicativo verso un **API Gateway** dedicato. Quest'ultimo costituisce il punto di ingresso unico per le richieste API e si occupa delle responsabilità trasversali di instradamento, verifica dei token, gestione del contesto di richiesta, controllo del traffico e aggregazione dello stato di salute dei servizi. Il nucleo della logica applicativa risiede nel **Core Service**, che governa i domini principali del sistema: autenticazione, anagrafiche, scheduling, clinical, pagamenti ed eventi. Accanto a esso opera un **AI Assistant** separato, dedicato al supporto consultivo del personale clinico. La persistenza è centralizzata in un'istanza MySQL organizzata secondo una strategia *schema-per-domain*, mentre l'intero stack viene distribuito e orchestrato tramite Docker Compose.

Questa impostazione consente di soddisfare esigenze diverse ma complementari. La presenza di ruoli distinti e di flussi operativi strutturati richiede infatti un backend in grado

di esprimere chiaramente il confine tra i domini di business; la necessità di proteggere le informazioni personali e cliniche richiede un punto di accesso controllato e verificabile; l'introduzione di un modulo AI impone l'isolamento di una componente con finalità specifiche e con vincoli di sicurezza propri; la necessità di eseguire il sistema in modo coerente e riproducibile richiede una definizione esplicita dei servizi runtime e delle loro dipendenze.

La visione d'insieme che emerge è quella di una piattaforma orientata alla **governance dei processi**, non alla sola esposizione di funzionalità puntuali. Ogni componente contribuisce a un'architettura nella quale i flussi sono definiti, i ruoli sono espliciti, i dati sono governati secondo responsabilità chiare e le interazioni tra gli elementi del sistema avvengono entro confini applicativi ben stabiliti.

2.2. ATTORI APPLICATIVI

La piattaforma è progettata attorno a quattro attori applicativi principali: **Admin**, **Clinician**, **Patient** e **Delegate**. Ciascuno di essi rappresenta un profilo operativo distinto, caratterizzato da specifiche finalità d'uso, responsabilità funzionali, vincoli di accesso e visibilità sui dati. La definizione esplicita di tali attori costituisce un elemento centrale del modello del sistema, poiché permette di allineare interfacce, permessi, flussi e controlli di sicurezza con le reali esigenze operative.

L'individuazione di attori distinti non risponde soltanto a un'esigenza organizzativa, ma rappresenta una scelta progettuale fondamentale per la corretta gestione della piattaforma. In un contesto in cui coesistono attività amministrative, attività cliniche e operazioni self-service dell'utente finale, la chiarezza dei ruoli è infatti una condizione necessaria per garantire **ordine operativo**, **separazione dei compiti**, **protezione delle informazioni** e **responsabilità coerenti**.

2.2.1. ADMIN

L'**Admin** rappresenta il profilo responsabile della gestione operativa e amministrativa della piattaforma. Questo attore dispone di strumenti dedicati alla supervisione delle anagrafiche, alla gestione dei profili di sistema, all'amministrazione delle deleghe, al controllo delle prenotazioni, alla configurazione delle disponibilità, alla registrazione del check-in, alla gestione dei pagamenti e alla consultazione dei log di audit.

Il suo ruolo è strettamente connesso alla continuità organizzativa della struttura. L'Admin interviene nei momenti in cui è necessario assicurare il corretto instradamento del paziente all'interno del processo: verifica la presenza di dati anagrafici coerenti, supporta o governa il ciclo di prenotazione, monitora lo stato degli appuntamenti, gestisce gli eventi amministrativi rilevanti e presidia i flussi economici. In tale prospettiva, l'Admin opera come soggetto abilitato alla conduzione delle attività di coordinamento e supporto, non come attore clinico.

La sua interazione con il sistema è caratterizzata da una forte componente di **controllo operativo**. Ciò include la possibilità di intervenire su entità sensibili, come appuntamenti, profili o stati di pagamento, nonché la necessità di disporre di viste aggregate e strumenti di consultazione che consentano di monitorare l'andamento delle operazioni. Proprio per la delicatezza di tali capacità, il sistema attribuisce all'Admin un insieme di privilegi ampio sul piano amministrativo, ma delimitato rispetto all'ambito clinico.

2.2.2. CLINICIAN

Il **Clinician** rappresenta il profilo sanitario che utilizza la piattaforma per la gestione del percorso clinico del paziente. Il suo dominio operativo è strettamente legato alla presa in carico del paziente nel contesto dell'appuntamento assegnato e comprende la consultazione della propria agenda, l'accesso all'encounter, la registrazione di anamnesi e parametri vitali, la gestione di ordini ed esecuzioni e la redazione del referto.

Questo attore opera entro un perimetro fortemente specializzato, in cui la centralità è attribuita alla **documentazione clinica** e alla **continuità della presa in carico**. L'accesso alle informazioni non avviene in modo indiscriminato, ma in funzione dell'effettiva relazione operativa tra clinico, appuntamento ed episodio assistenziale. Il Clinician non è quindi un utente con visibilità globale sul sistema, ma un attore la cui operatività è vincolata al proprio ambito professionale e alle proprie assegnazioni.

La piattaforma mette inoltre a disposizione del Clinician un modulo di **AI Assistant** consultivo, progettato come strumento di supporto informativo. Tale componente può essere utilizzata nell'ambito di richieste di *question answering* e di elaborazione di contenuti testuali, ma non modifica il fatto che la responsabilità clinica rimane integralmente in capo al professionista. Il sistema, sotto questo profilo, riconosce il Clinician come unico attore abilitato all'uso della componente AI, proprio in ragione della natura specialistica del servizio.

2.2.3. PATIENT

Il **Patient** rappresenta il soggetto destinatario della prestazione sanitaria e costituisce il principale utente finale della piattaforma. Attraverso il sistema, il paziente può gestire il proprio account, aggiornare il profilo, consultare il catalogo delle prestazioni, verificare le disponibilità, effettuare prenotazioni, ripianificare o annullare appuntamenti, compilare il pre-triage, consultare la documentazione resa disponibile, monitorare i pagamenti e ricevere notifiche.

La visione del Patient è orientata al **self-service controllato**. La piattaforma gli consente infatti di eseguire in autonomia un insieme significativo di operazioni che, in un modello tradizionale, richiederebbero l'intervento diretto del personale amministrativo. Ciò non significa, tuttavia, che il paziente disponga di pieno controllo sul processo complessivo: le sue capacità restano circoscritte alle operazioni che riguardano il proprio profilo, i propri appuntamenti, i propri pagamenti e la documentazione a lui destinata.

Il Patient costituisce quindi l'attore per il quale la piattaforma realizza la maggiore apertura verso l'esterno, ma entro un perimetro attentamente delimitato. L'utente finale deve poter interagire con il sistema in modo semplice ed efficace, ma sempre nel rispetto di regole di autorizzazione, vincoli di coerenza e controlli di identità che garantiscano la protezione dei dati e l'affidabilità dei flussi.

2.2.4. DELEGATE

Il **Delegate** rappresenta il soggetto autorizzato a operare per conto di un assistito, secondo le deleghe attive registrate nel sistema. La presenza di questo attore amplia il modello della piattaforma e ne rafforza l'aderenza a scenari reali nei quali determinate

operazioni possono essere svolte da terzi legittimati, come nel caso di familiari, accompagnatori o soggetti formalmente autorizzati.

Il Delegate dispone di funzionalità simili, per alcuni aspetti, a quelle del Patient, ma il suo perimetro non è originario: esso deriva dalla relazione di delega e dai relativi vincoli. L'accesso alle informazioni e alle operazioni non è mai assoluto, bensì subordinato alla validità della delega, all'identificazione dell'assistito di riferimento e alle specifiche autorizzazioni applicabili al contesto. Questo modello consente al sistema di distinguere con precisione tra titolarità del dato e legittimazione operativa.

Dal punto di vista della piattaforma, il Delegate non è un semplice duplicato del paziente, ma un attore autonomo con una semantica distinta. Egli opera infatti in rappresentanza di un altro soggetto e, proprio per questo, richiede controlli aggiuntivi di tracciabilità, correlazione e limitazione dei privilegi. La presenza di questo ruolo contribuisce ad aumentare la complessità del sistema sul piano autorizzativo, ma rende la soluzione più completa e più aderente a esigenze operative articolate.

2.3. RESPONSABILITA' OPERATIVE PER RUOLO

Le responsabilità operative assegnate ai diversi ruoli sono definite in modo da garantire una distribuzione ordinata delle attività lungo l'intero ciclo di vita del servizio. Il sistema non attribuisce i privilegi sulla base di una logica generica di appartenenza, ma costruisce per ciascun attore un insieme coerente di operazioni, dati visibili e azioni consentite. Questa impostazione consente di ridurre sovrapposizioni funzionali, chiarire le competenze e rafforzare la sicurezza dell'intero ecosistema applicativo.

L'**Admin** è responsabile delle operazioni di natura amministrativa e gestionale. In particolare, rientrano nel suo perimetro la gestione delle identità di sistema per alcune categorie di utenti, l'amministrazione di pazienti, delegati e clinici, la consultazione e manutenzione delle anagrafiche, la supervisione del catalogo delle prestazioni, la gestione delle disponibilità e degli slot, il monitoraggio delle prenotazioni, l'esecuzione del check-in, la registrazione dei pagamenti in sede, la riconciliazione degli esiti di pagamento e la consultazione dei log di audit. Le sue responsabilità si collocano quindi sul piano dell'**operatività organizzativa**, della **continuità dei flussi** e del **controllo amministrativo**.

Il **Clinician** è responsabile delle attività strettamente connesse alla presa in carico del paziente in ambito sanitario. Egli consulta la propria agenda, accede agli encounter che rientrano nelle sue assegnazioni, visualizza le informazioni necessarie alla visita, registra anamnesi e parametri vitali, inserisce ordini clinici, documenta eventuali esecuzioni e redige il referto. A ciò si aggiunge la possibilità di interagire con l'assistente AI per finalità consultive. Il suo ruolo è pertanto centrato sulla **gestione clinica dell'episodio assistenziale** e sulla produzione della documentazione sanitaria.

Il **Patient** è responsabile delle operazioni relative alla propria posizione personale e ai propri flussi self-service. Questo comprende la registrazione e gestione dell'account, l'aggiornamento del profilo, la consultazione del catalogo, la ricerca delle disponibilità, la prenotazione di appuntamenti, l'eventuale ripianificazione o annullamento, la compilazione del pre-triage, la consultazione della documentazione pubblicata a suo favore, la verifica dei pagamenti e la ricezione delle comunicazioni di sistema. La sua responsabilità

operativa si esprime quindi nella gestione autonoma e controllata del proprio rapporto con la struttura.

Il **Delegate** è responsabile delle operazioni che il sistema consente di eseguire per conto dell'assistito, entro il perimetro definito dalla delega. Egli può consultare le informazioni pertinenti, operare su appuntamenti riferiti all'assistito, compilare il pre-triage quando previsto e interagire con il sistema nei limiti della rappresentanza riconosciuta. La sua responsabilità operativa è dunque di natura **rappresentativa** e si esercita sempre in modo derivato, mai originario.

La distinzione tra tali responsabilità produce effetti concreti sul comportamento della piattaforma. Un'operazione amministrativa non viene esposta a un soggetto clinico; una funzionalità clinica non viene resa disponibile a un profilo paziente; una vista destinata al delegato non coincide con quella del titolare del dato; un controllo di audit viene attribuito a soggetti specificamente autorizzati. La progettazione per ruolo diventa così un criterio ordinatore dell'intero sistema, capace di collegare in maniera coerente requisiti funzionali, modello dei dati, design delle interfacce e politiche di autorizzazione.

2.4. SEPARAZIONE DEI COMPITI E VINCOLI DI ACCESSO

La separazione dei compiti costituisce uno dei principi fondamentali su cui si fonda la piattaforma. In un sistema che gestisce contemporaneamente identità, dati amministrativi, informazioni cliniche, pagamenti, deleghe e audit log, non è sufficiente identificare l'utente autenticato: è necessario stabilire con precisione *quale tipo di attività* egli possa svolgere, *su quali dati* e *in quale contesto operativo*. La piattaforma implementa questa esigenza attraverso un modello di accesso basato su **ruoli**, **contesto applicativo** e **vincoli di relazione** tra utente, risorsa e processo.

La prima dimensione di tale separazione riguarda il confine tra **ambito amministrativo** e **ambito clinico**. L'Admin deve poter governare i processi organizzativi, ma non deve operare come soggetto titolare del contenuto clinico. Il Clinician deve poter documentare e gestire l'episodio sanitario, ma non deve acquisire la possibilità di svolgere operazioni economico-amministrative o di supervisione generale che esulino dal proprio ruolo. Questa distinzione non è soltanto una scelta di buona organizzazione, ma una misura di controllo indispensabile per preservare la correttezza operativa e limitare accessi impropri a dati e funzioni sensibili.

La seconda dimensione riguarda il rapporto tra **titolarità del dato** e **legittimazione all'azione**. Il Patient agisce sul proprio profilo e sui propri flussi; il Delegate opera invece in forza di una delega valida e nel solo ambito riconosciuto dall'applicazione. La piattaforma non tratta quindi la rappresentanza come una semplice estensione dei privilegi del paziente, ma come una relazione esplicita da verificare e tracciare. Ogni azione del Delegate è subordinata alla sussistenza del collegamento corretto con l'assistito di riferimento, il che rafforza sia la trasparenza operativa sia il controllo sulle operazioni effettuate da terzi.

La terza dimensione riguarda il **contesto di processo**. In particolare, l'accesso del Clinician non è definito in termini generici di appartenenza alla categoria professionale, ma viene ulteriormente vincolato all'esistenza di un appuntamento pertinente, di una presa in carico coerente e di un encounter effettivamente riferibile al soggetto autenticato. Questo

approccio evita che la semplice disponibilità del ruolo sanitario si traduca in una visibilità indiscriminata sui dati clinici del sistema.

I vincoli di accesso si manifestano anche sul piano tecnico. L'autenticazione è gestita tramite token JWT, che permettono al sistema di associare l'identità dell'utente al suo ruolo applicativo. Il gateway verifica tali token per le richieste protette e, nei casi previsti, si appoggia ai meccanismi di introspection del Core Service per determinare la validità e lo stato della sessione. A livello applicativo, i servizi interni applicano poi ulteriori controlli per verificare che l'utente autenticato sia effettivamente autorizzato a operare sulla specifica risorsa richiesta. L'autorizzazione, pertanto, non è affidata a un singolo punto del sistema, ma a una combinazione di controlli distribuiti e coerenti.

La separazione dei compiti si riflette direttamente nella progettazione delle interfacce. Le pagine, le azioni disponibili, le viste di dettaglio e i dati esposti sono differenziati per ruolo. Questo significa che il controllo degli accessi non opera soltanto a livello di backend, ma viene tradotto anche in una precisa **segmentazione dell'esperienza utente**, tale per cui ciascun soggetto interagisce con una porzione del sistema coerente con le proprie funzioni. Ne deriva una maggiore leggibilità operativa, una riduzione della complessità percepita e un rafforzamento della coerenza tra modello autorizzativo e interfaccia applicativa.

Un ulteriore elemento di rilievo è rappresentato dalla **tracciabilità**. La piattaforma non si limita a bloccare gli accessi non autorizzati, ma conserva informazioni sulle operazioni rilevanti tramite audit log e meccanismi di registrazione degli eventi. In questo modo, la separazione dei compiti non si esaurisce in una politica di prevenzione, ma si estende anche a una capacità di verifica ex post, utile per il monitoraggio, la responsabilizzazione degli utenti e il controllo delle operazioni sensibili.

La combinazione tra ruoli distinti, contesti operativi vincolati, relazioni di delega esplicite, controlli applicativi e tracciabilità degli eventi definisce un modello nel quale l'accesso al sistema è sempre subordinato a una logica di **necessità operativa, pertinenza funzionale e coerenza di processo**. Questo principio permea l'intera piattaforma e ne costituisce uno degli elementi più rilevanti sotto il profilo progettuale.

3. REQUISITI DELLA SOLUZIONE

3.1. REQUISITI FUNZIONALI PRINCIPALI

La piattaforma deve supportare in modo organico l'intero ciclo operativo di una struttura sanitaria privata, garantendo continuità tra attività amministrative, interazioni self-service degli utenti finali e gestione del percorso clinico. I requisiti funzionali principali sono quindi definiti in funzione dei processi che il sistema è chiamato a governare e dei diversi attori che vi prendono parte.

Un primo gruppo di requisiti riguarda la **gestione dell'identità e dell'accesso**. Il sistema deve consentire la registrazione degli account per le categorie di utenti previste, l'attivazione dell'utenza tramite flussi controllati, l'autenticazione, il logout, il recupero delle credenziali e la consultazione delle informazioni essenziali relative alla sessione corrente. A tali capacità si affianca la necessità di gestire il profilo utente, permettendo l'aggiornamento dei dati consentiti e la visualizzazione delle informazioni coerenti con il ruolo associato all'account.

Un secondo gruppo di requisiti riguarda la **gestione anagrafica e organizzativa**. La piattaforma deve consentire la creazione, consultazione e manutenzione delle anagrafiche di pazienti, delegati e clinici, nonché la gestione delle relazioni di delega e dei consensi applicativi. Tali funzionalità devono permettere di mantenere coerenti le informazioni di base che costituiscono il presupposto per l'operatività dei flussi successivi, garantendo che i soggetti presenti nel sistema siano correttamente identificati e associati ai rispettivi contesti.

La soluzione deve inoltre supportare la **gestione del catalogo delle prestazioni** e delle informazioni necessarie alla prenotazione. Ciò comprende la possibilità di rendere disponibili le prestazioni erogabili, di esporne la descrizione essenziale nel frontend e di collegarle ai meccanismi di disponibilità e scheduling. Il catalogo non rappresenta soltanto una lista informativa, ma costituisce un elemento funzionale al processo di prenotazione e alla definizione del contesto clinico-amministrativo dell'appuntamento.

Tra i requisiti centrali rientra la **gestione delle disponibilità e degli appuntamenti**. Il sistema deve consentire la configurazione e la consultazione degli slot disponibili, la prenotazione di una prestazione in una determinata fascia temporale, la ripianificazione e l'annullamento dell'appuntamento, nonché il monitoraggio dello stato dell'evento prenotato. Le funzionalità di scheduling devono essere coerenti con il ruolo dell'utente che le utilizza e con le regole che governano la disponibilità effettiva delle risorse cliniche e organizzative.

Sul piano operativo, la piattaforma deve supportare le attività di **check-in** e di **presa in carico**. La registrazione dell'arrivo del paziente costituisce infatti un passaggio essenziale per raccordare il flusso di prenotazione con il successivo avvio del percorso clinico. Tale passaggio deve potersi collocare correttamente all'interno della sequenza degli stati dell'appuntamento, in modo da riflettere il reale avanzamento del processo.

Per quanto concerne il dominio sanitario, il sistema deve soddisfare requisiti legati alla **gestione dell'encounter clinico**. Deve quindi essere possibile accedere all'episodio assistenziale nel contesto corretto, consultare le informazioni pertinenti, registrare

anamnesi e parametri vitali, inserire ordini clinici, documentare eventuali esecuzioni e redigere il referto. La piattaforma deve inoltre supportare il workflow del referto nelle sue principali fasi applicative, includendo la produzione, la firma e la pubblicazione del documento nei limiti previsti dal flusso.

La soluzione deve includere anche il **pre-triage**, quale funzionalità preliminare di raccolta di informazioni da parte del paziente o del delegato, da rendere disponibile in coerenza con l'appuntamento di riferimento. Questo requisito ha una duplice valenza: da un lato migliora la continuità informativa del processo, dall'altro permette di predisporre dati utili al successivo percorso clinico.

Tra i requisiti funzionali principali rientra altresì la **gestione dei pagamenti**. Il sistema deve consentire sia il pagamento effettuato tramite interazione applicativa sia la registrazione del pagamento in sede, garantendo la rappresentazione degli intenti di pagamento, delle transazioni e del relativo stato. Deve inoltre essere supportata la riconciliazione degli esiti, in modo da collegare gli eventi economici al contesto prenotativo e amministrativo corretto.

La piattaforma deve anche offrire meccanismi di **notifica e comunicazione**. È necessario poter generare notifiche applicative, promemoria e comunicazioni e-mail in base agli eventi rilevanti del sistema, quali registrazione, attivazione account, reset password, variazioni dello stato degli appuntamenti o aggiornamenti connessi ai pagamenti. Tali comunicazioni non rappresentano una funzione accessoria, ma una parte integrante del corretto funzionamento dei flussi operativi.

Infine, la soluzione deve prevedere un **modulo AI consultivo** riservato al ruolo clinico. Tale componente deve permettere l'invio di richieste di *question answering* e, ove previsto, di contenuti allegati da elaborare, restituendo una risposta generata localmente nel rispetto dei vincoli di autenticazione, autorizzazione e sicurezza. Questo requisito si colloca in continuità con il flusso clinico, ma resta confinato a una funzione di supporto informativo e non sostituisce le responsabilità del professionista.

3.2. REQUISITI NON FUNZIONALI

I requisiti non funzionali definiscono le proprietà qualitative che la piattaforma deve garantire affinché i processi applicativi risultino non solo disponibili, ma anche **affidabili, manutenibili, comprensibili e coerenti** con un contesto operativo strutturato. In un sistema che coinvolge più ruoli, più domini applicativi e più servizi runtime, tali requisiti assumono un rilievo pari a quello delle funzionalità vere e proprie.

Un primo requisito non funzionale è la **coerenza operativa**. La soluzione deve presentare comportamenti prevedibili e uniformi, sia sul piano dell'interfaccia utente sia sul piano dei flussi applicativi. Le transizioni di stato, i messaggi di errore, le modalità di accesso alle funzionalità e la visibilità delle informazioni devono seguire regole chiare e non ambigue, così da ridurre l'incertezza dell'utente e limitare il rischio di usi impropri.

Un secondo requisito riguarda l'**usabilità contestuale**. La piattaforma deve offrire interfacce distinte e coerenti con i diversi ruoli, evitando di esporre in modo uniforme funzionalità che appartengono a contesti operativi differenti. L'esperienza d'uso deve quindi essere costruita intorno al principio di pertinenza: ogni attore deve poter interagire

con il sistema attraverso pagine, azioni e dati compatibili con il proprio ruolo e con il proprio scopo operativo.

Particolare rilievo assume anche la **manutenibilità**. La soluzione deve essere organizzata in modo tale da rendere possibile l'evoluzione del codice, l'isolamento delle responsabilità e la comprensione delle dipendenze tra i componenti. Questo requisito incide direttamente sulle scelte architetturali adottate, quali la separazione tra frontend, gateway, core applicativo e modulo AI, nonché sull'organizzazione interna dei layer e dei moduli del servizio principale.

La piattaforma deve inoltre garantire **modularità e contenimento dell'accoppiamento**. Le componenti del sistema devono poter cooperare senza annullare i propri confini funzionali. Ciò significa che le responsabilità trasversali devono essere concentrate nei punti appropriati, che i domini applicativi devono mantenere una propria identità chiara e che la logica di business non deve essere dispersa in modo incontrollato tra i vari livelli della soluzione.

Un ulteriore requisito non funzionale è la **riproducibilità dell'esecuzione**. Il sistema deve poter essere avviato e configurato in modo consistente, riducendo al minimo le differenze tra ambienti e rendendo esplicite le dipendenze applicative e infrastrutturali. La containerizzazione dei servizi e l'orchestrazione tramite Docker Compose rispondono direttamente a questa esigenza, poiché consentono di dichiarare in modo formale i componenti runtime, le relative configurazioni, i volumi e le connessioni interne.

La soluzione deve anche soddisfare requisiti di **osservabilità minima**. In particolare, è necessario che i servizi espongano meccanismi di health check, che le richieste possano essere correlate tramite identificativi appropriati e che gli errori siano trattati con modalità coerenti e interpretabili. Un sistema distribuito su più container, anche quando opera in un contesto limitato, richiede infatti strumenti elementari ma solidi per la diagnosi dei problemi e la verifica della disponibilità dei componenti.

Dal punto di vista della persistenza, la piattaforma deve garantire **coerenza dei dati e chiarezza nell'ownership** informativa. I dati devono essere memorizzati in modo strutturato, con separazione dei domini logici e con accessi disciplinati tra i diversi ambiti applicativi. La strategia *schema-per-domain* risponde a un requisito non funzionale preciso: mantenere una struttura del dato leggibile, isolabile e coerente con i confini del modello applicativo.

Tra i requisiti non funzionali rientra inoltre la **robustezza del comportamento applicativo**. Il sistema deve essere in grado di gestire input non validi, accessi non autorizzati, incoerenze di stato e indisponibilità parziali dei servizi senza compromettere la stabilità complessiva del flusso. La robustezza, in questo contesto, non coincide con la sola assenza di errori, ma con la capacità della piattaforma di reagire in modo controllato a condizioni avverse o inattese.

3.3. REQUISITI DI SICUREZZA

La sicurezza rappresenta un requisito strutturale dell'intera soluzione e deve essere considerata una proprietà nativa della piattaforma, non un livello accessorio aggiunto in un secondo momento. La natura dei dati trattati, la presenza di ruoli differenti e la molteplicità dei flussi operativi impongono una progettazione nella quale autenticazione,

autorizzazione, protezione dei dati e controllo delle interazioni siano affrontati in modo sistematico.

Il primo requisito di sicurezza riguarda l'**autenticazione** degli utenti. Il sistema deve poter identificare in modo affidabile il soggetto che accede alla piattaforma e associare tale identità a un contesto di sessione verificabile. L'uso di token JWT risponde a questa esigenza, permettendo di rappresentare l'identità autenticata e di trasportare le informazioni necessarie alla gestione delle richieste protette. L'autenticazione deve essere integrata con i flussi di attivazione account, reset password e revoca dei token, così da coprire l'intero ciclo di vita dell'accesso.

Un secondo requisito fondamentale concerne l'**autorizzazione**. Non è sufficiente che un utente sia autenticato: è necessario che ogni operazione venga verificata rispetto al ruolo, alla risorsa richiesta e al contesto nel quale l'azione viene eseguita. Il sistema deve quindi applicare controlli di autorizzazione sia a livello di gateway, sia a livello dei servizi interni, garantendo che l'accesso effettivo alle funzionalità avvenga solo quando sussiste una reale legittimazione operativa.

La soluzione deve poi garantire la **separazione tra ambito amministrativo e ambito clinico**. Tale requisito di sicurezza ha una valenza sia funzionale sia organizzativa: un soggetto amministrativo non deve poter operare sulle funzioni cliniche come se fosse un professionista sanitario, così come un soggetto clinico non deve acquisire automaticamente poteri di gestione economica o di supervisione trasversale del sistema. La protezione dei confini tra domini applicativi è quindi una misura di sicurezza a tutti gli effetti.

Particolare rilievo assume la gestione delle **deleghe**. Il Delegate deve poter operare esclusivamente nell'ambito derivato dalla delega valida e nei limiti consentiti dalla relazione con l'assistito. Questo implica che il sistema debba verificare non solo il ruolo del soggetto autenticato, ma anche l'esistenza e la validità del legame che abilita l'azione richiesta. La sicurezza, in questo caso, si fonda sull'intersezione tra identità, ruolo e vincolo relazionale.

La piattaforma deve inoltre assicurare la **protezione delle richieste in ingresso** attraverso controlli trasversali applicati dal gateway. Rientrano in tale ambito la verifica dei token per le richieste protette, la gestione del traffico cross-origin secondo criteri espliciti, il contenimento del traffico eccessivo tramite meccanismi di rate limiting e la gestione controllata degli errori di upstream. Il gateway svolge quindi una funzione di filtro e presidio che contribuisce in modo diretto alla sicurezza complessiva del sistema.

Un ulteriore requisito concerne la **protezione dei dati personali e clinici**. Le informazioni esposte a ciascun utente devono essere ridotte a quelle strettamente pertinenti, evitando visibilità eccessive o non motivate. La sicurezza si realizza dunque anche attraverso il principio di minimizzazione dell'accesso, che limita la superficie informativa accessibile a ciascun attore e riduce il rischio di esposizione impropria di dati sensibili.

La soluzione deve poi garantire la **revocabilità dell'accesso**. Un token precedentemente emesso non deve essere considerato automaticamente valido fino alla sola scadenza temporale: il sistema deve poter verificare che esso non sia stato revocato e che la sessione associata non sia stata invalidata. Questo requisito è particolarmente rilevante

per impedire l'uso improprio di credenziali ancora formalmente attive ma ormai non più autorizzate.

Rientra infine nei requisiti di sicurezza la necessità di adottare misure specifiche per il **modulo AI consultivo**. L'accesso a tale servizio deve essere riservato al solo ruolo clinico, gli input devono essere sottoposti a controlli di sicurezza appropriati e il comportamento del modulo deve rimanere confinato a un uso consultivo. La componente AI non deve introdurre un canale privilegiato che aggiri i normali vincoli autorizzativi del sistema, ma deve integrarsi con essi in modo coerente e controllato.

3.4. REQUISITI DI TRACCIABILITA' E AUDITABILITA'

La piattaforma deve garantire la possibilità di ricostruire, verificare e monitorare le operazioni rilevanti che attraversano il sistema. In un contesto in cui più ruoli agiscono su dati amministrativi, clinici ed economici, la sola esecuzione corretta dei flussi non è sufficiente: è necessario che le azioni sensibili siano anche **tracciabili, attribuibili e consultabili** secondo criteri coerenti.

Un primo requisito è la **tracciabilità delle richieste applicative**. Ogni richiesta rilevante deve poter essere correlata a un contesto univoco, in modo da rendere possibile il collegamento tra eventi generati in punti diversi della catena di elaborazione. L'uso di identificativi di richiesta, propagati tra i componenti, risponde a questa esigenza e permette di rafforzare sia la diagnosi tecnica sia la ricostruzione operativa degli eventi.

Un secondo requisito riguarda l'**auditabilità delle operazioni sensibili**. Il sistema deve essere in grado di registrare eventi significativi relativi, ad esempio, a modifiche di stato, operazioni amministrative su profili o appuntamenti, passaggi rilevanti dei pagamenti, accessi a funzionalità dedicate e azioni che presentano particolare impatto sul comportamento della piattaforma. La registrazione di tali eventi deve essere strutturata, persistente e orientata alla successiva consultazione.

La soluzione deve inoltre permettere l'**attribuzione dell'azione al soggetto che l'ha eseguita**. Un log di audit non è utile se non è in grado di collegare l'operazione a un'identità, a un ruolo e a un contesto applicativo. La tracciabilità deve quindi integrare le informazioni relative al soggetto autenticato, alla tipologia di azione compiuta, al momento in cui essa è stata effettuata e, quando rilevante, alla risorsa coinvolta.

Un ulteriore requisito consiste nella **consultabilità delle informazioni di audit** da parte dei soggetti abilitati. La piattaforma non deve limitarsi a produrre dati di tracciamento per finalità interne, ma deve anche offrire funzionalità coerenti con la loro consultazione e analisi da parte dei profili autorizzati. Questo aspetto contribuisce alla trasparenza operativa del sistema e rafforza la capacità di controllo delle attività amministrative e applicative.

La tracciabilità deve riguardare anche gli **eventi di notifica e comunicazione**. Le notifiche pianificate, emesse o inviate costituiscono infatti eventi operativi rilevanti, poiché incidono sul rapporto tra piattaforma e utente finale. Il sistema deve quindi poter registrare e gestire lo stato delle comunicazioni, in modo da mantenere visibile il collegamento tra evento applicativo originario e notifica prodotta.

Anche il dominio dei pagamenti richiede elevata auditabilità. La rappresentazione degli intenti di pagamento, delle transazioni e dei relativi esiti deve essere strutturata in modo da consentire verifiche puntuali sul workflow economico. La capacità di associare un evento di pagamento all'appuntamento, al soggetto e allo stato corretto costituisce un requisito essenziale per la correttezza del sistema sotto il profilo operativo e documentale.

La presenza di meccanismi di audit risponde inoltre a un'esigenza di **responsabilizzazione**. In una piattaforma con ruoli diversi e privilegi differenziati, sapere che le operazioni vengono registrate in modo consultabile contribuisce a rafforzare l'uso corretto delle funzionalità e a ridurre la probabilità di comportamenti impropri o non coerenti con il perimetro operativo assegnato.

3.5. REQUISITI DI MODULARITA' E PORTABILITA'

La soluzione deve essere progettata in modo da favorire la separazione delle responsabilità, la chiarezza dei confini applicativi e la possibilità di distribuire l'intero sistema in maniera coerente e ripetibile. I requisiti di modularità e portabilità rispondono a questa esigenza e rappresentano un asse portante della qualità complessiva della piattaforma.

Sul piano della modularità, il sistema deve essere suddiviso in **componenti con responsabilità distinte**. Il frontend deve presidiare la presentazione e l'interazione utente, il gateway deve costituire il punto di ingresso unico e governare le responsabilità trasversali, il Core Service deve detenere la logica di business principale, il modulo AI deve rimanere isolato come servizio specialistico, mentre il livello dati deve riflettere in modo chiaro i confini tra i diversi domini applicativi. Questa separazione deve emergere sia nella struttura del repository sia nella topologia di runtime.

Il requisito di modularità si estende anche all'organizzazione interna del servizio principale. La soluzione deve mantenere una distinzione esplicita tra livelli applicativi, evitando che la logica di presentazione, la logica di business, il modello di dominio e i dettagli infrastrutturali si sovrappongano in modo incontrollato. Una chiara articolazione in layer rende il sistema più leggibile, più testabile e più semplice da evolvere.

La piattaforma deve inoltre favorire la **modularità funzionale**, ossia la possibilità di trattare aree come autenticazione, registry, scheduling, clinical, payments ed events come moduli logicamente distinguibili, pur all'interno di un nucleo applicativo coordinato. Questo non implica necessariamente la scomposizione di ogni modulo in un microservizio autonomo, ma richiede che il codice, le responsabilità e i dati siano organizzati in modo coerente con i confini del dominio.

Per quanto riguarda la portabilità, il sistema deve poter essere eseguito in modo consistente senza dipendere da configurazioni manuali opache o da presupposti impliciti difficili da replicare. Tutte le dipendenze rilevanti devono essere descritte in maniera esplicita e dichiarativa, così da rendere l'ambiente di esecuzione il più possibile prevedibile. La containerizzazione dei servizi risponde direttamente a questo requisito, poiché consente di confezionare componenti e dipendenze in unità runtime controllate.

L'orchestrazione deve permettere di avviare l'intero stack applicativo come **insieme coerente di servizi cooperanti**, definendo reti, variabili d'ambiente, volumi, dipendenze e mapping delle porte. La possibilità di ricostruire l'ambiente di esecuzione in modo

standardizzato è essenziale non solo per la distribuzione, ma anche per la manutenzione, la diagnosi e l'allineamento tra i diversi contesti in cui il sistema può essere eseguito.

La portabilità riguarda anche la gestione degli elementi di supporto. Servizi come il sink SMTP locale o la cache del modello AI devono poter essere integrati nel runtime senza introdurre dipendenze ambigue o difficili da riprodurre. L'obiettivo è mantenere un ecosistema di esecuzione sufficientemente completo da riflettere il comportamento della piattaforma, ma al tempo stesso sufficientemente controllato da poter essere ricreato in modo fedele.

3.6. REQUISITI DEL MODULO AI CONSULTATIVO

Il modulo AI consultivo costituisce una componente specialistica della piattaforma e, proprio per la sua natura, richiede requisiti dedicati che ne definiscano finalità, limiti, modalità di accesso e comportamento atteso. La sua presenza non modifica il ruolo centrale del Core Service nella gestione dei processi clinici, ma introduce un servizio complementare orientato al supporto informativo del personale sanitario.

Il primo requisito riguarda la **natura consultiva del servizio**. Il modulo AI deve fornire funzionalità di *question answering* e di supporto informativo, senza assumere il ruolo di motore decisionale autonomo, di sistema diagnostico o di componente prescrittiva. La responsabilità clinica deve rimanere interamente in capo al professionista che utilizza il sistema. L'AI Assistant deve quindi essere progettato come strumento di supporto e non come fonte autoritativa sostitutiva del giudizio umano.

Un secondo requisito fondamentale concerne la **limitazione soggettiva dell'accesso**. Il servizio deve essere utilizzabile esclusivamente dagli utenti autenticati con ruolo **Clinician**. Tale vincolo deve essere verificato in modo rigoroso, integrando i meccanismi di autenticazione già presenti nella piattaforma con un controllo specifico del ruolo e della validità del token. Il modulo AI non deve costituire un canale autonomo sganciato dalle politiche di sicurezza generali del sistema.

La componente deve inoltre supportare la **gestione di input testuali e documentali** secondo modalità coerenti con il contesto di utilizzo. Il servizio deve poter ricevere richieste strutturate, elaborare contenuti forniti direttamente dall'utente e, nei casi previsti, estrarre testo da allegati. La gestione degli input deve però avvenire nel rispetto di controlli applicativi che ne verifichino la conformità e ne riducano i rischi operativi.

Tra i requisiti specifici rientra la necessità di applicare **salvaguardie sui contenuti in ingresso**. Il modulo deve essere in grado di rilevare la presenza di dati identificativi diretti o di contenuti non compatibili con il proprio uso previsto, così da limitare l'elaborazione impropria di informazioni sensibili. Questo requisito è particolarmente importante perché il servizio opera su input forniti dinamicamente e non può quindi presupporre a priori la conformità dei contenuti ricevuti.

Il modulo deve poi soddisfare un requisito di **isolamento architetturale**. L'AI Assistant deve essere implementato come servizio separato rispetto al Core Service, così da contenere l'accoppiamento e mantenere chiaro il confine tra logica clinico-amministrativa e funzionalità di supporto AI. Tale separazione permette anche di circoscrivere più efficacemente i vincoli di sicurezza, le dipendenze tecnologiche e le responsabilità di runtime della componente.

Un ulteriore requisito è il **funzionamento stateless**. Il servizio non deve possedere una propria persistenza locale per i dati di dominio e non deve diventare titolare di uno store applicativo autonomo. Le richieste devono essere elaborate in modo puntuale, senza introdurre una memoria persistente interna che alteri il modello informativo generale della piattaforma. Questo requisito semplifica i confini della componente e riduce il rischio di duplicazione o dispersione dei dati.

Il modulo deve inoltre poter effettuare l'**inferenza locale** tramite modello precaricato e gestito in modo controllato. Questo significa che la generazione delle risposte deve avvenire facendo leva su risorse disponibili nell'ambiente di esecuzione previsto, con caricamento del modello secondo modalità compatibili con il runtime del servizio. La dipendenza dal modello deve essere esplicita e governata, evitando meccanismi opachi che rendano difficile comprendere o riprodurre il comportamento della componente.

Infine, il servizio deve integrarsi con l'architettura generale della piattaforma senza compromettere la coerenza dei flussi e delle politiche di accesso. Il gateway deve poter instradare le richieste verso il modulo AI in modo controllato, il Core Service deve poter fungere da riferimento per la validazione dei token, e il frontend deve esporre la funzionalità solo nel contesto appropriato. Il requisito non riguarda soltanto il corretto funzionamento del modulo in sé, ma anche la sua **integrazione disciplinata** nell'ecosistema complessivo del sistema.

4. FLUSSI OPERATIVI PRINCIPALI

4.1. GESTIONE DELL'ACCESSO E DEL CICLO DI VITA DELL'ACCOUNT

La gestione dell'accesso costituisce il punto di ingresso dell'intero sistema e rappresenta il fondamento su cui si basa la corretta associazione tra **identità digitale**, **ruolo applicativo** e **perimetro operativo**. Il ciclo di vita dell'account non si esaurisce nell'autenticazione iniziale, ma comprende un insieme coordinato di operazioni che permettono la creazione dell'utenza, la sua abilitazione effettiva, la gestione della sessione e il recupero controllato delle credenziali.

Il flusso è progettato in modo differenziato a seconda della categoria di utente. Per gli attori che operano in modalità self-service, il sistema consente la registrazione diretta attraverso interfacce dedicate; per i profili gestiti in modo amministrato, l'abilitazione avviene secondo procedure controllate e coerenti con il ruolo. In entrambi i casi, l'account non è considerato operativo fino al completamento delle verifiche richieste dal flusso di attivazione.

L'intero ciclo di vita è costruito con una logica di **controllo progressivo**: prima viene acquisita l'identità, poi viene validata la possibilità di accesso, successivamente viene generato il contesto di sessione e, in caso di necessità, vengono messi a disposizione meccanismi di recupero delle credenziali. Questo approccio riduce l'ambiguità operativa e garantisce una gestione più rigorosa degli accessi.

4.1.1. REGISTRAZIONE

Il flusso di registrazione consente la creazione iniziale dell'account per le tipologie di utente per cui è prevista un'abilitazione diretta tramite piattaforma. La registrazione ha la funzione di raccogliere le informazioni minime necessarie a identificare il soggetto, creare la relativa utenza applicativa e predisporre il profilo per le successive fasi di attivazione.

Dal punto di vista operativo, l'utente accede alla schermata dedicata, inserisce i dati richiesti e invia la richiesta di registrazione. Il sistema esegue le verifiche di validità sintattica e semantica sugli input, controlla l'eventuale esistenza di identità già presenti e, in assenza di conflitti, crea l'account nello stato iniziale previsto dal ciclo di abilitazione. La registrazione non coincide quindi con l'immediata piena operatività dell'utente, ma con la formalizzazione della sua presenza nel sistema.

La registrazione svolge anche una funzione di strutturazione del profilo. I dati raccolti in questa fase costituiscono infatti la base per i flussi successivi di autenticazione, gestione del profilo, notifiche e associazione alle funzionalità del ruolo. In un sistema multi-ruolo, tale fase è particolarmente rilevante, poiché definisce il primo legame tra soggetto, account e contesto applicativo.

4.1.2. ATTIVAZIONE ACCOUNT

L'attivazione dell'account rappresenta la fase che trasforma una registrazione formalmente acquisita in un'utenza effettivamente utilizzabile. Il sistema distingue in modo esplicito tra account creato e account abilitato, evitando che la sola registrazione comporti un accesso immediato e non controllato alla piattaforma.

Il flusso di attivazione avviene attraverso un meccanismo dedicato che collega l'identità registrata a una prova di possesso o a una conferma applicativa coerente con il processo previsto. In termini operativi, dopo la registrazione il sistema genera le informazioni necessarie all'attivazione e le rende disponibili tramite il canale configurato per tale finalità. L'utente completa poi il processo seguendo il collegamento o la procedura prevista, e il sistema aggiorna lo stato dell'account rendendolo idoneo all'accesso.

Questa fase ha una funzione sia di **abilitazione** sia di **contenimento del rischio**. Da un lato, garantisce che l'utenza sia stata formalmente confermata prima dell'uso; dall'altro, impedisce che account incompleti o non verificati possano essere trattati come pienamente operativi. L'attivazione costituisce dunque un passaggio essenziale per la qualità complessiva del ciclo di identità.

4.1.3. LOGIN E LOGOUT

Il login consente all'utente di aprire una sessione autenticata associata alla propria identità e al proprio ruolo. Una volta inserite le credenziali, il sistema verifica la correttezza delle informazioni, controlla che l'account sia nello stato appropriato e, in caso di esito positivo, genera il contesto di autenticazione che verrà utilizzato nelle richieste successive verso le API protette.

Il flusso di login non ha soltanto lo scopo di concedere l'accesso, ma anche di determinare la **proiezione operativa** dell'utente all'interno della piattaforma. In base al ruolo associato all'account, il sistema indirizza infatti l'utente verso l'area funzionale di competenza, esponendo interfacce, dati e operazioni coerenti con il suo perimetro autorizzativo. L'autenticazione diviene così il punto di raccordo tra identità, autorizzazione e esperienza d'uso.

Il logout costituisce il flusso complementare di chiusura della sessione. Attraverso questa operazione, il sistema invalida il contesto di accesso corrente secondo la strategia prevista, interrompendo la possibilità di utilizzare il token o la sessione associata per successive richieste protette. Il logout non è quindi una semplice uscita dall'interfaccia, ma una vera operazione di chiusura controllata del contesto autenticato.

La presenza di un flusso di logout esplicito assume particolare rilievo in un sistema che tratta informazioni amministrative, cliniche ed economiche. Essa consente infatti di ridurre il rischio di utilizzo improprio di una sessione ancora formalmente valida e si inserisce in modo coerente nel più ampio modello di revocabilità dell'accesso.

4.1.4. RESET PASSWORD

Il reset della password consente all'utente di recuperare la possibilità di accesso al sistema in caso di smarrimento o inutilizzabilità delle credenziali. Il flusso è progettato per preservare la sicurezza dell'identità digitale, evitando che la modifica della password possa avvenire in modo arbitrario o non verificato.

Operativamente, l'utente avvia la procedura dalla schermata dedicata e fornisce l'informazione richiesta per identificare l'account. Il sistema genera quindi un meccanismo temporaneo e controllato di recupero, che viene veicolato tramite il canale previsto. Una volta ricevuto tale elemento, l'utente può completare la procedura impostando una nuova password secondo le regole di validità stabilite.

Il reset password costituisce un flusso delicato, poiché interviene direttamente su un elemento critico della sicurezza dell'account. Per questa ragione, il sistema lo tratta come una sequenza distinta, verificabile e strettamente legata allo stato dell'utenza. La nuova credenziale diviene il nuovo riferimento per le autenticazioni successive, mentre il processo lascia evidenza applicativa coerente con la logica di controllo dell'accesso.

4.2. GESTIONE ANAGRAFICHE, PROFILI, DELEGHE E CONSENSI

La gestione delle anagrafiche e dei profili rappresenta uno dei flussi centrali della piattaforma, poiché fornisce la base informativa su cui si appoggiano tutti i processi successivi. Il sistema deve infatti conoscere con precisione i soggetti che operano o sono rappresentati nell'applicazione, il loro ruolo, le informazioni identificative essenziali e le relazioni che li collegano.

Sul piano operativo, questo flusso consente la creazione, consultazione e manutenzione dei profili di pazienti, delegati e clinici, nonché l'aggiornamento delle informazioni ammesse per il soggetto autenticato. La distinzione tra profilo personale e anagrafica gestita amministrativamente è rilevante: alcune informazioni vengono infatti governate direttamente dall'utente, mentre altre rientrano in processi di amministrazione o supervisione da parte di soggetti autorizzati.

Particolare importanza assume la gestione delle **deleghe**. Il sistema consente di formalizzare il rapporto tra un delegato e l'assistito di riferimento, rendendo possibile l'operatività per rappresentanza entro limiti chiaramente definiti. La delega non viene trattata come una semplice associazione informativa, ma come un vincolo applicativo che incide direttamente sulle capacità operative del soggetto autenticato. Le operazioni disponibili al Delegate dipendono infatti dalla validità della delega e dal contesto in cui essa viene esercitata.

Accanto alle deleghe, il sistema governa anche i **consensi** e le informazioni correlate al corretto uso della piattaforma. Questi elementi non hanno soltanto un valore formale, ma incidono sull'ammissibilità di alcuni passaggi applicativi e sul livello di completezza del profilo. Il flusso di gestione anagrafica deve quindi essere letto come un processo di consolidamento della **qualità del dato**, della **legittimazione operativa** e della **coerenza del contesto utente**.

4.3. GESTIONE DEL CATALOGO PRESTAZIONI

Il catalogo delle prestazioni costituisce il punto di incontro tra descrizione dell'offerta sanitaria e operatività applicativa. Non si tratta di un semplice elenco statico di servizi, ma di un elemento strutturale del sistema, utilizzato come riferimento nei flussi di consultazione, prenotazione e percorso clinico.

Dal punto di vista operativo, il catalogo consente di esporre in modo ordinato le prestazioni rese disponibili dalla piattaforma, corredandole delle informazioni essenziali necessarie alla selezione da parte dell'utente o alla gestione amministrativa da parte dei soggetti autorizzati. Il catalogo funge quindi da base per la scelta della prestazione da prenotare e per la contestualizzazione dell'appuntamento successivo.

Il suo ruolo è duplice. Da un lato, supporta il **front-office digitale**, offrendo a pazienti e delegati una vista comprensibile delle prestazioni disponibili; dall'altro, alimenta i processi

interni del sistema, poiché la prestazione selezionata diventa parte integrante del contesto schedulativo e clinico. La coerenza del catalogo incide direttamente sulla qualità del flusso di prenotazione e sulla leggibilità delle informazioni lungo l'intero ciclo operativo.

4.4. GESTIONE DISPONIBILITA', SLOT E APPUNTAMENTI

La gestione delle disponibilità, degli slot e degli appuntamenti rappresenta il nucleo organizzativo del sistema. Questo flusso mette in relazione il catalogo delle prestazioni, la disponibilità temporale delle risorse cliniche, le scelte effettuate dall'utente e il successivo avanzamento del processo amministrativo e clinico.

La piattaforma governa tale ambito attraverso un modello che distingue tra **disponibilità configurata**, **slot prenotabile** e **appuntamento effettivamente creato**. Questa distinzione è essenziale per evitare ambiguità: la disponibilità esprime il potenziale organizzativo, lo slot identifica una finestra temporale concreta, mentre l'appuntamento rappresenta l'istanza operativa che collega soggetto, prestazione, orario e stato del flusso.

La gestione degli appuntamenti non si limita alla creazione iniziale, ma comprende anche la loro evoluzione nel tempo. Un appuntamento può essere ricercato, prenotato, ripianificato, annullato, sottoposto a check-in e successivamente collegato all'avvio del percorso clinico. Tutte queste transizioni devono essere coerenti con le regole del sistema e con il ruolo dell'utente che le attiva.

4.4.1. RICERCA DISPONIBILITA'

La ricerca delle disponibilità consente a pazienti, delegati o soggetti autorizzati di individuare gli slot prenotabili in funzione della prestazione selezionata e dei criteri temporali applicabili. Il flusso ha la finalità di trasformare l'offerta astratta del catalogo in una possibilità concreta di prenotazione.

L'utente seleziona la prestazione di interesse e, ove previsto, applica i parametri di ricerca disponibili. Il sistema interroga quindi il dominio di scheduling e restituisce gli slot effettivamente prenotabili, filtrati secondo le regole di disponibilità e lo stato corrente del calendario. L'esito della ricerca costituisce la base informativa per il passaggio successivo di prenotazione.

Questo flusso ha un ruolo determinante anche sul piano della qualità percepita del sistema. Una ricerca chiara e coerente riduce il carico operativo sul personale amministrativo, favorisce il self-service e contribuisce a mantenere allineati catalogo, disponibilità e calendari clinici.

4.4.2. PRENOTAZIONE

La prenotazione è il flusso attraverso il quale uno slot disponibile viene trasformato in un appuntamento associato a un soggetto, a una prestazione e a un preciso contesto temporale. L'operazione avviene a valle della ricerca delle disponibilità e costituisce uno dei passaggi più rilevanti dell'intero sistema.

Una volta selezionato lo slot, l'utente o l'operatore autorizzato conferma la richiesta di prenotazione. Il sistema verifica la validità del contesto, l'effettiva disponibilità residua e la coerenza degli elementi coinvolti; in caso di esito positivo, crea l'appuntamento e ne

assegna lo stato iniziale previsto dal workflow. Contestualmente, il sistema può attivare la produzione di eventi collegati, come notifiche o promemoria applicativi.

La prenotazione non produce soltanto un evento di agenda, ma un vero **punto di ancoraggio** per i flussi successivi. L'appuntamento diventa infatti il riferimento per il pre-triage, per il check-in, per il percorso clinico e, ove previsto, per il pagamento. La correttezza di questa fase incide direttamente sulla robustezza di tutti i processi successivi.

4.4.3. RIPIANIFICAZIONE

La ripianificazione consente di modificare la collocazione temporale di un appuntamento già esistente, preservando la continuità del processo senza imporre la cancellazione completa del flusso. Tale operazione risponde all'esigenza di adattare l'organizzazione del calendario a mutate disponibilità, richieste dell'utente o necessità operative.

Dal punto di vista applicativo, la ripianificazione comporta la selezione di una nuova collocazione compatibile e l'aggiornamento dell'appuntamento esistente secondo le regole di stato previste. Il sistema deve garantire che il nuovo slot sia effettivamente disponibile e che la modifica sia ammissibile nel contesto corrente del flusso. Una volta completata, la ripianificazione può generare ulteriori eventi, in particolare comunicazioni e notifiche relative alla variazione intervenuta.

La rilevanza di questo flusso risiede nella sua capacità di mantenere la continuità informativa dell'appuntamento, evitando la frammentazione che deriverebbe da una sequenza di annullamenti e nuove prenotazioni non correlate.

4.4.4. ANNULLAMENTO

L'annullamento consente di chiudere anticipatamente il ciclo di un appuntamento quando esso non deve più avere esecuzione. Il flusso può essere attivato, nei limiti consentiti, da soggetti diversi in funzione del ruolo e del contesto operativo, ma deve sempre rispettare le regole di validità previste dal sistema.

Quando viene richiesta la cancellazione, la piattaforma verifica che l'operazione sia ammissibile rispetto allo stato corrente dell'appuntamento e aggiorna quindi l'istanza interessata secondo la semantica prevista. L'annullamento può inoltre generare effetti collaterali rilevanti, come il rilascio dello slot, l'emissione di notifiche, l'aggiornamento dei flussi correlati e la produzione di eventi di audit.

Questo flusso è particolarmente importante perché interrompe una catena operativa già avviata. Proprio per questo, il sistema deve trattarlo con rigore, evitando che una cancellazione impropria produca incoerenze nei domini di scheduling, notifiche o pagamenti.

4.5. CHECK-IN AMMINISTRATIVO E PRESA IN CARICO

Il check-in rappresenta il passaggio che collega il mondo della prenotazione a quello dell'effettiva presenza del paziente presso la struttura. Si tratta di un flusso di natura prevalentemente amministrativa, ma con impatto diretto sul successivo avvio del percorso clinico.

Operativamente, un soggetto autorizzato registra l'arrivo del paziente per l'appuntamento di riferimento. Il sistema aggiorna lo stato dell'evento prenotato e rende esplicito che il soggetto si trova in una condizione idonea alla successiva presa in carico. Questo passaggio permette di evitare che il percorso clinico venga aperto su appuntamenti non ancora maturi dal punto di vista del processo organizzativo.

La presa in carico costituisce il momento di raccordo tra check-in e attività sanitaria. Una volta che l'appuntamento ha raggiunto lo stato appropriato, il Clinician può operare nel contesto corretto e avviare l'encounter. Il sistema utilizza quindi il check-in non come adempimento formale isolato, ma come **snodo di validazione del flusso**, essenziale per collegare l'organizzazione amministrativa alla gestione clinica.

4.6. PERCORSO CLINICO

Il percorso clinico rappresenta il flusso attraverso il quale un appuntamento validamente inserito nel processo organizzativo viene trasformato in episodio assistenziale documentato. In questa fase la piattaforma passa dalla gestione amministrativa del servizio alla gestione del contenuto clinico, con tutte le implicazioni di autorizzazione, tracciabilità e responsabilità professionale che ne derivano.

Il percorso è organizzato in modo progressivo. Si parte dall'accesso all'encounter nel contesto appropriato, si prosegue con la registrazione delle informazioni raccolte durante la visita o la prestazione, si gestiscono eventuali ordini ed esecuzioni e si conclude con la produzione del referto. Ogni passaggio arricchisce il contesto clinico e costruisce la documentazione dell'episodio assistenziale.

4.6.1. ACCESSO ALL'ENCOUNTER

L'accesso all'encounter costituisce il punto di avvio del flusso clinico. Il Clinician, partendo dalla propria agenda o dal contesto dell'appuntamento assegnato, accede all'episodio assistenziale corrispondente e ne assume la gestione documentale.

Il sistema non consente un accesso generico e indiscriminato agli encounter, ma lo vincola alla presenza di un contesto corretto: appuntamento pertinente, ruolo appropriato e relazione operativa effettiva con il soggetto autenticato. Una volta verificati tali presupposti, l'encounter può essere aperto o consultato e diventa il contenitore logico delle informazioni cliniche prodotte nel corso del processo.

L'encounter rappresenta quindi l'unità applicativa su cui si innestano anamnesi, parametri, ordini, esecuzioni e referto. La sua corretta apertura è essenziale per garantire continuità documentale e coerenza tra scheduling e clinical domain.

4.6.2. REGISTRAZIONE ANAMNESI

La registrazione dell'anamnesi consente al Clinician di documentare le informazioni cliniche raccolte nel corso dell'interazione con il paziente. Questo flusso ha una funzione fondamentale di contestualizzazione dell'episodio assistenziale, poiché permette di formalizzare gli elementi descrittivi rilevanti per la comprensione del caso trattato.

Dal punto di vista operativo, il professionista inserisce nell'encounter le informazioni anamnestiche secondo la struttura prevista dal sistema. Tali dati diventano parte integrante della documentazione clinica dell'episodio e possono costituire la base per le

successive decisioni e registrazioni applicative. L'anamnesi non è quindi un contenuto separato, ma un elemento costitutivo del flusso clinico.

4.6.3. REGISTRAZIONE PARAMETRI VITALI

La registrazione dei parametri vitali permette di documentare valori e rilevazioni associati al percorso clinico del paziente nel contesto dell'encounter. Questo flusso ha lo scopo di integrare la documentazione testuale con informazioni misurabili utili alla descrizione dello stato osservato o del contesto della visita.

Operativamente, il Clinician inserisce i parametri disponibili attraverso le interfacce dedicate e il sistema li associa all'episodio assistenziale corretto. La loro memorizzazione strutturata consente di mantenere separate, ma coordinate, le diverse componenti del contenuto clinico: elementi descrittivi, valori rilevati, ordini, esecuzioni e referto.

4.6.4. GESTIONE ORDINI CLINICI

La gestione degli ordini clinici consente di formalizzare attività, richieste o indicazioni generate nell'ambito del percorso sanitario. Gli ordini costituiscono un elemento di raccordo tra la valutazione clinica e le eventuali azioni conseguenti documentate nel sistema.

Quando il Clinician inserisce un ordine, il sistema lo registra nel contesto dell'encounter e lo rende parte del quadro operativo dell'episodio assistenziale. L'ordine diventa così un'entità tracciabile e consultabile, utile sia per la continuità del processo sia per la coerenza della documentazione prodotta.

Questo flusso contribuisce a strutturare il percorso clinico, evitando che le decisioni o indicazioni del professionista restino implicite o disperse in note prive di collocazione applicativa.

4.6.5. REGISTRAZIONE ESECUZIONI

La registrazione delle esecuzioni consente di documentare le attività effettivamente svolte nell'ambito dell'encounter, in particolare quelle connesse a ordini o passaggi operativi rilevanti del processo clinico. Si tratta di un flusso che rafforza la capacità del sistema di rappresentare non soltanto ciò che è stato previsto, ma anche ciò che è stato concretamente eseguito.

Il Clinician inserisce le informazioni pertinenti e il sistema le collega all'episodio assistenziale, mantenendo così coerenza tra intenzione operativa e azione documentata. La distinzione tra ordine ed esecuzione è importante, poiché consente di descrivere il percorso clinico con maggiore precisione e di evitare ambiguità tra pianificazione e svolgimento effettivo delle attività.

4.6.6. REDAZIONE, FIRMA E PUBBLICAZIONE DEL REFERTO

La redazione del referto rappresenta la fase conclusiva del percorso clinico documentato nel sistema. In questa fase, il Clinician sintetizza e formalizza gli esiti dell'episodio assistenziale, producendo il documento che costituisce l'output principale del flusso sanitario.

Il sistema supporta un workflow che distingue tra produzione del contenuto, eventuale consolidamento mediante firma e successiva pubblicazione del referto. Questa articolazione consente di mantenere separati il momento di elaborazione del documento, il momento di conferma da parte del professionista e il momento in cui il referto diventa disponibile nel circuito applicativo previsto.

La pubblicazione segna il passaggio del referto da documento clinico in lavorazione a documento fruibile secondo le regole del sistema, ad esempio per la consultazione da parte del paziente o del delegato autorizzato. Il flusso garantisce così una transizione ordinata tra produzione interna e disponibilità esterna del contenuto.

4.7. PRE-TRIAGE LATO PATIENT E DELEGATE

Il pre-triage costituisce un flusso preliminare attraverso il quale il paziente o il delegato possono fornire informazioni utili prima dell'esecuzione della prestazione o della visita. La piattaforma lo collega a uno specifico appuntamento, evitando che esso venga trattato come raccolta informativa generica e slegata dal contesto.

Dal punto di vista operativo, il soggetto autorizzato accede alla funzionalità dedicata, visualizza il questionario o le informazioni richieste e completa l'inserimento dei dati secondo il formato previsto. Il sistema associa quindi tali contenuti all'appuntamento corretto e li rende disponibili nel contesto del successivo percorso clinico.

Questo flusso migliora la continuità informativa tra front-office digitale e presa in carico sanitaria. Il pre-triage consente infatti di anticipare la raccolta di informazioni, ridurre tempi operativi e predisporre un contesto più completo per l'attività del Clinician, mantenendo al contempo il legame con il soggetto che ha effettivamente fornito i dati.

4.8. GESTIONE DEI PAGAMENTI

La gestione dei pagamenti costituisce il flusso economico della piattaforma e mette in relazione appuntamenti, stato amministrativo e tracciamento delle transazioni. Il sistema è progettato per supportare sia il pagamento effettuato tramite interazione applicativa sia la registrazione del pagamento direttamente in sede, mantenendo una rappresentazione coerente degli eventi economici rilevanti.

Questo flusso non deve essere interpretato come semplice registrazione contabile isolata. Ogni pagamento è infatti collegato a un contesto applicativo preciso: un appuntamento, un soggetto, un eventuale stato di checkout e una sequenza di eventi che ne determinano l'evoluzione. Il dominio dei pagamenti interagisce quindi strettamente con scheduling, notifiche ed eventi amministrativi.

4.8.1. PAGAMENTO IN APP

Il pagamento in app consente all'utente finale di avviare il flusso economico direttamente dalla piattaforma, nel contesto dell'appuntamento o della posizione amministrativa pertinente. Operativamente, il sistema genera l'entità necessaria a rappresentare l'intenzione di pagamento e ne governa lo stato lungo il workflow previsto.

Una volta avviato il processo, il sistema registra gli eventi rilevanti e mantiene il collegamento tra intenzione di pagamento, transazione e appuntamento di riferimento. Pur in presenza di un provider simulato, il flusso è modellato in modo da rappresentare

correttamente le fasi essenziali della gestione economica digitale, incluse la generazione della richiesta, l'aggiornamento dello stato e la consultazione dell'esito.

Questo approccio permette di integrare il pagamento nel processo complessivo della piattaforma, evitando che esso resti un evento esterno non governato dal modello applicativo.

4.8.2. PAGAMENTO IN SEDE

Il pagamento in sede rappresenta il flusso alternativo in cui l'operazione economica viene registrata dall'operatore autorizzato nel contesto fisico della struttura. In questo caso la piattaforma non funge da interfaccia di checkout per l'utente finale, ma da strumento di formalizzazione e tracciamento dell'avvenuto pagamento.

L'operatore registra l'evento economico attraverso le funzionalità amministrative dedicate e il sistema aggiorna il relativo stato in coerenza con l'appuntamento o la posizione amministrativa interessata. Questo flusso è particolarmente importante per mantenere allineati il mondo operativo della struttura e la rappresentazione digitale degli eventi economici, senza imporre l'uso esclusivo di un canale di pagamento applicativo.

4.8.3. RICONCILIAZIONE ED ESITI PROVIDER

La riconciliazione rappresenta il flusso attraverso cui il sistema aggiorna e consolida lo stato dei pagamenti sulla base degli esiti restituiti dal modello di provider utilizzato. Anche in presenza di un provider simulato, la piattaforma tratta questa fase come un passaggio esplicito e tracciabile del processo economico.

Il sistema riceve o genera l'evento di esito, lo associa alla transazione corretta e aggiorna lo stato della posizione economica di riferimento. Questa fase consente di distinguere tra semplice avvio del pagamento e sua effettiva conferma, rafforzando la precisione del modello dei pagamenti e la coerenza con gli eventi successivi, come notifiche, consultazioni amministrative o disponibilità dell'informazione per l'utente.

4.9. NOTIFICHE, PROMEMORIA E AUDIT LOG

Le notifiche e i promemoria costituiscono il flusso comunicativo della piattaforma, mentre l'audit log ne rappresenta il flusso di tracciamento strutturato. Sebbene appartengano a domini differenti, questi elementi condividono una funzione comune: rendere il comportamento del sistema **visibile, documentato e operativamente governabile**.

Le notifiche vengono generate in corrispondenza di eventi applicativi rilevanti, quali registrazione, attivazione account, reset password, prenotazione, ripianificazione, annullamento, aggiornamento dei pagamenti o altri passaggi significativi del ciclo di vita degli oggetti gestiti. Il sistema mantiene tali eventi all'interno di un dominio dedicato, consentendone sia la produzione sia la successiva consegna secondo le regole previste. Quando il canale di destinazione è la posta elettronica, il flusso si appoggia a un meccanismo schedulato di dispatch che interagisce con il sottosistema SMTP configurato.

I promemoria rappresentano una categoria specifica di comunicazione, orientata non tanto alla segnalazione di un evento appena accaduto quanto al supporto temporale dell'operatività futura, come nel caso di appuntamenti imminenti o passaggi che richiedono attenzione da parte dell'utente.

L'audit log, invece, documenta le operazioni sensibili effettuate nel sistema e ne conserva traccia in modo persistente e consultabile. Questo flusso consente di associare a un'azione un soggetto, un contesto, un momento temporale e una risorsa coinvolta, rafforzando la capacità di controllo della piattaforma e la responsabilizzazione degli utenti abilitati.

4.10. CONSULTAZIONE DELL'ASSISTENTE AI DA PARTE DEL CLINICO

La consultazione dell'assistente AI rappresenta un flusso specialistico, accessibile esclusivamente al **Clinician** e progettato come supporto informativo al percorso professionale del soggetto autenticato. Il sistema non tratta questa funzionalità come parte del workflow amministrativo o come servizio generalista, ma come componente mirata e strettamente controllata.

Operativamente, il Clinician accede alla pagina dedicata, formula una richiesta di *question answering* e, ove previsto, può allegare contenuti da sottoporre a elaborazione. La richiesta viene inoltrata attraverso il gateway al servizio AI, il quale verifica la validità del token, controlla il ruolo del soggetto autenticato, applica le salvaguardie previste sugli input e avvia il processo di inferenza locale tramite il modello configurato.

La risposta prodotta viene restituita all'interfaccia nel contesto della sessione corrente e mantiene una natura puramente consultiva. Il flusso è quindi progettato per offrire supporto informativo senza alterare il principio secondo cui la responsabilità clinica resta integralmente in capo al professionista. Anche in presenza di una componente AI, il sistema conserva una chiara gerarchia di ruoli e responsabilità: l'assistente fornisce contenuto di supporto, mentre il Clinician resta l'unico titolare delle decisioni e della documentazione clinica.

5. ARCHITETTURA DEL SISTEMA

5.1. PRINCIPI ARCHITETTURALI ADOTTATI

L'architettura della piattaforma è stata definita secondo un insieme di principi orientati a garantire **chiarezza strutturale, separazione delle responsabilità, controllo degli accessi, coerenza dei flussi e governabilità del ciclo di esecuzione**. In un sistema che integra funzionalità amministrative, cliniche, informative ed economiche, la qualità dell'architettura non si misura soltanto nella capacità di far cooperare più componenti, ma soprattutto nella capacità di mantenere espliciti i confini tra le rispettive responsabilità.

Un primo principio fondamentale è la **separazione dei ruoli architetturali**. Il frontend è responsabile dell'interazione con l'utente e della presentazione delle funzionalità; il gateway costituisce il punto di ingresso unificato per il traffico API e concentra le responsabilità trasversali; il core applicativo governa la logica di business e i principali domini funzionali; il modulo AI resta confinato in un servizio autonomo con finalità consultive; il livello dati è organizzato in modo da riflettere i confini tra i diversi domini. Questa ripartizione riduce il rischio di sovrapposizioni improprie, facilita la comprensione del sistema e rafforza la leggibilità delle dipendenze.

Un secondo principio è la **centralizzazione del controllo d'accesso al perimetro API**. Tutte le richieste provenienti dal frontend transitano attraverso un unico punto di ingresso logico, rappresentato dall'API Gateway. Tale scelta consente di applicare in modo uniforme i controlli sul traffico, la verifica dei token, la gestione del contesto di richiesta, la correlazione tecnica e la protezione degli endpoint. In un'architettura multi-servizio, l'esistenza di un front door esplicito evita che i componenti interni siano esposti direttamente al client e rende più disciplinato il modello di sicurezza.

Un terzo principio riguarda la **specializzazione per dominio**. Il Core Service è articolato internamente in aree funzionali coerenti con i principali contesti applicativi: autenticazione, registry, scheduling, clinical, payments ed events. Questa organizzazione non produce una frammentazione arbitraria del sistema, ma consente di modellare i processi in modo aderente alle responsabilità di business. La separazione per dominio migliora la coerenza del modello, rende più controllabili le dipendenze tra le parti e facilita il mantenimento della logica applicativa.

La piattaforma adotta inoltre il principio di **isolamento delle responsabilità trasversali**. Le funzioni di instradamento, verifica JWT, CORS, rate limiting, request correlation, health aggregation e gestione uniforme degli errori non sono disperse nei singoli moduli di business, ma sono governate dal gateway o da componenti dedicati. Allo stesso modo, l'invio differito delle notifiche e-mail è affidato a un background service specializzato, mentre il modulo AI gestisce al proprio interno autenticazione applicativa, controlli sugli input e inferenza locale. Ciò consente di evitare che la logica di dominio venga contaminata da responsabilità infrastrutturali o trasversali.

Un altro principio architetturale essenziale è la **distinzione tra responsabilità applicative e responsabilità di supporto**. La piattaforma differenzia con chiarezza i servizi che producono valore di business diretto da quelli che forniscono capacità di supporto tecnico. Il frontend, il gateway, il core service e l'AI assistant costituiscono il cuore applicativo del sistema; MySQL, Mailpit e la cache locale del modello AI rappresentano invece servizi di

supporto o dipendenze infrastrutturali. Questa separazione permette di descrivere il sistema in modo più preciso e di chiarire quali elementi siano effettivamente titolari dei processi e quali svolgano funzioni abilitanti.

La soluzione è inoltre guidata dal principio di **ownership esplicita dei dati**. Ogni dominio applicativo deve poter essere ricondotto a uno spazio informativo chiaro e riconoscibile. La scelta di utilizzare un'unica istanza MySQL organizzata secondo una strategia *schema-per-domain* risponde precisamente a questa esigenza: preservare l'unità del livello di persistenza, ma senza rinunciare alla leggibilità dei confini informativi. Tale approccio evita la commistione incontrollata tra dati di natura differente e rende più chiaro il rapporto tra moduli applicativi e informazioni di loro competenza.

Un ulteriore principio è la **riduzione dell'accoppiamento diretto tra client e servizi interni**. Il browser non comunica direttamente né con il Core Service né con l'AI Assistant, ma interagisce esclusivamente con l'origine del frontend. Quest'ultimo, tramite Nginx, inoltra il traffico API verso il gateway, che a sua volta dispatcha le richieste verso i servizi downstream. In questo modo, il sistema mantiene un percorso di comunicazione controllato, evita esposizioni non necessarie e rafforza la coerenza del perimetro applicativo.

La piattaforma incorpora anche il principio di **portabilità dell'esecuzione**. Tutti i componenti runtime sono definiti come servizi containerizzati e orchestrati in modo coordinato tramite Docker Compose. Questa scelta garantisce una maggiore riproducibilità del comportamento del sistema, rende esplicite le dipendenze tra i componenti e contribuisce a mantenere allineati configurazione, networking interno, volumi condivisi e modalità di avvio.

Infine, l'architettura è guidata dal principio di **contenimento funzionale del modulo AI**. L'assistente non è trattato come componente pervasiva, ma come servizio separato, autenticato, stateless e accessibile solo al ruolo clinico. La sua presenza non altera la struttura fondamentale della piattaforma né ridefinisce la proprietà dei processi clinici, che rimangono governati dal Core Service. Questo confinamento architetturale è essenziale per preservare la leggibilità del sistema e per evitare che una componente specialistica introduca dipendenze o responsabilità non coerenti con il modello generale.

5.2. VISTA ARCHITETTURALE COMPLESSIVA

La vista architetturale complessiva della piattaforma può essere letta come una composizione di più livelli cooperanti, organizzati secondo una catena logica che va dalla presentazione verso il dato persistente, attraversando un punto di ingresso unificato e un insieme di servizi applicativi specializzati. Tale struttura è stata progettata per garantire un equilibrio tra **semplicità di fruizione lato client, rigore del controllo lato backend e specializzazione delle responsabilità interne**.

Il livello più esterno è costituito dal **Presentation Tier**, rappresentato dal frontend web servito da Nginx. Questo strato offre un'applicazione web a pagine multiple, con interfacce differenziate per i diversi ruoli applicativi e script dedicati ai relativi flussi. Il frontend non implementa la logica di business del sistema, ma si occupa della resa delle interfacce, della gestione del comportamento lato client e della generazione delle richieste HTTP

verso l'origine applicativa. Nginx, oltre a distribuire gli asset statici, svolge una funzione di proxy per tutte le richieste `/api/*`, inoltrandole al gateway.

Il livello successivo è l'**Integration & Security Tier**, centrato sull'API Gateway sviluppato in Go con framework Gin. Questo componente funge da **front door unico** per il traffico API e concentra le responsabilità di instradamento e controllo che non appartengono ai singoli moduli di business. Il gateway intercetta le richieste provenienti dal frontend, applica la logica di dispatching verso il servizio corretto e governa le principali esigenze trasversali dell'architettura: verifica dei token, CORS, rate limiting, request ID, logging strutturato, gestione uniforme degli errori e health aggregata.

A valle del gateway si colloca l'**Application Services Tier**, che contiene i due servizi applicativi principali: il **Core Service** e l'**AI Assistant**. Il Core Service, sviluppato in ASP.NET Core 8, rappresenta il centro del modello di business e governa i domini applicativi essenziali della piattaforma. Al suo interno sono distinguibili moduli logici dedicati ad autenticazione e identità, registry, scheduling, clinical, payments, events, oltre a componenti di supporto come l'email dispatcher e l'health probe. Il modulo AI, sviluppato con Django e Django REST Framework, è invece un servizio specialistico separato, dedicato all'elaborazione consultiva di richieste di question answering da parte del Clinician.

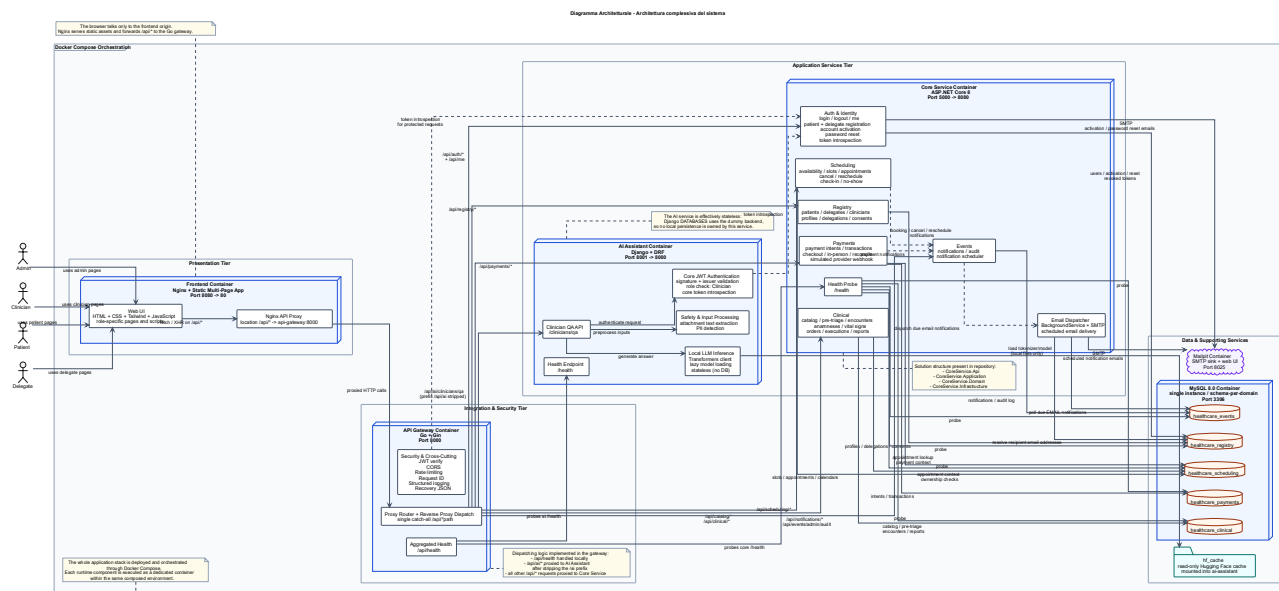
La persistenza e i servizi ausiliari sono organizzati nel livello **Data & Supporting Services**. In questo livello si colloca l'istanza MySQL, configurata come singolo database server ma articolata internamente in più schemi di dominio: `healthcare_registry`, `healthcare_scheduling`, `healthcare_clinical`, `healthcare_payments` e `healthcare_events`. Tale organizzazione consente di mantenere una persistenza centralizzata, ma strutturalmente leggibile. Nello stesso livello rientrano anche Mailpit, utilizzato come sink SMTP locale per i flussi e-mail, e la cartella `hf_cache`, montata come volume in sola lettura nel container AI per il caricamento locale del modello Hugging Face.

L'intera composizione è racchiusa e orchestrata da **Docker Compose**, che costituisce il contenitore logico del deployment di runtime. Docker Compose definisce i servizi, le relative immagini o build, il networking condiviso, i volumi, le porte esposte e le dipendenze necessarie all'esecuzione coordinata del sistema. Questo elemento non è un semplice dettaglio infrastrutturale, ma una parte integrante della vista architetturale complessiva, poiché rende esplicito il modo in cui i componenti vengono effettivamente istanziati e messi in comunicazione tra loro.

Sotto il profilo del flusso logico, la vista complessiva del sistema si sviluppa in questo modo: l'utente interagisce con la Web UI; la Web UI genera richieste HTTP verso il path `/api/*`; Nginx inoltra tali richieste al gateway; il gateway decide se gestire localmente il caso, come nel caso della health aggregata, oppure se inoltrare la richiesta al Core Service o all'AI Assistant; i servizi applicativi elaborano la richiesta, interagiscono con i rispettivi schemi MySQL o con i servizi di supporto e restituiscono la risposta lungo la stessa catena. Questo percorso rende chiara la sequenza di attraversamento del sistema e rafforza il modello di separazione dei confini tra client, controllo d'accesso, logica di business e persistenza.

5.3. DIAGRAMMA ARCHITETTURALE DELLA PIATTAFORMA

Il diagramma architetturale della piattaforma rappresenta la formalizzazione grafica della struttura logica e runtime del sistema. La figura illustra i principali attori, i tier applicativi, i componenti in esecuzione, le responsabilità attribuite a ciascun servizio, i percorsi di comunicazione e la relazione tra i moduli di business e i rispettivi spazi di persistenza.



La rappresentazione si fonda su una suddivisione in quattro aree principali. La prima è il **Presentation Tier**, nel quale il frontend web viene descritto come container Nginx contenente sia la Web UI sia il proxy applicativo verso il gateway. La seconda è l'**Integration & Security Tier**, che contiene l'API Gateway e ne rende esplicite sia la funzione di *Proxy Router + Reverse Proxy Dispatch* sia le responsabilità trasversali associate ai middleware. La terza è l'**Application Services Tier**, che mostra il Core Service e l'AI Assistant come servizi distinti, mettendo in evidenza le rispettive capacità funzionali. La quarta è il livello **Data & Supporting Services**, in cui sono collocati MySQL, Mailpit e la cache locale del modello AI.

Il diagramma esprime in modo particolarmente chiaro il principio secondo cui il **browser non comunica mai direttamente con i servizi di backend**. Gli attori applicativi — Admin, Clinician, Patient e Delegate — interagiscono con la Web UI; la Web UI invia le richieste verso l'origine del frontend; Nginx inoltra il traffico API al gateway; il gateway effettua il dispatching verso il Core Service o verso l'AI Assistant, a seconda del prefisso della rotta e della logica di instradamento applicata. Questa scelta architettonica, resa visivamente esplicita nel diagramma, costituisce uno dei punti chiave dell'intero sistema.

La figura mette inoltre in evidenza l'articolazione logica interna del Core Service, distinguendo i principali moduli applicativi: **Auth & Identity**, **Registry**, **Scheduling**, **Clinical**, **Payments**, **Events**, **Email Dispatcher** e **Health Probe**. Sebbene tali elementi convivano all'interno dello stesso container e dello stesso servizio ASP.NET Core, il diagramma li rappresenta separatamente per sottolineare la loro distinzione funzionale e la differente responsabilità rispetto ai domini di business e alle interazioni con la persistenza.

Particolarmente significativa è anche la rappresentazione del modulo AI. Il diagramma evidenzia che l'AI Assistant espone un endpoint di **Clinician QA API**, applica controlli di autenticazione basati sul token emesso dal Core Service, verifica il ruolo Clinician, esegue preprocessing e controlli sugli input, effettua inferenza locale tramite modello Hugging Face e non possiede un database applicativo di proprietà. La nota inserita accanto al container rende esplicito che il servizio è effettivamente **stateless**, poiché la configurazione Django utilizza un backend database dummy e non introduce persistenza locale.

La parte relativa a MySQL è altrettanto importante, in quanto mostra la scelta di una singola istanza database organizzata in più schemi di dominio. Il diagramma non presenta MySQL come contenitore generico di tutte le informazioni, ma come elemento strutturato in database logici distinti, associati ai diversi domini della piattaforma. Ciò rafforza visivamente il principio di *ownership* dei dati e rende più chiaro il rapporto tra moduli applicativi e schemi persistenti.

5.4. TOPOLOGIA DI RUNTIME

La topologia di runtime descrive il modo in cui i componenti della piattaforma vengono effettivamente istanziati, distribuiti e messi in comunicazione durante l'esecuzione. In questa soluzione, la topologia è basata su un insieme di container coordinati in un ambiente comune tramite Docker Compose, nel quale ogni servizio occupa un ruolo esplicito e dispone di un proprio contesto di processo, pur condividendo il networking necessario alla cooperazione.

Il **Frontend Container** espone il frontend statico tramite Nginx. Dal punto di vista runtime, esso costituisce il punto di esposizione verso il browser e rende disponibili sia gli asset dell'applicazione web sia la regola di proxy che inoltra tutte le richieste `/api/*` all'API Gateway. Questo container rappresenta quindi l'unica origine applicativa visibile al client web.

L'**API Gateway Container** esegue il servizio Go/Gin che riceve le richieste inoltrate dal frontend. La sua topologia è caratterizzata da un endpoint di dispatching catch-all per il traffico API, da una gestione locale della health aggregata e da una connessione logica verso i due servizi downstream: Core Service e AI Assistant. Dal punto di vista del runtime, il gateway svolge una funzione di intermediazione attiva, non di semplice pass-through, poiché applica controlli e logica di trattamento delle richieste prima del proxy vero e proprio.

Il **Core Service Container** ospita il servizio ASP.NET Core 8 che detiene la logica di business principale. Pur essendo un singolo servizio runtime, esso contiene più moduli applicativi interni e interagisce con diversi schemi MySQL in base al dominio funzionale coinvolto. Il container espone il proprio endpoint HTTP agli altri servizi della rete Compose e costituisce il punto di riferimento per autenticazione, token introspection, registry, scheduling, clinical, payments, events, invio e-mail schedulato e health probe.

L'**AI Assistant Container** esegue il servizio Django/DRF specializzato nel question answering per il ruolo clinico. Questo container espone un proprio endpoint HTTP verso la rete interna, riceve traffico esclusivamente dal gateway, accede in sola lettura alla cache locale del modello Hugging Face e non possiede alcuna persistenza applicativa dedicata.

La sua topologia è volutamente contenuta: autenticazione, validazione del ruolo, preprocessing degli input, inferenza locale e risposta.

Nel livello dati, il **MySQL Container** rappresenta il nodo persistente centrale del sistema. Sebbene si tratti di un'unica istanza runtime, esso supporta più schemi logici, permettendo ai diversi domini applicativi di conservare i propri dati in spazi separati ma all'interno dello stesso server. A esso si affianca il **Mailpit Container**, che funge da servizio SMTP locale di supporto per le comunicazioni e-mail emesse dal sistema. Il volume `hf_cache`, infine, non è un servizio attivo, ma una risorsa runtime montata nell'AI Assistant per rendere disponibile il modello locale senza dover dipendere da fetch dinamici non controllati.

Sotto il profilo della rete interna, Docker Compose definisce un ambiente condiviso nel quale i container possono risolversi tramite il nome del servizio e comunicare su porte interne note. Questa impostazione rende la topologia **esplicita e dichiarativa**, poiché ogni servizio conosce i propri interlocutori in base alla configurazione e non tramite presupposti impliciti esterni al runtime. Le dipendenze tra servizi non vengono quindi nascoste nella documentazione o nelle assunzioni operative, ma emergono direttamente dalla topologia di esecuzione.

5.5. COMUNICAZIONI TRA I COMPONENTI

Le comunicazioni tra i componenti seguono un modello rigorosamente controllato e coerente con la separazione dei ruoli architetturali. La piattaforma evita comunicazioni dirette non necessarie tra client e backend e struttura i percorsi di interazione in modo da rendere chiaro il tragitto di ogni richiesta lungo il sistema.

Il primo tratto di comunicazione avviene tra **browser e frontend origin**. L'utente interagisce con la Web UI caricata dal container Nginx e tutte le richieste applicative partono da questo contesto. Quando le operazioni richiedono dati o azioni lato server, il frontend genera chiamate HTTP verso path `/api/*` sulla stessa origine applicativa. Questa scelta riduce la complessità del client, semplifica il modello di deployment e mantiene il traffico coerente con il punto di esposizione del sistema.

Il secondo tratto di comunicazione avviene tra **frontend e API Gateway**. Nginx inoltra tutte le richieste `/api/*` verso il gateway, che diventa il punto di ricezione delle chiamate applicative. In questa fase il gateway non si limita a trasmettere passivamente la richiesta, ma la sottopone a una catena di responsabilità comprendente middleware di sicurezza, controllo e osservabilità. Solo dopo tale trattamento il traffico viene instradato verso il servizio destinatario.

Il terzo tratto di comunicazione è quello tra **API Gateway e servizi downstream**. Il gateway adotta una logica di dispatching che distingue tra health aggregata, traffico destinato all'AI Assistant e traffico destinato al Core Service. Le richieste `/api/health` sono gestite localmente dal gateway mediante interrogazione dei rispettivi endpoint di health dei servizi applicativi; le richieste `/api/ai/*` vengono inoltrate all'AI Assistant dopo l'opportuna trasformazione del path; tutte le altre richieste `/api/*` vengono inoltrate al Core Service. Questa logica mantiene chiara la distribuzione delle responsabilità e preserva l'idea di un solo punto di ingresso esterno.

All'interno della catena di comunicazione, un ruolo importante è svolto dalla **token introspection** verso il Core Service. Sia il gateway, per le richieste protette, sia l'AI

Assistant, per la validazione del token ricevuto, possono appoggiarsi al Core Service per verificare che il token presentato sia valido, coerente e non revocato. Tale meccanismo rafforza la centralità del dominio Auth & Identity e impedisce che i servizi downstream adottino politiche autonome e potenzialmente divergenti di validazione dell'accesso.

Le comunicazioni tra **Core Service** e **MySQL** seguono invece la distribuzione dei domini applicativi. Auth & Identity e Registry utilizzano lo schema `healthcare_registry`; Scheduling utilizza `healthcare_scheduling`; Clinical utilizza principalmente `healthcare_clinical`, ma può interrogare il dominio scheduling per i controlli di contesto; Payments utilizza `healthcare_payments` e, quando necessario, il dominio scheduling per collegare il pagamento all'appuntamento corretto; Events utilizza `healthcare_events`; l'Email Dispatcher accede al dominio eventi per individuare le notifiche da inviare e al dominio registry per risolvere gli indirizzi dei destinatari.

Le comunicazioni tra **Core Service** e **Mailpit** sono limitate ai flussi SMTP. Il dominio Auth & Identity utilizza tale canale per e-mail legate ad attivazione account e reset password; l'Email Dispatcher lo utilizza per la consegna delle notifiche e-mail pianificate. Questi flussi non coinvolgono il browser in modo diretto, ma si inseriscono nella dimensione asincrona e di supporto del sistema.

Infine, le comunicazioni interne dell'**AI Assistant** con la cache locale del modello avvengono tramite accesso a file montati in sola lettura. Non si tratta di traffico di rete, ma di una dipendenza runtime importante, poiché consente all'inferenza di avvenire localmente e in modo coerente con il modello di esecuzione offline definito per il servizio.

5.6. CONFINI APPLICATIVI E OWNERSHIP DEI DATI

L'architettura della piattaforma definisce in modo esplicito i **confini applicativi** tra i diversi servizi e, in parallelo, l'**ownership dei dati** associata ai domini funzionali. Questa chiarezza è fondamentale per evitare sovrapposizioni di responsabilità, ridurre l'accoppiamento tra le componenti e mantenere leggibile la relazione tra processo di business e persistenza informativa.

Il primo confine rilevante è quello tra **frontend** e **backend**. Il frontend non detiene logica di business né persistenza; esso si occupa della presentazione e della gestione del comportamento lato client. Il backend, viceversa, detiene le responsabilità applicative effettive. Questa distinzione impedisce che regole di dominio o verifiche di autorizzazione vengano delegate al browser, preservando così la coerenza e la sicurezza del sistema.

Il secondo confine è quello tra **gateway** e **servizi di business**. Il gateway non è titolare dei domini applicativi né dei dati di business; il suo compito è gestire l'ingresso del traffico, applicare responsabilità trasversali e instradare correttamente le richieste. Il Core Service e l'AI Assistant restano i veri owner delle rispettive capacità applicative. Questa distinzione è essenziale per evitare che il gateway si trasformi in un contenitore improprio di logica di dominio.

Il terzo confine, particolarmente importante, è quello tra **Core Service** e **AI Assistant**. Il Core Service è il proprietario dei processi amministrativi, clinici, di scheduling, di pagamento e di notifica; l'AI Assistant non governa alcun dominio di business della piattaforma, ma offre una capacità specialistica di supporto consultivo. Il modulo AI non possiede dati di dominio, non detiene la verità applicativa sul paziente, sull'appuntamento

o sul referto e non introduce un archivio autonomo. Questa separazione protegge il sistema da una contaminazione semantica tra processo clinico e componente AI.

L'ownership dei dati all'interno del **Core Service** è ulteriormente chiarita dalla strategia *schema-per-domain*. Il dominio **Auth & Identity** possiede utenti, attivazioni, reset password e token revocati; il dominio **Registry** possiede profili, deleghe e consensi; il dominio **Scheduling** possiede slot, calendari e appuntamenti; il dominio **Clinical** possiede catalogo prestazioni, pre-triage, encounter e referti; il dominio **Payments** possiede intenti di pagamento e transazioni; il dominio **Events** possiede notifiche e audit log. Questa distribuzione non elimina le relazioni logiche tra domini, ma le rende più leggibili e controllabili.

Esistono infatti casi di **accesso cross-domain controllato**, in cui un dominio deve consultare dati collocati altrove per mantenere la coerenza del processo. Il dominio Clinical può accedere al contesto scheduling per verificare l'appropriatezza di un encounter rispetto a un appuntamento; il dominio Payments può consultare scheduling per associare correttamente il pagamento alla prenotazione di riferimento; l>Email Dispatcher può interrogare events e registry per completare la consegna delle comunicazioni. Tali accessi, tuttavia, non annullano l'ownership del dato: la consultazione avviene in funzione del processo, ma la titolarità semantica della risorsa resta nel dominio che la governa.

La definizione dei confini applicativi incide anche sulla **leggibilità operativa** della piattaforma. Sapere quale servizio è responsabile di una certa funzionalità, quale schema è owner di un certo insieme di dati e quale componente applica un determinato controllo rende il sistema più governabile sia in fase di sviluppo sia in fase di manutenzione. In un'architettura multi-servizio, questa chiarezza non è soltanto desiderabile, ma necessaria per evitare che il sistema evolva verso una struttura opaca e difficile da presidiare.

Per la redazione dei capitoli più strettamente legati alla struttura del codice sorgente, i file caricati in precedenza non risultano più accessibili dal sistema; quando vorrai procedere con i capitoli più implementativi, sarà utile ricaricare nuovamente lo ZIP aggiornato del progetto.

6. TECNOLOGIE ADOTTATE E MOTIVAZIONI PROGETTUALI

6.1. FRONTEND: HTML, CSS, TAILWIND CSS E JAVASCRIPT

Il frontend della piattaforma è stato realizzato mediante una combinazione di **HTML**, **CSS**, **Tailwind CSS** e **JavaScript**, adottando un modello applicativo di tipo **multi-page web application**. Tale scelta risponde a un'esigenza di chiarezza architetturale e di coerenza con la natura del sistema: la piattaforma non richiede infatti un'interfaccia orientata a navigazioni fortemente dinamiche tipiche di una single-page application, ma una struttura composta da pagine distinte, ciascuna dedicata a uno specifico contesto operativo e a un determinato ruolo applicativo.

L'uso di **HTML** consente di definire una struttura semantica chiara delle interfacce, favorendo leggibilità, manutenibilità e separazione tra contenuto e comportamento. In un sistema con più ruoli e numerose viste dedicate, la chiarezza del markup rappresenta un elemento importante non soltanto per l'ordine del codice, ma anche per la prevedibilità del comportamento dell'interfaccia.

Il **CSS** tradizionale è impiegato per la definizione degli stili condivisi e per la gestione degli aspetti di presentazione non opportunamente espressi tramite utility classes. Accanto a esso, l'adozione di **Tailwind CSS** introduce un approccio dichiarativo e modulare alla costruzione dell'interfaccia, particolarmente adatto a una piattaforma con numerose schermate, layout ricorrenti, componenti visuali simili e necessità di mantenere coerenza stilistica tra sezioni distinte dell'applicazione. Tailwind permette infatti di costruire rapidamente interfacce consistenti, riducendo la dispersione degli stili e favorendo una maggiore uniformità del design.

Sul piano del comportamento lato client, **JavaScript** svolge il ruolo di strumento principale per la gestione dell'interazione con il backend, della manipolazione dinamica del DOM, della validazione preliminare degli input, della gestione della sessione lato browser e dell'aggiornamento delle viste in risposta agli esiti delle chiamate API. La scelta di JavaScript "vanilla", senza l'introduzione di framework frontend complessi, risponde a una precisa motivazione progettuale: mantenere il frontend **leggero, trasparente e** strettamente aderente alla natura multi-page dell'applicazione, evitando un incremento di complessità non necessario rispetto agli obiettivi della piattaforma.

Questa combinazione tecnologica offre alcuni vantaggi rilevanti. In primo luogo, consente una netta separazione tra struttura, stile e comportamento, senza rinunciare alla rapidità di sviluppo. In secondo luogo, rende il frontend facilmente distribuibile come insieme di asset statici, serviti da Nginx senza richiedere un runtime applicativo dedicato lato presentazione. In terzo luogo, si integra in modo naturale con un backend esposto tramite API REST e con un gateway che funge da punto di ingresso unificato.

La scelta di una **multi-page application** anziché di una SPA presenta inoltre una valenza funzionale e organizzativa. Ogni ruolo dispone di pagine e script specifici, coerenti con il proprio dominio operativo; ciò rafforza la segmentazione delle interfacce, semplifica l'isolamento delle funzionalità e riduce il rischio di sovrapporre in modo artificiale contesti d'uso profondamente differenti. In una piattaforma che distingue in modo netto tra area amministrativa, area clinica, area paziente e area delegato, questo approccio risulta particolarmente adatto.

Un ulteriore elemento a favore di questa scelta è rappresentato dalla **facilità di deploy**. Il frontend può essere pubblicato come contenuto statico all'interno del container Nginx, mantenendo una catena di esecuzione semplice e prevedibile. Nginx non funge soltanto da web server per gli asset statici, ma anche da proxy iniziale verso il gateway, consentendo al browser di interagire sempre con una sola origine applicativa. Questa soluzione migliora la coerenza del runtime e semplifica il modello di esposizione del sistema.

6.2. API GATEWAY: GO E GIN

L'API Gateway è stato sviluppato in **Go**, utilizzando il framework **Gin**, con l'obiettivo di realizzare un componente dedicato alla gestione del traffico API in ingresso, delle responsabilità trasversali e del dispatching verso i servizi downstream. La scelta di Go risponde a esigenze di **efficienza, semplicità operativa, ridotto overhead di runtime e buona aderenza alla realizzazione di servizi HTTP compatti e affidabili**.

Go è particolarmente adatto alla costruzione di componenti infrastrutturali come un gateway, in quanto offre un modello di concorrenza semplice, tempi di avvio rapidi, binari autocontenuti e una buona capacità di gestire richieste HTTP con un'architettura lineare e leggibile. In una piattaforma che necessita di un front door unico per tutte le API, la presenza di un servizio snello, focalizzato e facilmente distribuibile costituisce un vantaggio significativo.

Il framework **Gin** è stato scelto perché consente di costruire applicazioni HTTP in Go con una sintassi essenziale ma espressiva, offrendo primitive utili per la gestione delle rotte, dei middleware e del contesto di richiesta. Nel caso del gateway, tali capacità risultano centrali: il sistema deve infatti intercettare tutte le richieste `/api/*`, distinguere i casi da gestire localmente da quelli da inoltrare ai servizi downstream, applicare i controlli previsti e mantenere uniforme il trattamento degli errori.

Una motivazione progettuale fondamentale per l'introduzione del gateway consiste nella **centralizzazione delle responsabilità cross-cutting**. Invece di duplicare logiche di verifica JWT, gestione CORS, rate limiting, request ID, logging strutturato e recovery degli errori in più servizi applicativi, la piattaforma le concentra in un componente dedicato, posto a monte del Core Service e dell'AI Assistant. Questa scelta migliora la coerenza del sistema, riduce la duplicazione del codice e rende più leggibile il perimetro di sicurezza e osservabilità.

L'API Gateway non è concepito come un servizio di business, ma come un **componente di integrazione e controllo**. Esso non possiede domini applicativi propri, non è titolare dei dati di business e non introduce logiche decisionali legate ai processi amministrativi o clinici. La sua funzione è quella di ricevere il traffico dal frontend, verificare e qualificare la richiesta, gestire le responsabilità trasversali e inoltrare correttamente il carico verso il Core Service o l'AI Assistant. Tale specializzazione giustifica pienamente la scelta di una tecnologia orientata alla realizzazione di servizi infrastrutturali leggeri ed efficienti.

L'adozione di Go e Gin favorisce anche la **portabilità** del componente. Il gateway può essere compilato in un binario compatto, facilmente containerizzabile e rapido da avviare, riducendo la complessità del deployment. In un'architettura orchestrata tramite Docker

Compose, questa caratteristica contribuisce a mantenere basso il costo operativo del componente e a semplificarne la gestione.

Un ulteriore aspetto rilevante riguarda la **trasparenza del dispatching**. Il gateway implementa una logica di proxy routing che distingue chiaramente tra traffico diretto al Core Service, traffico diretto all'AI Assistant e gestione locale della health aggregata. La combinazione tra Go, Gin e reverse proxy HTTP consente di implementare tale comportamento in modo leggibile e controllato, preservando l'idea di un punto di ingresso unificato e di un instradamento disciplinato verso i servizi interni.

6.3. CORE SERVICE: ASP.NET CORE 8

Il nucleo applicativo della piattaforma è stato realizzato in **ASP.NET Core 8**, adottato come tecnologia principale per la costruzione del servizio che governa i domini di business della soluzione. La scelta di ASP.NET Core 8 risponde a motivazioni di **solidità architetturale, maturità dell'ecosistema, integrazione efficace con pattern multilayer, elevata qualità degli strumenti per la realizzazione di API REST e buon supporto per componenti infrastrutturali e background services**.

ASP.NET Core 8 si presta particolarmente bene alla realizzazione di un servizio centrale che deve coordinare più domini applicativi, gestire autenticazione e autorizzazione, interagire con una base dati relazionale, esporre endpoint REST strutturati e integrare logiche asincrone di supporto, come la schedulazione delle notifiche e l'invio di e-mail. La piattaforma richiede infatti un backend non solo funzionalmente ricco, ma anche **organizzabile in strati e moduli coerenti**, e ASP.NET Core offre un contesto tecnologico particolarmente favorevole a questo tipo di progettazione.

Una motivazione rilevante della scelta è la possibilità di strutturare il servizio secondo un'architettura interna chiara, articolata in layer distinti come **API, Application, Domain e Infrastructure**. Questa organizzazione consente di separare in modo esplicito esposizione degli endpoint, orchestrazione dei casi d'uso, modellazione del dominio e dettagli di persistenza o integrazione tecnica. In un sistema che include authentication, registry, scheduling, clinical, payments ed events, tale separazione contribuisce in modo diretto alla **manutenibilità**, alla **leggibilità** e alla **evolvibilità** della soluzione.

ASP.NET Core 8 offre inoltre un modello molto efficace per la costruzione di **API HTTP robuste e consistenti**. La piattaforma necessita di endpoint ben definiti, validazione degli input, gestione ordinata delle risposte, integrazione con middleware applicativi e controlli coerenti sui flussi autenticati. Il framework risponde bene a queste esigenze, permettendo di costruire un backend strutturato senza compromettere la chiarezza del codice.

Un altro elemento decisivo riguarda la capacità del framework di integrare facilmente **servizi in background**. La piattaforma richiede componenti non puramente request-response, come l'email dispatcher e i meccanismi di polling o scheduling delle notifiche. ASP.NET Core offre strumenti nativi per definire e gestire BackgroundService, rendendo naturale l'integrazione di queste funzionalità nel perimetro del servizio principale senza introdurre complessità eccessiva o dipendenze esterne non necessarie.

Sotto il profilo della sicurezza applicativa, ASP.NET Core 8 consente di implementare in modo rigoroso i flussi di autenticazione, attivazione account, reset password, introspection dei token, gestione dei profili e verifica dei ruoli. Tale capacità è particolarmente importante

in un sistema in cui il Core Service non è soltanto uno dei backend disponibili, ma costituisce il **punto autorevole** per identità, processi applicativi e ownership dei dati di dominio.

La scelta di ASP.NET Core 8 si dimostra inoltre coerente con l'esigenza di realizzare un servizio containerizzabile, scalabile in modo moderato e facilmente orchestrabile in ambiente Docker. Il framework si integra bene con una pipeline di build basata su container e consente di mantenere chiara la distinzione tra codice di business, configurazione di ambiente e modalità di esecuzione.

6.4. AI ASSISTANT: DJANGO, DJANGO REST FRAMEWORK E TRANSFORMERS

Il modulo AI consultivo è stato sviluppato mediante **Django**, **Django REST Framework** e librerie basate su **Transformers**, con l'obiettivo di realizzare un servizio separato, stateless e specializzato nel *question answering* per il ruolo Clinician. La scelta di questo stack risponde all'esigenza di isolare la componente AI dal Core Service, mantenendo una chiara separazione tra gestione dei processi di business e funzionalità di inferenza linguistica.

Django è stato adottato come base del servizio per la sua maturità, per la qualità dell'ecosistema Python e per la semplicità con cui consente di strutturare applicazioni web in modo ordinato anche quando la persistenza locale non è utilizzata. In questo contesto, Django non viene impiegato come framework monolitico completo di tutte le sue capacità tradizionali, ma come base solida per costruire un servizio HTTP specialistico, con attenzione a configurazione, sicurezza applicativa, middleware e organizzazione modulare del codice.

Django REST Framework costituisce il livello naturale per l'esposizione dell'endpoint di question answering. La libreria offre strumenti adatti alla definizione di API REST, alla gestione delle request/response, alla validazione degli input e all'integrazione dei meccanismi di autenticazione e permesso. In una componente che deve accettare input JSON o multipart, eseguire controlli applicativi, elaborare allegati e restituire output strutturati, DRF risponde in modo efficace ai requisiti del servizio.

L'adozione di **Python** e dell'ecosistema **Transformers** è motivata dal fatto che la componente AI richiede un ambiente naturalmente orientato all'elaborazione del linguaggio naturale e all'integrazione di modelli pre-addestrati. Python rappresenta lo standard de facto per questo tipo di workload, grazie alla disponibilità di librerie mature, ampia documentazione, facilità di integrazione con modelli locali e buona espressività nella gestione del preprocessing dei contenuti. Utilizzare Transformers consente di caricare e utilizzare un modello locale in modo coerente con una pipeline di inferenza controllata, senza dover introdurre dipendenze da servizi remoti esterni.

Un aspetto centrale della motivazione progettuale consiste nell'**isolamento funzionale del modulo AI**. La piattaforma non ingloba la capacità di inferenza all'interno del Core Service, ma la colloca in un servizio dedicato, con tecnologie specifiche e finalità circoscritte. Questa scelta evita di contaminare il backend principale con dipendenze e pattern propri del mondo ML/NLP, preserva la chiarezza del dominio applicativo e rende più semplice governare i vincoli specifici del modulo, come il caricamento del modello, il trattamento degli allegati e le salvaguardie sugli input.

L'uso di Django e DRF si integra bene anche con il requisito di **statelessness**. Il servizio AI non possiede un database di dominio e non introduce persistenza locale applicativa. L'elaborazione avviene per richiesta, con validazione del token e del ruolo, preprocessing dell'input, inferenza locale e restituzione della risposta. Questo comportamento risulta pienamente coerente con la natura consultiva del modulo e con la scelta di mantenere il Core Service come unico owner dei dati di business.

Infine, l'adozione di uno stack Python dedicato consente di mantenere separati anche i cicli evolutivi delle componenti. Il modulo AI può essere aggiornato, esteso o rifinito nei propri aspetti di preprocessing e inferenza senza alterare il perimetro tecnologico del Core Service, preservando così una maggiore **indipendenza architetturale**.

6.5. PERSISTENZA DATI: MYSQL 8.0

Per il livello di persistenza è stata adottata **MySQL 8.0**, configurata come singola istanza relazionale organizzata secondo una strategia **schema-per-domain**. Questa scelta è stata guidata da esigenze di **affidabilità, chiarezza del modello dati, semplicità operativa, compatibilità con i framework backend adottati e buon equilibrio tra centralizzazione della persistenza e separazione logica dei domini**.

MySQL 8.0 rappresenta una soluzione matura e consolidata per la persistenza relazionale. In una piattaforma che deve gestire utenti, profili, deleghe, appuntamenti, encounter, referti, pagamenti, notifiche e audit log, l'utilizzo di un database relazionale è particolarmente appropriato, in quanto consente di modellare con precisione entità, relazioni, vincoli e collegamenti tra i diversi elementi del dominio. La natura strutturata dei dati trattati rende infatti preferibile una persistenza che garantisca chiarezza semantica e controllo del modello.

La scelta di utilizzare **una sola istanza MySQL** anziché più server distinti risponde a una motivazione di contenimento della complessità infrastrutturale. La piattaforma richiede una persistenza ben organizzata, ma non necessita della frammentazione operativa che deriverebbe da più database server separati. Al tempo stesso, si è ritenuto importante evitare una base dati unica e indistinta, nella quale tutti i domini confluiscono senza separazione semantica. Per questa ragione è stata adottata la strategia **schema-per-domain**, che mantiene un'infrastruttura dati centralizzata ma distingue con chiarezza gli spazi logici di persistenza.

Gli schemi `healthcare_registry`, `healthcare_scheduling`, `healthcare_clinical`, `healthcare_payments` e `healthcare_events` riflettono i principali domini applicativi del sistema. Questa organizzazione rende immediatamente leggibile la relazione tra dominio e persistenza, rafforzando il principio di **ownership del dato**. La suddivisione non ha soltanto una finalità ordinativa, ma contribuisce anche a contenere l'accoppiamento semantico tra i moduli, facilitando la comprensione del modello e la manutenzione del sistema.

MySQL 8.0 è inoltre ben supportata dal Core Service e si integra in modo naturale con un backend ASP.NET Core strutturato in layer e moduli. L'interoperabilità con l'ecosistema applicativo adottato rende più agevole la gestione delle connessioni, delle entità persistenti, delle operazioni CRUD e delle interrogazioni necessarie ai diversi flussi. Anche

il fatto di poter inizializzare facilmente gli schemi tramite script eseguiti in fase di bootstrap del container rappresenta un vantaggio importante per il deploy riproducibile del sistema.

Dal punto di vista progettuale, la scelta di una persistenza relazionale centralizzata ma organizzata in domini consente di mantenere un buon compromesso tra **coerenza strutturale** e **semplicità di esercizio**. I dati rimangono in un unico livello persistente, ma le responsabilità restano comprensibili; i moduli possono accedere agli schemi necessari senza dissolvere i confini logici; il sistema può essere distribuito in modo relativamente semplice, senza rinunciare alla disciplina del modello.

6.6. CONTAINERIZZAZIONE E ORCHESTRAZIONE: DOCKER E DOCKER COMPOSE

La piattaforma adotta **Docker** per la containerizzazione dei servizi e **Docker Compose** per la loro orchestrazione coordinata. Questa scelta non ha una funzione puramente operativa, ma costituisce una parte essenziale del disegno architetturale del sistema, poiché rende esplicite le modalità di esecuzione, le dipendenze tra i componenti, la configurazione di rete interna e i volumi necessari al funzionamento complessivo.

L'uso di **Docker** consente di impacchettare ciascun servizio con il proprio ambiente di esecuzione, le relative dipendenze e la configurazione necessaria, riducendo la dipendenza da installazioni manuali o da differenze incontrollate tra ambienti. Questa caratteristica è particolarmente importante in una piattaforma che utilizza tecnologie differenti per componenti diversi: frontend statico con Nginx, gateway in Go, core service in ASP.NET Core, AI assistant in Python/Django, MySQL, Mailpit e risorse locali per il modello AI. Senza un meccanismo di containerizzazione, l'allineamento dell'ambiente di esecuzione sarebbe significativamente più complesso.

Docker Compose è stato scelto come strumento di orchestrazione perché consente di descrivere in modo dichiarativo l'intero stack: servizi, porte, dipendenze, network, variabili d'ambiente, volumi e ordine logico di avvio. In un sistema come questo, Compose non si limita a "far partire più container", ma formalizza il runtime architetturale della piattaforma, rendendo chiaro come il frontend raggiunga il gateway, come il gateway comunichi con i servizi downstream, come il Core Service acceda a MySQL e a Mailpit e come l'AI Assistant utilizzi la cache locale del modello.

Una motivazione progettuale rilevante è la **riproducibilità dell'esecuzione**. Definire lo stack tramite Compose consente di ridurre l'ambiguità su quali servizi debbano essere avviati, con quali parametri e in quale configurazione. Ciò rende il sistema più governabile, più trasparente e più semplice da mantenere. La presenza di un ambiente di esecuzione dichiarato è inoltre coerente con l'obiettivo di documentare in modo professionale l'intera soluzione, poiché lega strettamente il progetto software alla sua effettiva modalità di avvio.

La containerizzazione favorisce anche il **contenimento delle responsabilità di runtime**. Ogni componente viene eseguito come unità indipendente, pur all'interno della rete condivisa dell'applicazione. Questo rende più esplicito il ruolo di ciascun servizio, semplifica l'analisi dei confini applicativi e contribuisce a evitare sovrapposizioni improprie tra ambienti o dipendenze.

Un ulteriore vantaggio è la capacità di includere nel runtime non soltanto i servizi di business, ma anche i **servizi di supporto** e le risorse accessorie. Il sink SMTP, il

database relazionale e la cache del modello AI entrano a far parte della topologia di esecuzione in modo ordinato, rendendo l'ambiente più completo e più fedele al comportamento atteso del sistema.

6.7. SERVIZI DI SUPPORTO: MAILPIT E CACHE LOCALE HUGGING FACE

Accanto ai componenti principali, la piattaforma utilizza alcuni **servizi di supporto** che, pur non detenendo logica di business, svolgono un ruolo importante per il corretto funzionamento e la completezza dell'ambiente di esecuzione. Tra questi, assumono particolare rilievo **Mailpit** e la **cache locale Hugging Face** montata nel servizio AI.

Mailpit è adottato come sink SMTP locale e interfaccia di osservazione dei messaggi e-mail generati dal sistema. La sua presenza consente di gestire in modo controllato i flussi di posta relativi ad attivazione account, reset password e notifiche pianificate, senza richiedere l'integrazione con un servizio SMTP esterno reale. Dal punto di vista progettuale, Mailpit rappresenta una soluzione molto efficace perché permette di mantenere il comportamento dell'applicazione coerente con quello di un sistema che invia e-mail, pur evitando la complessità, i vincoli e i rischi operativi derivanti dall'uso immediato di un provider esterno.

L'adozione di Mailpit supporta anche la **verificabilità del comportamento applicativo**. Le e-mail prodotte dal sistema possono essere osservate e ispezionate attraverso un'interfaccia web dedicata, rendendo più trasparente il flusso di notifica e facilitando il controllo del corretto funzionamento dei processi che generano comunicazioni verso l'utente.

La **cache locale Hugging Face**, montata nel container dell'AI Assistant, risponde invece a una motivazione progettuale differente: rendere disponibile il modello di inferenza in modo **locale, controllato e riproducibile**. Il modulo AI è progettato per eseguire inferenza senza dipendere da fetch dinamici remoti o da una disponibilità esterna non governata. Per questa ragione, tokenizer e modello vengono caricati da una directory locale montata come volume in sola lettura.

Questa scelta presenta diversi vantaggi. In primo luogo, rafforza il carattere **stateless ma controllato** del servizio AI: il modulo non possiede persistenza di dominio, ma dispone delle risorse necessarie all'inferenza in un formato esplicitamente definito dal runtime. In secondo luogo, migliora la prevedibilità del comportamento del servizio, riducendo la dipendenza da connettività esterna o da tempi di download non deterministici. In terzo luogo, consente di allineare la documentazione architetturale con l'effettiva modalità di caricamento del modello, rendendo più chiaro il ruolo della componente AI all'interno del sistema.

Mailpit e la cache locale Hugging Face mostrano come il progetto non si limiti a definire i soli servizi di business, ma costruisca un ambiente di esecuzione completo, nel quale anche gli elementi di supporto sono scelti e integrati secondo criteri di **controllo, trasparenza e coerenza architetturale**.

7. PROGETTAZIONE DEL FRONTEND

7.1. ARCHITETTURA DEL FRONTEND MULTI-PAGE

Il frontend della piattaforma è stato progettato secondo un modello **multi-page application** basato su pagine HTML statiche servite da Nginx e arricchite da script JavaScript dedicati ai singoli contesti operativi. Questa scelta architetturale risponde all'esigenza di mantenere una chiara corrispondenza tra **ruolo applicativo, pagina, flusso funzionale e logica client-side**, evitando l'introduzione di un livello di complessità tipico delle single-page application quando non strettamente necessario.

L'architettura multi-page consente di strutturare il frontend come un insieme di viste indipendenti ma coerenti, ciascuna associata a uno scenario d'uso preciso. In questo modello, l'utente naviga tra pagine distinte che rappresentano contesti applicativi ben delimitati: autenticazione, dashboard, prenotazioni, agenda clinica, pagamenti, notifiche, audit e funzionalità AI. Ogni pagina possiede un proprio markup, un proprio insieme di dipendenze lato client e, quando necessario, uno script specifico incaricato di dialogare con le API e aggiornare dinamicamente l'interfaccia.

Dal punto di vista strutturale, il frontend è organizzato in modo da separare chiaramente:

- le **pagine HTML** per ruolo e funzionalità;
- gli **asset stilistici** condivisi;
- gli **script JavaScript comuni**;
- gli **script JavaScript specializzati** per singola area applicativa.

Questa suddivisione riflette una precisa strategia di progettazione: mantenere il frontend il più possibile **esplicito, leggibile e modulare**, in modo che ogni funzionalità lato client possa essere ricondotta a un punto preciso della struttura del progetto.

L'adozione di una soluzione statica servita da Nginx comporta inoltre un vantaggio architetturale significativo. Il frontend non richiede un runtime applicativo dedicato alla generazione delle pagine, poiché le interfacce sono distribuite come asset statici; allo stesso tempo, il web server svolge il ruolo di punto di esposizione unico verso il browser e di proxy iniziale per le richieste API. Il risultato è un modello di esecuzione semplice, prevedibile e coerente con l'architettura complessiva del sistema.

Il modello multi-page favorisce anche una migliore **segmentazione dell'esperienza utente**. In una piattaforma con ruoli fortemente differenziati, la suddivisione in pagine dedicate permette di evitare un'interfaccia eccessivamente generalista o condizionata da logiche di rendering dinamico troppo pervasive. Ogni area dell'applicazione presenta soltanto le funzionalità pertinenti al contesto corrente e il caricamento delle dipendenze lato client avviene in maniera selettiva, limitando l'esposizione di codice non necessario per il singolo scenario.

Sul piano del comportamento applicativo, l'architettura del frontend adotta una logica di **enhanced static pages**: il documento HTML fornisce la struttura della vista, mentre JavaScript interviene per popolare contenuti dinamici, validare input, recuperare dati dal backend, gestire modali, aggiornare stati visivi e orchestrare interazioni con le API. Questa

impostazione preserva la chiarezza del markup e rende la logica client-side più facilmente localizzabile e manutenibile.

7.2. ORGANIZZAZIONE DELLE INTERFACCE PER RUOLO

L'organizzazione delle interfacce è costruita attorno a un principio di **segmentazione per ruolo**, che rappresenta una delle scelte più rilevanti del frontend. La piattaforma non adotta una navigazione uniforme per tutti gli utenti, ma distingue in modo netto le aree funzionali in base al contesto operativo di appartenenza: autenticazione, amministrazione, attività clinica, operatività del paziente e operatività del delegato.

Questa organizzazione non ha un valore soltanto estetico o ordinativo, ma svolge una funzione architeturale precisa. La separazione delle interfacce per ruolo consente infatti di:

- ridurre la complessità percepita dall'utente;
- allineare la navigazione al modello autorizzativo;
- limitare la presenza di elementi non pertinenti nel contesto corrente;
- rendere più chiara la corrispondenza tra flusso applicativo e vista;
- mantenere un migliore isolamento del codice client-side dedicato alle singole aree.

Ogni area è quindi progettata come uno **spazio applicativo coerente**, con proprie pagine, proprie azioni prevalenti e propri script dedicati.

7.2.1. AREA AUTH

L'area **auth** raccoglie tutte le interfacce legate alla gestione del ciclo di vita dell'account e dell'accesso al sistema. Essa costituisce il punto di ingresso del frontend e include le pagine dedicate a registrazione, attivazione account, autenticazione, recupero delle credenziali e reimpostazione della password.

Dal punto di vista progettuale, questa area è caratterizzata da un'interazione fortemente orientata ai **form di input**, alla validazione dei dati inseriti e alla gestione dei messaggi di stato restituiti dal backend. Le pagine di autenticazione non sono semplici schermate di raccolta credenziali, ma vere interfacce di ingresso controllato alla piattaforma, nelle quali il comportamento del client deve accompagnare l'utente in un flusso chiaro, rigoroso e coerente con i controlli di sicurezza applicati dal sistema.

L'area auth ha inoltre una funzione di raccordo tra utente anonimo e utente autenticato. In questa zona del frontend vengono inizializzati i primi passaggi del rapporto tra soggetto e piattaforma: creazione dell'account, completamento dell'attivazione, apertura della sessione, reindirizzamento verso l'area corretta in base al ruolo.

7.2.2. AREA ADMIN

L'area **admin** è progettata per supportare le attività di coordinamento amministrativo e operativo della struttura. Le sue interfacce sono orientate alla gestione di entità e processi ad alta densità informativa: anagrafiche, clinici, delegati, catalogo prestazioni, disponibilità, appuntamenti, check-in, pagamenti, notifiche e audit log.

Dal punto di vista della UX applicativa, questa area privilegia viste tabellari, filtri, modali di dettaglio, form strutturati, pulsanti di azione contestuale e pagine di consultazione con capacità operative. L'obiettivo è offrire all'utente amministrativo un ambiente di lavoro nel quale sia possibile sia ottenere una **visione sintetica** del sistema, sia intervenire in modo puntuale sulle singole risorse.

L'area admin è anche quella in cui emerge con maggiore evidenza la necessità di mantenere un equilibrio tra **densità funzionale** e **chiarezza dell'interfaccia**. Le pagine non devono solo esporre dati, ma consentire operazioni guidate, con feedback chiari, azioni coerenti con lo stato corrente delle risorse e piena integrazione con il modello autorizzativo del backend.

7.2.3. AREA CLINICIAN

L'area **clinician** è costruita intorno al percorso operativo del professionista sanitario. Le sue interfacce sono orientate alla consultazione dell'agenda, all'accesso al dettaglio degli appuntamenti, alla gestione degli encounter clinici, alla compilazione delle informazioni sanitarie rilevanti, alla redazione del referto e all'utilizzo del modulo AI consultivo.

Questa area presenta un'impostazione diversa rispetto all'area amministrativa. La priorità non è la supervisione globale del sistema, ma il supporto a un flusso operativo **mirato, sequenziale e contestualizzato**. Le interfacce del Clinician devono accompagnare il professionista nel passaggio dall'agenda all'episodio assistenziale, dalla consultazione dei dati disponibili all'inserimento della documentazione clinica, fino alla formalizzazione del referto.

Il frontend clinician è quindi progettato per valorizzare la **continuità del contesto**. L'utente deve poter comprendere con chiarezza quale appuntamento sta gestendo, quale encounter è in corso, quali informazioni sono già presenti e quali azioni sono ammesse in quel preciso momento del flusso. Anche la sezione AI è inserita in questo perimetro, come strumento consultivo e non come area autonoma scollegata dal contesto professionale.

7.2.4. AREA PATIENT

L'area **patient** è progettata per offrire un'esperienza self-service orientata alla semplicità d'uso e alla chiarezza operativa. Le principali funzionalità disponibili in questa sezione riguardano il profilo personale, la consultazione del catalogo, la ricerca delle disponibilità, la prenotazione, la gestione degli appuntamenti, la compilazione del pre-triage, la consultazione dei referti, i pagamenti e le notifiche.

Dal punto di vista progettuale, questa area si differenzia sia da quella amministrativa sia da quella clinica, poiché il suo obiettivo non è la gestione interna del sistema, ma la **fruizione controllata dei servizi digitali** da parte dell'utente finale. Le interfacce devono quindi essere dirette, leggibili e orientate al completamento di azioni ben definite, con particolare attenzione alla comprensibilità dei form, alla visibilità dello stato degli appuntamenti e alla chiarezza delle informazioni mostrate.

L'area patient traduce sul piano dell'interfaccia il principio di autonomia operativa dell'utente finale, ma sempre entro un perimetro fortemente delimitato. Il frontend riflette questo modello evitando di esporre dati o azioni che eccedano il dominio personale del soggetto autenticato.

7.2.5. AREA DELEGATE

L'area **delegate** riprende parte delle logiche dell'area patient, ma le adatta a un modello di operatività per rappresentanza. Le interfacce dedicate al delegato consentono infatti di agire per conto dell'assistito, a condizione che esista una relazione di delega valida e che il contesto selezionato sia coerente con essa.

Dal punto di vista della progettazione frontend, questa area richiede una particolare attenzione alla **distinzione tra soggetto autenticato e soggetto rappresentato**. Le pagine devono rendere esplicito su quale assistito il delegato stia operando, evitare ambiguità nella visualizzazione dei dati e guidare l'utente attraverso un'interazione contestualizzata, nella quale il perimetro operativo dipende dalla selezione dell'assistito e dalla validità della delega.

L'area delegate non è quindi una semplice replica del frontend paziente. Pur condividendo alcune funzionalità — come prenotazioni, pre-triage, notifiche o consultazione documentale — introduce una semantica diversa, nella quale la rappresentanza e la tracciabilità delle azioni diventano elementi centrali anche sul piano dell'interfaccia.

7.3. COMPONENTI E SCRIPT CONDIVISI

La progettazione del frontend non si limita alla definizione di pagine distinte, ma prevede anche un insieme di **componenti logici e script condivisi** che garantiscono coerenza trasversale tra le diverse aree dell'applicazione. Questa scelta riduce la duplicazione del codice, migliora la consistenza del comportamento lato client e facilita la manutenzione dell'intero frontend.

Gli elementi condivisi riguardano anzitutto gli aspetti di **utility applicativa**, come la gestione delle richieste HTTP, la manipolazione di token o informazioni di sessione, la normalizzazione dei messaggi di errore, il rendering di elementi ricorrenti, la gestione di modali o componenti UI riutilizzabili e il supporto a funzionalità trasversali come logout, redirect o protezione delle pagine.

La presenza di script comuni consente di evitare che ciascuna pagina implementi in modo autonomo logiche standard, come:

- verifica della sessione corrente;
- aggiunta degli header di autorizzazione alle richieste;
- gestione uniforme delle risposte non autorizzate;
- formattazione di date, stati o etichette;
- rendering di navigazione, header o componenti condivisi;
- apertura e chiusura di modali o pannelli contestuali.

Dal punto di vista architetturale, questi script costituiscono il **collante del frontend**. Essi non appartengono a un singolo flusso di business, ma forniscono capacità riusabili che permettono alle pagine specializzate di concentrarsi sulla propria responsabilità specifica. In questo modo il codice client-side resta più vicino a una struttura a responsabilità separate: utilities comuni da un lato, logiche di pagina dall'altro.

Anche i componenti visivi condivisi hanno un ruolo importante. Header, breadcrumb, cards, tabelle, badge di stato, modali di conferma, pannelli di filtro e blocchi di messaggistica contribuiscono a creare un linguaggio visivo uniforme tra aree differenti del sistema. La coerenza dell'interfaccia non dipende soltanto dagli stili globali, ma anche dalla ripetizione disciplinata di pattern ricorrenti che l'utente può riconoscere e interpretare con facilità.

7.4. GESTIONE DELLA SESSIONE LATO CLIENT

La gestione della sessione lato client rappresenta uno degli aspetti più delicati della progettazione del frontend, poiché collega il comportamento dell'interfaccia con il modello di autenticazione e autorizzazione governato dal backend. Il browser deve infatti mantenere un contesto sufficiente a operare come utente autenticato, ma senza assumere responsabilità che spettano al server in termini di validazione sostanziale dell'accesso.

Sul piano funzionale, il frontend deve essere in grado di:

- memorizzare il contesto di autenticazione nella forma prevista dal sistema;
- recuperarlo quando necessario;
- utilizzarlo per le richieste protette verso le API;
- reagire correttamente in caso di sessione assente, scaduta o non valida;
- riallineare il comportamento dell'interfaccia con il ruolo effettivo dell'utente autenticato.

La sessione lato client non è quindi un semplice dato tecnico, ma il presupposto per il corretto rendering delle aree protette e per la corretta composizione delle richieste HTTP. Gli script comuni del frontend hanno il compito di recuperare il token o le informazioni di sessione, inserirli negli header appropriati, verificare se l'utente possa accedere alla pagina corrente e, in caso contrario, reindirizzarlo verso il flusso di autenticazione.

Un aspetto particolarmente importante è la **coerenza tra sessione e ruolo**. Il frontend non deve limitarsi a sapere che esiste un utente autenticato, ma deve anche comprendere quale area applicativa debba essere resa disponibile. Questo si traduce in controlli lato client finalizzati a prevenire accessi impropri all'interfaccia e a garantire che la navigazione segua il perimetro funzionale corretto.

La gestione della sessione lato client deve inoltre essere robusta nei confronti di scenari anomali: token mancanti, sessioni revoked, scadenze, risposte 401 o 403, accessi diretti a URL protetti, cambio di stato dell'account. In tutti questi casi il frontend deve reagire in modo coerente, ripristinando il flusso corretto di autenticazione o mostrando un messaggio comprensibile all'utente.

7.5. INTERAZIONE CON LE API TRAMITE FRONTEND ORIGIN

Uno dei principi più importanti della progettazione del frontend è che il browser interagisce **sempre e soltanto con l'origine del frontend**, senza comunicare direttamente con i servizi di backend. Questa scelta costituisce un elemento architeturale essenziale della piattaforma e influenza in modo diretto sia l'organizzazione delle richieste lato client sia il modello di deployment.

Quando una pagina necessita di recuperare dati o inviare un'azione al backend, lo script JavaScript non costruisce richieste verso host interni del Core Service, dell'AI Assistant o del gateway. Le chiamate vengono invece indirizzate verso path `/api/*` sulla stessa origine che ha servito la pagina. A livello di runtime, è Nginx a intercettare tali richieste e a inoltrarle verso l'API Gateway.

Questo approccio offre diversi vantaggi progettuali. In primo luogo, il frontend rimane **agnostico rispetto alla topologia interna dei servizi**, poiché non deve conoscere nomi di container, porte o indirizzi downstream. In secondo luogo, il browser opera sempre nello stesso perimetro di origine, semplificando la gestione della sicurezza e riducendo la complessità del modello di comunicazione. In terzo luogo, l'interfaccia resta coerente con il principio secondo cui il gateway è l'unico punto di ingresso API del sistema.

Dal punto di vista dell'implementazione client-side, tale scelta si traduce in una disciplina molto precisa nella costruzione delle richieste. Gli script devono:

- utilizzare percorsi relativi coerenti con il proxy `/api/*`;
- includere gli header di autorizzazione quando richiesti;
- interpretare in modo uniforme gli esiti delle risposte;
- non introdurre dipendenze dirette da endpoint interni non esposti al client.

L'interazione tramite frontend origin favorisce anche la **manutenibilità del frontend**. Cambiamenti nella topologia interna del backend, nel posizionamento del gateway o nella struttura della rete containerizzata non richiedono necessariamente modifiche al codice JavaScript lato browser, purché resti invariata la convenzione di accesso attraverso l'origine frontend.

7.6. GESTIONE DEGLI ERRORI E FALLBACK APPLICATIVI

La progettazione del frontend include una strategia esplicita di gestione degli errori e dei fallback applicativi, necessaria per mantenere il comportamento dell'interfaccia coerente anche in presenza di input non validi, risposte negative del backend, indisponibilità temporanee dei servizi o condizioni di accesso non soddisfatte.

La prima dimensione di questo problema riguarda gli **errori di validazione e di interazione utente**. Le pagine che includono form o azioni sensibili devono intercettare dati incompleti, incoerenti o non ammissibili e fornire un feedback immediato e leggibile. Tale gestione non sostituisce la validazione server-side, ma migliora la qualità dell'esperienza utente e riduce l'invio di richieste palesemente non corrette.

La seconda dimensione riguarda gli **errori applicativi restituiti dalle API**. Il frontend deve essere in grado di interpretare risposte come autenticazione fallita, autorizzazione insufficiente, risorsa non trovata, vincoli di stato non rispettati o errori di business, traducendole in messaggi comprensibili e in azioni coerenti. Questo implica una progettazione attenta della UI di errore: non tutte le anomalie devono produrre lo stesso comportamento, poiché cambia il significato operativo del problema riscontrato.

Un terzo ambito è quello degli **errori infrastrutturali o di connettività**, come timeout, indisponibilità del servizio, risposta non valida o interruzione della comunicazione con il backend. In questi casi il frontend deve evitare comportamenti opachi o silenziosi e offrire

meccanismi di fallback appropriati, come messaggi informativi, disabilitazione temporanea di azioni, tentativi di ricarica controllati o ritorno a uno stato di interfaccia consistente.

La gestione dei fallback è particolarmente importante nelle pagine che dipendono da dati caricati dinamicamente. Dashboard, tabelle, dettagli di appuntamenti, modali operative e schermate AI devono poter distinguere tra stato di caricamento, stato di successo, stato vuoto e stato di errore. Questa differenziazione migliora la leggibilità del frontend e rende l'interazione più robusta.

Un ulteriore aspetto riguarda la **gestione delle risposte non autorizzate**. Quando il backend restituisce un errore di autenticazione o autorizzazione, il frontend deve riallinearsi con il modello di sessione: reindirizzare al login, mostrare un messaggio coerente, impedire la permanenza in aree protette o segnalare che l'utente non dispone dei privilegi richiesti. Anche questo è un fallback applicativo a tutti gli effetti, poiché impedisce che l'interfaccia resti in uno stato ambiguo o non coerente con il contesto di accesso.

La presenza di fallback disciplinati contribuisce in modo diretto alla qualità professionale del frontend. Un'interfaccia ben progettata non si distingue soltanto per la correttezza dei flussi positivi, ma anche per la capacità di reagire in modo ordinato, leggibile e coerente a condizioni di errore, prevenendo disorientamento dell'utente e stati applicativi inconsistenti.

8. PROGETTAZIONE DELL'API GATEWAY

8.1. RUOLO DEL GATEWAY NELL'ARCHITETTURA

L'**API Gateway** costituisce il punto di ingresso unificato per tutto il traffico API proveniente dal frontend e rappresenta uno degli elementi architetturali più rilevanti dell'intera piattaforma. La sua presenza consente di evitare che il browser comunichi direttamente con i servizi applicativi interni, introducendo un livello intermedio dedicato al **controllo**, all'**instradamento** e alla **gestione delle responsabilità trasversali**.

Dal punto di vista logico, il gateway occupa una posizione di confine tra il livello di presentazione e il livello dei servizi applicativi. A monte riceve richieste HTTP inoltrate dal frontend tramite Nginx; a valle smista tali richieste verso il **Core Service** oppure verso l'**AI Assistant**, sulla base del path richiesto e delle regole di dispatching definite. Questo modello realizza un'architettura nella quale i servizi di business rimangono isolati rispetto al client e vengono raggiunti soltanto attraverso un canale di accesso controllato.

La funzione del gateway non si esaurisce nel mero inoltro delle richieste. Esso agisce come **front door applicativa**, concentrando in un unico punto i controlli che devono essere applicati in maniera uniforme a più servizi. In questa prospettiva, il gateway assolve funzioni di sicurezza, osservabilità minima, trattamento coerente degli errori, qualificazione del contesto di richiesta e protezione del perimetro API. Tale centralizzazione riduce la duplicazione delle logiche cross-cutting nei servizi downstream e rende il sistema più leggibile dal punto di vista architetturale.

Un ulteriore aspetto importante è la distinzione tra responsabilità del gateway e responsabilità dei servizi applicativi. Il gateway non è proprietario dei dati di business e non governa i domini funzionali della piattaforma. Esso non prende decisioni relative a scheduling, clinical, payments o registry, ma si occupa di ricevere la richiesta, verificarne alcuni presupposti, gestire le componenti trasversali e instradare il traffico verso il servizio competente. Questa delimitazione è essenziale per evitare che il gateway degeneri in un livello impropriamente caricato di logica applicativa.

Dal punto di vista della sicurezza, il gateway contribuisce a rendere il sistema più disciplinato. La verifica dei token per le richieste protette, la gestione del traffico cross-origin, il rate limiting, la correlazione delle richieste tramite identificativi dedicati e la normalizzazione degli errori HTTP non sono lasciati alla libera implementazione di ciascun servizio, ma vengono governati a monte. In questo modo il perimetro esposto al client risulta più coerente e più facilmente controllabile.

Sul piano della manutenibilità, l'introduzione del gateway semplifica anche l'evoluzione della piattaforma. Il frontend continua a interagire con un solo endpoint logico `/api/*`, indipendentemente dal numero e dalla natura dei servizi downstream. Eventuali cambiamenti interni nella distribuzione delle responsabilità tra Core Service e AI Assistant non impongono necessariamente modifiche al client, purché resti invariato il contratto esposto dal gateway. Questa proprietà rafforza il disaccoppiamento tra livello di presentazione e topologia interna del backend.

8.2. ROUTING E REVERSE PROXY

Il comportamento del gateway è fondato su una logica di **routing centralizzato** e di **reverse proxy dispatch**. Tutte le richieste API inoltrate dal frontend confluiscono in un unico perimetro applicativo e vengono poi smistate dal gateway verso il destinatario corretto in funzione del path richiesto e delle regole di gestione definite.

La piattaforma adotta un modello di instradamento basato su un **catch-all applicativo** per il namespace `/api/*`. Questa impostazione consente al gateway di intercettare l'intero traffico API rilevante e di sottoporlo a una catena uniforme di trattamenti, senza frammentare il controllo su percorsi differenti o esporre logiche di dispatching distribuite in più punti. Il risultato è un meccanismo di accesso più leggibile e architetturealmente più robusto.

All'interno di questo schema, il gateway distingue tre grandi categorie di richieste. La prima comprende gli endpoint di **health aggregata**, gestiti localmente dal gateway stesso. La seconda riguarda le richieste dirette verso il servizio **AI Assistant**, riconoscibili dal prefisso dedicato e inoltrate al relativo servizio downstream dopo l'eventuale trasformazione del path. La terza categoria comprende tutte le altre richieste API, che vengono inoltrate al **Core Service**, titolare dei principali domini di business della piattaforma.

Il reverse proxy non deve essere interpretato come un semplice meccanismo trasparente di forwarding HTTP. Nel contesto di questa piattaforma esso opera infatti come parte integrante del modello di controllo: prima dell'inoltro, la richiesta attraversa i middleware del gateway, viene qualificata rispetto al proprio contesto e può essere soggetta a verifiche di sicurezza, limitazioni o arricchimenti tecnici. Solo a valle di tali passaggi viene generato l'instradamento effettivo verso il servizio destinatario.

Un elemento rilevante di questa progettazione è la distinzione tra **instradamento locale** e **instradamento verso upstream**. Alcune richieste, come quelle legate alla verifica aggregata della salute del sistema, sono processate direttamente dal gateway senza delega a un singolo servizio; altre vengono invece inoltrate al servizio corretto, preservando l'idea secondo cui il gateway è contemporaneamente punto di ingresso, punto di controllo e nodo di integrazione.

La logica di reverse proxy implementa anche una forma di **normalizzazione del perimetro esposto**. Dal punto di vista del browser, tutti i servizi backend risultano astratti dietro lo stesso namespace API. Questa unificazione riduce la complessità lato client, impedisce l'emersione della topologia interna dei servizi e contribuisce a mantenere il backend esposto come un unico sistema coerente, pur essendo internamente articolato in più componenti.

Particolarmente importante è il caso del modulo AI. Le richieste dirette all'assistente non vengono esposte come accesso separato o come host differente, ma transitano attraverso il gateway secondo una convenzione uniforme. Ciò consente di mantenere anche il servizio AI entro il medesimo perimetro di sicurezza, logging e controllo applicato al resto della piattaforma.

8.3. MIDDLEWARE CROSS-ROUTING

I middleware cross-cutting rappresentano il cuore del comportamento trasversale del gateway. Essi consentono di applicare a tutte le richieste, o a insiemi coerenti di esse, un insieme di politiche comuni che non appartengono ai domini di business ma risultano indispensabili per la qualità operativa dell'intero sistema.

La presenza di middleware dedicati risponde a una precisa scelta progettuale: isolare in un unico livello infrastrutturale le responsabilità che devono essere **uniformi, ripetibili e coerenti** per tutto il traffico API. In questo modo, i servizi downstream possono concentrarsi sulla logica di dominio, mentre il gateway governa aspetti come correlazione delle richieste, logging, gestione delle eccezioni, policy CORS, contenimento del traffico e verifica preliminare dell'accesso.

L'adozione di un set strutturato di middleware migliora la disciplina dell'architettura. Le richieste non arrivano ai servizi applicativi in forma "grezza", ma attraversano un livello che le rende più qualificate e meglio controllate. Ciò rafforza la coerenza del sistema e riduce il rischio di comportamenti divergenti tra endpoint appartenenti a servizi diversi.

8.3.1. REQUEST ID

Il **Request ID** ha la funzione di attribuire a ogni richiesta un identificativo univoco o, quando già presente, di preservarlo e propagarlo lungo la catena di elaborazione. Questo meccanismo costituisce uno strumento fondamentale di correlazione tecnica e operativa, poiché consente di collegare tra loro i diversi eventi prodotti dal sistema durante il trattamento della stessa richiesta.

In un'architettura nella quale il traffico attraversa frontend, gateway e servizi downstream, la presenza di un identificativo coerente migliora sensibilmente la possibilità di comprendere il percorso effettivo della richiesta. Log, errori, eventi tecnici e risposte possono essere associati allo stesso contesto, facilitando la diagnosi dei problemi, la ricostruzione dei flussi e l'analisi del comportamento del sistema.

La scelta di introdurre il Request ID a livello di gateway è coerente con la natura del componente. Essendo il primo punto di contatto per il traffico API, il gateway è il luogo più appropriato in cui generare o consolidare un identificativo condiviso, così da garantire uniformità e tracciabilità tra i servizi downstream.

8.3.2. LOGGING STRUTTURATO

Il **logging strutturato** consente al gateway di registrare informazioni rilevanti sul traffico HTTP in un formato coerente, interpretabile e adatto alla correlazione con altri eventi del sistema. A differenza di un logging puramente testuale e informale, il logging strutturato organizza i dati della richiesta e della risposta in modo più leggibile e più utile dal punto di vista operativo.

Nel contesto del gateway, il logging ha una duplice funzione. Da un lato, consente di monitorare il comportamento del front door applicativo, registrando le richieste ricevute, gli esiti prodotti e gli eventuali errori o anomalie. Dall'altro, permette di associare tali eventi agli identificativi di richiesta, ai path coinvolti, agli status code e agli altri elementi che qualificano il flusso tecnico.

L'importanza del logging strutturato aumenta in presenza di routing e reverse proxy, poiché il gateway costituisce il punto in cui convergono richieste destinate a servizi differenti. Disporre di una registrazione coerente delle interazioni consente di ottenere una vista più ordinata del comportamento complessivo del sistema e di isolare più facilmente eventuali problemi di instradamento, sicurezza o disponibilità degli upstream.

8.3.3. RECOVERY JSON

Il middleware di **Recovery JSON** ha il compito di intercettare condizioni di errore non gestite correttamente all'interno della catena di elaborazione e di trasformarle in una risposta HTTP strutturata, coerente con il comportamento atteso del gateway. Questa funzione è essenziale per evitare che eccezioni runtime o malfunzionamenti interni producano risposte opache, HTML non previsti o errori incoerenti con la natura API del sistema.

La presenza di un meccanismo di recovery a livello gateway migliora la robustezza del perimetro esposto. Anche in presenza di condizioni inattese, il client riceve una risposta nel formato appropriato, comprensibile dal punto di vista applicativo e coerente con il contratto generale delle API. Questo contribuisce a mantenere il comportamento del sistema più prevedibile e più facilmente gestibile dal frontend.

L'adozione di una recovery espressamente orientata a **JSON** risulta particolarmente importante in una piattaforma nella quale il browser, tramite JavaScript, si aspetta risposte strutturate per l'interpretazione degli errori. Un gateway che restituisse errori non normalizzati renderebbe più fragile la gestione delle anomalie lato client e comprometterebbe la coerenza dell'integrazione.

8.3.4. CORS

La gestione del **CORS** risponde all'esigenza di controllare in modo esplicito le politiche di accesso cross-origin verso il perimetro API. Sebbene il modello architetturale della piattaforma sia costruito in modo che il browser interagisca normalmente con una sola origine applicativa, la presenza del gateway come punto di ingresso delle API rende comunque necessario presidiare il comportamento HTTP legato a origini, metodi e header consentiti.

Il middleware CORS ha quindi la funzione di dichiarare, in modo centralizzato, quali richieste cross-origin siano ammissibili e secondo quali condizioni. Questa esplicitazione contribuisce sia alla sicurezza sia alla prevedibilità del sistema, poiché evita comportamenti impliciti o incoerenti nell'accesso alle API.

Dal punto di vista progettuale, collocare la gestione del CORS nel gateway è una scelta naturale. Trattandosi del punto unico di ingresso per le API, il gateway è il livello più appropriato in cui applicare una policy uniforme, invece di delegare tale responsabilità a ciascun servizio downstream. Ciò rafforza il principio di centralizzazione delle responsabilità trasversali e riduce il rischio di configurazioni divergenti tra componenti diversi.

8.3.5. RATE LIMITING

Il **rate limiting** ha la funzione di contenere il traffico eccessivo verso il perimetro API, proteggendo il sistema da usi impropri, carichi anomali o comportamenti client-side non

controllati. In una piattaforma esposta tramite un punto di ingresso unico, la capacità di limitare in modo uniforme la frequenza delle richieste rappresenta una misura importante sia dal punto di vista della stabilità sia da quello della sicurezza.

L'introduzione del rate limiting nel gateway consente di applicare questa protezione prima che il traffico raggiunga i servizi di business. Tale scelta riduce la pressione sugli upstream, migliora la resilienza complessiva del sistema e contribuisce a evitare che richieste eccessive compromettano la disponibilità di funzionalità critiche.

Il rate limiting non deve essere letto come misura puramente difensiva nei confronti di attacchi intenzionali. Esso è utile anche in scenari meno estremi, ad esempio quando errori di integrazione lato client o comportamenti ripetitivi non previsti generano un numero elevato di richieste verso gli stessi endpoint. In questo senso, il middleware svolge una funzione di **contenimento preventivo** e di regolarizzazione del traffico.

8.3.6. VERIFICA JWT

La **verifica JWT** costituisce uno dei meccanismi più importanti del gateway sotto il profilo della sicurezza. Il middleware dedicato ha il compito di controllare, per le richieste protette, la presenza di un token di accesso valido e coerente con il modello di autenticazione della piattaforma.

Questa verifica rappresenta un primo filtro di accesso al perimetro API. Il gateway non sostituisce il controllo di autorizzazione applicativa svolto dai servizi downstream, ma si occupa di impedire che richieste prive di un contesto di autenticazione corretto possano oltrepassare il front door del sistema. Ciò rafforza l'idea che l'accesso protetto debba essere governato fin dal primo punto di ingresso, non solo nei livelli interni della piattaforma.

La verifica JWT si inserisce in una strategia di sicurezza a più livelli. Il gateway controlla il token e, nei casi previsti, può appoggiarsi al Core Service per la validazione del suo stato effettivo; successivamente, i servizi applicativi effettuano i controlli di autorizzazione specifici relativi al ruolo, alla risorsa e al contesto di business. Questa stratificazione migliora la robustezza del sistema e riduce il rischio che l'accesso venga trattato in modo troppo permissivo o troppo frammentario.

8.4. TOKEN INTROSPECTION VERSO IL CORE SERVICE

La **token introspection** verso il Core Service rappresenta il meccanismo attraverso cui il gateway verifica, nei casi previsti, che un token formalmente presentato dal client sia anche sostanzialmente valido nel contesto corrente della piattaforma. Questo passaggio è particolarmente importante perché la sicurezza del sistema non può basarsi esclusivamente sulla presenza materiale del token o sulla sola correttezza sintattica della sua struttura.

Il Core Service costituisce il punto autorevole per il dominio **Auth & Identity** e, in quanto tale, è il soggetto più appropriato a confermare lo stato del token, la validità della sessione associata e l'assenza di condizioni di revoca o invalidazione. Il gateway delega quindi a questo servizio la parte di validazione che richiede conoscenza dello stato applicativo dell'accesso, mantenendo così allineata la gestione della sicurezza tra front door e backend principale.

Dal punto di vista architetturale, la token introspection rafforza il principio di **centralizzazione dell'autorità sull'identità**. Il gateway esercita il controllo sul traffico in ingresso, ma non si sostituisce al servizio proprietario delle informazioni di autenticazione. Tale ripartizione è corretta sia sul piano logico sia sul piano della manutenibilità: il gateway filtra e qualifica, il Core Service decide in quanto owner del dominio identità.

Questo modello risulta utile anche per mantenere coerente la verifica dei token tra più componenti. L'AI Assistant, pur essendo un servizio separato, fa anch'esso riferimento al Core Service per confermare la validità del token e il ruolo del soggetto autenticato. In questo modo l'intera piattaforma evita la proliferazione di logiche autonome e potenzialmente divergenti nella gestione dell'accesso.

L'introspection contribuisce inoltre alla **revocabilità effettiva** delle sessioni. Un token che non sia scaduto dal punto di vista temporale potrebbe comunque non essere più autorizzato all'uso, ad esempio a seguito di logout o invalidazione esplicita. La consultazione del Core Service consente di intercettare questi casi e di impedire che il gateway consideri valido un contesto di accesso ormai non più ammesso.

8.5. AGGREGAZIONE DELLO STATO DI SALUTE DEI SERVIZI

L'aggregazione dello stato di salute dei servizi è una delle responsabilità locali più importanti del gateway. Essa consente di esporre una vista sintetica e centralizzata della disponibilità dei principali componenti applicativi downstream, rendendo più semplice la verifica operativa del sistema e fornendo un punto unico di osservazione del suo stato runtime essenziale.

Il gateway gestisce in modo specifico l'endpoint dedicato alla **health aggregata**, interrogando i servizi applicativi rilevanti — in particolare Core Service e AI Assistant — attraverso i rispettivi endpoint di health e componendo poi una risposta coerente verso il chiamante. In questo modo, chi interroga il gateway non deve conoscere la topologia interna dei servizi né invocare separatamente ogni singolo endpoint di verifica.

Questa funzionalità rafforza il ruolo del gateway come **facciata operativa** dell'ecosistema backend. Non si tratta soltanto di un router per richieste di business, ma anche di un componente in grado di esprimere una visione elementare ma unificata dello stato del sistema. Tale capacità risulta particolarmente utile in contesti di deploy containerizzato, nei quali l'effettiva disponibilità di un'applicazione dipende dal corretto funzionamento coordinato di più servizi.

È importante distinguere la health aggregata dal mero stato locale del gateway. Il fatto che il gateway sia raggiungibile non implica automaticamente che siano disponibili anche il Core Service e l'AI Assistant. L'aggregazione serve precisamente a colmare questa differenza, offrendo una risposta che tenga conto non solo della vitalità del front door, ma anche della raggiungibilità e della reattività dei servizi da esso mediati.

Dal punto di vista progettuale, collocare questa responsabilità nel gateway è coerente con la sua posizione architetturale. Essendo il punto di ingresso unificato del backend, il gateway è il luogo naturale in cui sintetizzare le informazioni minime di disponibilità dell'ecosistema applicativo esposto. Questa scelta migliora la leggibilità operativa del sistema e riduce la frammentazione delle informazioni di health tra componenti separati.

9. PROGETTAZIONE DEL CORE SERVICE

9.1. ORGANIZZAZIONE DELLA SOLUTION

Il **Core Service** rappresenta il nucleo applicativo della piattaforma e concentra la logica di business relativa ai principali domini funzionali del sistema: autenticazione e identità, anagrafiche e profili, scheduling, percorso clinico, pagamenti, notifiche ed eventi. La sua progettazione segue una struttura a layer, organizzata in una solution composta da più progetti distinti, ciascuno dei quali presidia un preciso livello di responsabilità.

Questa organizzazione non risponde a una mera esigenza formale, ma a un criterio di **separazione architetturale**. In un servizio che governa processi eterogenei e sensibili, la chiarezza dei confini tra esposizione HTTP, orchestrazione dei casi d'uso, modello di dominio e dettagli infrastrutturali è essenziale per mantenere il codice leggibile, manutenibile e coerente con l'evoluzione del sistema.

La solution è articolata in quattro componenti principali:

- **CoreService.Api**
- **CoreService.Application**
- **CoreService.Domain**
- **CoreService.Infrastructure**

Questa ripartizione permette di evitare che la logica applicativa venga dispersa in modo indistinto tra controller, modelli persistenti e servizi tecnici. Ogni progetto contribuisce a una parte diversa del comportamento del Core Service e partecipa a un modello nel quale la dipendenza concettuale procede dall'esterno verso l'interno: il layer API espone il sistema, il layer Application orchestra i casi d'uso, il layer Domain esprime il modello di business, il layer Infrastructure realizza i dettagli tecnici di persistenza e integrazione.

9.1.1. CoreService.Api

CoreService.Api costituisce il punto di esposizione HTTP del servizio ed è il progetto in cui risiede la configurazione dell'applicazione server-side, la definizione degli endpoint REST, la composizione dei middleware applicativi e l'avvio del runtime ASP.NET Core. Esso rappresenta il livello di contatto tra il Core Service e il resto della piattaforma, in particolare il gateway che ne inoltra le richieste.

In questo progetto trovano collocazione gli elementi necessari a ricevere le richieste, validarne i presupposti tecnici, invocare i casi d'uso del layer applicativo e trasformarne gli esiti in risposte HTTP coerenti. Il layer API non è il luogo in cui viene implementata la logica di business in senso stretto; il suo ruolo è piuttosto quello di **adattatore esterno**, responsabile della traduzione tra protocollo HTTP e comportamento applicativo.

Dal punto di vista architetturale, CoreService.Api svolge anche una funzione di composizione. È in questo progetto che vengono registrate le dipendenze, configurati i servizi, definiti i meccanismi di autenticazione, attivati i background services, predisposti i controller e integrati i componenti necessari all'esecuzione complessiva del Core Service. La sua responsabilità è quindi duplice: da un lato espone le API, dall'altro costruisce il runtime del servizio.

9.1.2. CoreService.Application

CoreService.Application costituisce il livello di orchestrazione dei casi d'uso. In questo progetto vengono modellate le operazioni applicative che il sistema è chiamato a eseguire in risposta alle richieste provenienti dal layer API o dai processi interni. Il focus non è sulla rappresentazione HTTP né sui dettagli della persistenza, ma sulla definizione di **azioni di business coordinate**, espresse in termini coerenti con il dominio della piattaforma.

Il layer Application organizza i flussi che riguardano, ad esempio, registrazione e attivazione account, gestione dei profili, deleghe, consensi, prenotazioni, check-in, apertura e gestione degli encounter, produzione dei referti, pagamenti, notifiche e audit. In tale livello si concentrano le regole di orchestrazione che combinano validazioni applicative, accesso al dominio, utilizzo di interfacce astratte e trasformazione dei dati in forme adatte alla comunicazione verso l'esterno.

Questo progetto rappresenta il **ponte tra il mondo esterno e il modello di dominio**. Da un lato riceve input strutturati tramite DTO, comandi o richieste applicative; dall'altro invoca servizi, entità e policy del dominio, eventualmente appoggiandosi a interfacce implementate dall'infrastruttura. La sua funzione è mantenere i casi d'uso leggibili e separati sia dai dettagli del trasporto HTTP sia da quelli dell'accesso al database.

9.1.3. CoreService.Domain

CoreService.Domain costituisce il centro concettuale del Core Service. In questo progetto viene espresso il modello di business della piattaforma attraverso entità, value object, enumerazioni, regole di stato, concetti di dominio e strutture semantiche che descrivono il comportamento fondamentale del sistema.

Il domain layer non è costruito per conoscere il protocollo HTTP, i controller, i dettagli del database o la configurazione dell'applicazione. Il suo compito è rappresentare, nel modo più puro possibile, i concetti che caratterizzano la piattaforma: utenti, profili, deleghe, consensi, appuntamenti, slot, encounter, anamnesi, parametri vitali, ordini clinici, esecuzioni, referti, intenti di pagamento, transazioni, notifiche, eventi di audit e relative regole di business.

La centralità di questo progetto risiede nel fatto che esso definisce **la semantica interna del sistema**. Le regole che governano gli stati degli appuntamenti, le condizioni di validità di un encounter, la distinzione tra ruoli, la consistenza delle entità e il significato dei dati trattati devono emergere anzitutto qui, prima di essere adattati o orchestrati dagli altri layer. In questo senso, il domain layer rappresenta la parte più stabile e più concettualmente densa dell'intera solution.

9.1.4. CoreService.Infrastructure

CoreService.Infrastructure raccoglie i dettagli tecnici necessari a rendere operativo il modello definito dai layer superiori. Esso include i meccanismi di persistenza, le implementazioni concrete delle interfacce applicative, l'accesso ai database di dominio, l'integrazione con servizi esterni o di supporto e i componenti tecnici che il Core Service utilizza per realizzare il proprio comportamento effettivo.

In questo progetto trovano posto i DbContext o le componenti equivalenti per l'accesso alla persistenza MySQL, le repository o i servizi di accesso ai dati, le implementazioni

legate a SMTP, ai meccanismi di emissione e revoca dei token, alle verifiche infrastrutturali di health, alla gestione delle notifiche pianificate e agli elementi tecnici necessari ai pagamenti simulati o alle integrazioni di supporto. Il layer Infrastructure non definisce il significato del dominio, ma fornisce i mezzi concreti attraverso cui esso viene persistito, recuperato e reso operativo.

La sua collocazione come progetto separato consente di confinare le dipendenze più esposte alla variabilità tecnologica: provider database, servizi di posta, configurazioni esterne, serializzazioni, accesso alle risorse di ambiente, meccanismi di scheduling tecnico. In questo modo, il Core Service mantiene più netta la distinzione tra **regola di business** e **dettaglio implementativo**.

9.2. LAYER API

Il **layer API** ha la responsabilità di esporre le capacità del Core Service tramite interfacce HTTP coerenti con il modello REST adottato dalla piattaforma. Questo livello rappresenta il punto in cui le richieste provenienti dal gateway vengono ricevute, interpretate e trasformate in invocazioni del layer applicativo.

La progettazione di questo layer segue una logica di **adattamento del protocollo**. I controller o gli endpoint non devono contenere la logica di business sostanziale del sistema, ma devono:

- ricevere input strutturati;
- applicare le validazioni preliminari necessarie al contesto HTTP;
- recuperare il contesto dell'utente autenticato quando richiesto;
- invocare il caso d'uso appropriato del layer Application;
- tradurre l'esito in una risposta HTTP coerente.

In questo livello trovano naturale collocazione aspetti quali la definizione delle rotte, la segmentazione degli endpoint per dominio funzionale, la modellazione dei codici di risposta, la trasformazione degli errori applicativi in status HTTP appropriati e l'integrazione con i meccanismi di autenticazione e autorizzazione propri del servizio.

Il layer API è anche il punto in cui il Core Service definisce la propria **superficie pubblica**. Gli endpoint relativi ad auth, registry, scheduling, clinical, payments, notifications, audit ed health fanno parte di questo strato e ne riflettono la funzione principale: rendere disponibile il comportamento del servizio in modo ordinato e interoperabile.

Una caratteristica importante di questo livello è la sua **dipendenza dal layer Application**, e non dai dettagli infrastrutturali in forma diretta. Questo permette di mantenere i controller più snelli, più leggibili e più facili da mantenere, evitando che essi diventino il luogo in cui si mescolano accesso ai dati, logica di business e dettagli tecnici.

9.3. LAYER APPLICATION

Il **layer Application** rappresenta il cuore operativo dei casi d'uso del Core Service. Esso coordina le operazioni che il sistema deve eseguire in risposta a una richiesta, trasformando input strutturati in azioni coerenti con il dominio e combinando, quando necessario, più componenti logiche o più dipendenze astratte.

La funzione principale di questo layer è l'**orchestrazione**. Esso non definisce il trasporto HTTP e non si occupa direttamente della persistenza concreta, ma decide come un determinato caso d'uso debba essere eseguito: quali entità o servizi di dominio coinvolgere, quali controlli applicativi applicare, quali dati recuperare, quali esiti restituire e quali effetti collaterali tecnici attivare tramite interfacce astratte.

In una piattaforma come questa, il layer Application è essenziale per evitare due derive architetture opposte: da un lato controller troppo intelligenti, che inglobano la logica di business; dall'altro infrastruttura troppo dominante, nella quale il comportamento del sistema emerge come semplice successione di operazioni su database. Il livello applicativo permette invece di esprimere i flussi in termini leggibili e aderenti ai casi d'uso reali della piattaforma.

All'interno di questo layer è naturale collocare:

- **DTO** di input e output;
- comandi, query o richieste applicative;
- servizi applicativi per i diversi domini;
- interfacce che astraggono repository o servizi tecnici;
- validazioni applicative legate al caso d'uso;
- regole di orchestrazione che combinano più elementi del dominio.

La centralità di questo livello emerge soprattutto nei flussi più articolati, come attivazione account, reset password, booking, reschedule, cancel, check-in, apertura encounter, firma e pubblicazione del referto, gestione dei payment intent, riconciliazione degli esiti, generazione di notifiche e produzione degli eventi di audit. In tutti questi casi il sistema non esegue una singola operazione tecnica isolata, ma coordina una sequenza di passi logicamente coerenti, e tale coordinamento appartiene propriamente al layer Application.

9.4. LAYER DOMAIN

Il **layer Domain** è il livello in cui il sistema esprime la propria semantica di business in forma indipendente dal protocollo HTTP e dai dettagli tecnici dell'infrastruttura. La sua funzione non è esporre servizi né orchestrare flussi complessi, ma rappresentare i concetti fondamentali della piattaforma e le regole che ne determinano il comportamento essenziale.

In questo layer trovano posto entità e costrutti che descrivono gli oggetti centrali del sistema: account, utenti, ruoli, profili, deleghe, consensi, prestazioni, slot, appuntamenti, encounter, anamnesi, parametri vitali, ordini, esecuzioni, referti, intenti di pagamento, transazioni, notifiche, eventi di audit. Tali elementi non sono semplici contenitori di dati, ma strutture dotate di significato e, quando necessario, di regole di validità e transizione.

Il layer Domain è anche il luogo naturale per esprimere gli **stati** e le **regole invarianti** del sistema. La distinzione tra un account non attivato e uno attivo, tra un appuntamento prenotato e uno sottoposto a check-in, tra un referto in bozza e uno pubblicato, tra una delega valida e una non utilizzabile, appartiene alla logica del dominio e non dovrebbe dipendere dal modo in cui tali informazioni vengono esposte o persistite.

Una progettazione corretta del dominio consente di mantenere il sistema più coerente nel tempo. Quando le regole di business sono espresse in modo chiaro a questo livello, diventa più difficile che controller, servizi applicativi o componenti infrastrutturali introducano comportamenti incoerenti o duplicati. Il domain layer agisce quindi come **centro di gravità semantico** dell'intera piattaforma.

Un ulteriore vantaggio di questa impostazione è la possibilità di mantenere il modello di dominio relativamente stabile anche quando cambiano dettagli di implementazione. Tecnologie di persistenza, meccanismi di trasporto o organizzazione dell'infrastruttura possono evolvere, ma i concetti fondamentali del sistema — paziente, prenotazione, encounter, pagamento, notifica — restano riconoscibili e governati da regole coerenti.

9.5. LAYER INFRASTRUCTURE

Il **layer Infrastructure** ha la funzione di tradurre in componenti concreti le astrazioni richieste dal layer Application e di supportare il funzionamento del dominio tramite servizi tecnici, persistenza e integrazioni di basso livello. Esso rappresenta il punto in cui il sistema entra in contatto con il database, con i provider tecnici e con le risorse esterne o di supporto necessarie all'esecuzione.

Dal punto di vista della persistenza, questo layer gestisce l'accesso ai diversi schemi MySQL del sistema e organizza i meccanismi necessari a leggere, scrivere e aggiornare i dati di dominio. Qui si collocano i contesti di accesso ai dati, le implementazioni concrete delle repository o degli adapter equivalenti, le configurazioni di mapping e, più in generale, le scelte che legano il modello applicativo a una tecnologia persistente specifica.

Il layer Infrastructure comprende anche le implementazioni relative a:

- emissione, verifica e revoca dei token dove richiesto dal Core Service;
- invio delle e-mail tramite SMTP;
- polling o dispatching delle notifiche pianificate;
- simulazione del provider di pagamento e gestione tecnica dei relativi esiti;
- health check verso risorse esterne o database;
- lettura delle configurazioni e integrazione con l'ambiente runtime.

Questa collocazione è architetturealmente corretta perché tali elementi, pur essendo indispensabili, non definiscono il significato del dominio. Il fatto che una notifica venga inviata via SMTP, che i dati siano salvati in MySQL o che il pagamento venga modellato tramite una simulazione provider appartiene ai **dettagli tecnici dell'implementazione**, non alla semantica del business.

Un aspetto importante del layer Infrastructure è la sua **sostituibilità relativa**. Organizzando le dipendenze tecniche in un progetto dedicato e collegandole al resto del sistema tramite interfacce o contratti definiti a livelli superiori, la piattaforma riduce l'impatto delle scelte infrastrutturali sul modello applicativo complessivo. Questa proprietà non elimina il costo dei cambiamenti tecnologici, ma ne rende più chiara la localizzazione e più contenuta la propagazione.

9.6. MOTIVAZIONI DELLA SEPARAZIONE IN LAYER

La separazione del Core Service in layer distinti risponde a una motivazione architettuale precisa: evitare che le responsabilità del sistema vengano fuse in un'unica massa di codice difficile da comprendere, evolvere e controllare. In una piattaforma che governa processi sensibili e domini differenti, la leggibilità strutturale non è un beneficio secondario, ma una condizione necessaria per la qualità del software.

Una prima motivazione riguarda la **manutenibilità**. Quando trasporto HTTP, orchestrazione dei casi d'uso, modello di dominio e dettagli di persistenza sono separati, ogni parte del sistema può essere compresa più facilmente nel proprio contesto. I controller restano focalizzati sull'esposizione delle API, i servizi applicativi sui flussi, il dominio sulle regole di business, l'infrastruttura sulle integrazioni tecniche. Questa distinzione riduce il rischio che un cambiamento in un'area produca effetti collaterali incontrollati in tutte le altre.

Una seconda motivazione concerne la **chiarezza delle dipendenze**. La separazione in layer consente di rendere visibile il verso corretto delle relazioni tra le componenti: il layer API dipende dall'Application, l'Application usa il Domain e interagisce con l'Infrastructure tramite astrazioni, il Domain non conosce i dettagli infrastrutturali, l'Infrastructure implementa quanto necessario per sostenere i livelli superiori. Questa disciplina riduce l'accoppiamento e aiuta a preservare la coerenza dell'architettura nel tempo.

Una terza motivazione è la **protezione del dominio**. Senza una separazione esplicita, vi è il rischio che le regole di business vengano disperse tra controller, query di persistenza e componenti tecnici. La presenza di un layer Domain distinto e di un Application layer che ne orchestra l'utilizzo permette di mantenere le decisioni centrali del sistema più vicine al loro significato, invece di lasciarle emergere accidentalmente da dettagli di implementazione.

Un quarto motivo riguarda la **testabilità concettuale** e la verificabilità del comportamento. Pur senza entrare nel dettaglio delle strategie di test, un sistema organizzato in layer rende più naturale isolare i casi d'uso, verificare le regole del dominio e distinguere ciò che dipende dal database o da servizi esterni da ciò che appartiene al comportamento puro dell'applicazione.

Vi è poi una motivazione legata alla **evoluzione del sistema**. In una piattaforma destinata a crescere nei domini di business, negli endpoint esposti e nelle integrazioni tecniche, una struttura a layer offre una base più adatta all'estensione ordinata. Nuovi casi d'uso possono essere inseriti nel layer Application, nuovi endpoint nel layer API, nuove regole nel dominio, nuove integrazioni nel layer Infrastructure, senza dissolvere l'ordine complessivo del servizio.

La separazione in layer favorisce infine una maggiore **disciplina progettuale**. Essa costringe a esplicitare dove appartenga una determinata responsabilità, evitando che decisioni strutturali vengano prese in modo implicito o opportunistico. In un Core Service che rappresenta il centro della piattaforma, questa disciplina è particolarmente importante perché da essa dipendono la qualità del codice, la coerenza dell'architettura e la capacità di mantenere il sistema allineato ai propri principi fondativi.

10. MODULI APPLICATIVI DEL CORE SERVICE

10.1. AUTH & IDENTITY

Il modulo **Auth & Identity** costituisce il punto di riferimento autorevole per la gestione dell'identità digitale all'interno della piattaforma. Esso governa i flussi di registrazione degli utenti self-service, l'autenticazione, la chiusura della sessione, la verifica dello stato dei token, l'attivazione degli account, il reset delle credenziali e la restituzione del contesto identitario dell'utente autenticato. La sua centralità architetturale deriva dal fatto che altri componenti della piattaforma — in particolare l'API Gateway e l'AI Assistant — si appoggiano a questo modulo per confermare la validità dell'accesso e la corretta qualificazione del soggetto che interagisce con il sistema.

Dal punto di vista progettuale, il modulo separa con chiarezza la gestione dell'identità dai restanti domini di business. L'obiettivo non è soltanto consentire il login, ma governare l'intero **ciclo di vita dell'accesso**, dalla creazione controllata dell'utenza fino alla sua eventuale invalidazione, mantenendo una relazione coerente tra utente, ruolo, token, stato dell'account e profilo applicativo. Tale approccio permette di preservare la consistenza del modello autorizzativo e di impedire che i singoli moduli applicativi debbano introdurre logiche autonome di gestione della sessione.

Il modulo è costruito in modo da supportare sia i flussi self-service, rivolti a **Patient e Delegate**, sia i flussi di utilizzo da parte degli altri attori del sistema una volta che l'account sia stato creato e reso operativo. Le informazioni persistenti associate a questo dominio comprendono utenti, ruoli, token di attivazione, token di reset password e token revocati, a conferma del fatto che l'identità viene trattata come un dominio completo e non come semplice funzione accessoria.

10.1.1. EMISSIONE JWT

L'emissione del **JSON Web Token** costituisce il cuore del processo di autenticazione e rappresenta il meccanismo attraverso cui il sistema materializza il contesto di accesso dell'utente autenticato. A valle di un login andato a buon fine, il modulo genera un token che incorpora l'identità del soggetto, il suo ruolo applicativo e gli elementi necessari a permettere la verifica della sessione nei punti della piattaforma deputati al controllo dell'accesso.

Dal punto di vista progettuale, l'emissione del JWT assolve una duplice funzione. Da un lato, rende disponibile al client un artefatto di autenticazione compatibile con l'uso stateless delle API protette; dall'altro, definisce un **contratto di identità** comune per tutti i componenti che devono interpretare il contesto dell'utente, inclusi gateway, Core Service e AI Assistant. Il token non è quindi un semplice dettaglio tecnico del login, ma un elemento strutturale dell'intero modello di sicurezza.

Il processo di emissione è strettamente legato alla validazione delle credenziali e allo stato dell'account. Un utente può ricevere un token soltanto quando il sistema ne riconosce la correttezza identificativa e l'abilitazione all'uso della piattaforma. In questo modo il JWT non rappresenta soltanto un mezzo di trasporto dell'identità, ma il risultato di una decisione applicativa coerente con il ciclo di vita dell'account.

Un ulteriore aspetto rilevante riguarda il fatto che il token emesso dal modulo viene poi utilizzato come riferimento trasversale per l'intero ecosistema backend. Il gateway lo verifica per le richieste protette, il Core Service lo interpreta per determinare il contesto dell'utente, l'AI Assistant lo sottopone a introspection e verifica del ruolo. La corretta progettazione dell'emissione JWT è quindi una condizione essenziale per la consistenza dell'intera piattaforma.

10.1.2. REVOCA TOKEN

La **revoca del token** è la controparte necessaria dell'emissione JWT e risponde all'esigenza di evitare che un token precedentemente rilasciato venga considerato valido fino alla sola scadenza temporale. Il modulo Auth & Identity introduce infatti una gestione esplicita dei token revocati, permettendo di interrompere la validità di un contesto di accesso anche prima del termine naturale della sua durata.

Questo comportamento è particolarmente rilevante in corrispondenza del **logout** e, più in generale, in tutti gli scenari in cui la piattaforma deve impedire il riutilizzo di un token ancora formalmente integro ma sostanzialmente non più autorizzato. Il modello adottato evita quindi una visione puramente passiva del JWT e attribuisce al Core Service il ruolo di **autorità sostanziale** sul suo stato effettivo.

Dal punto di vista architetturale, la revoca è importante perché rende possibile una vera **token introspection** a livello di sistema. Il gateway e gli altri componenti non si limitano a verificare struttura, firma e scadenza del token, ma possono consultare il dominio identità per comprendere se quel token sia ancora ammesso all'uso. Ciò rafforza la sicurezza dell'intera piattaforma e riduce il rischio di accessi impropri dovuti a sessioni invalidate ma non ancora scadute.

Il fatto che la revoca venga modellata come parte integrante del modulo Auth & Identity dimostra che il sistema non considera l'autenticazione come evento puntuale, ma come processo governato nel tempo, con possibilità di emissione, validazione e invalidazione controllata.

10.1.3. ME ENDPOINT

L'endpoint **Me** ha la funzione di restituire il contesto identitario dell'utente autenticato e rappresenta un servizio fondamentale di raccordo tra backend e frontend. Attraverso tale endpoint, la piattaforma consente al client di recuperare in modo affidabile le informazioni essenziali relative alla sessione corrente, incluse identità, ruolo e stato operativo del soggetto autenticato.

Dal punto di vista progettuale, questo endpoint riduce la necessità di ricostruire il contesto utente esclusivamente a partire dai dati locali disponibili nel browser. Il frontend può così verificare in modo autorevole chi sia l'utente corrente, quale ruolo possieda e quale area applicativa debba essergli resa disponibile. L'endpoint Me svolge quindi una funzione di **riallineamento** tra contesto di sessione lato client e verità applicativa lato server.

La presenza di questo endpoint contribuisce anche alla pulizia architetturale del frontend. Invece di basarsi su assunzioni implicite o su parsing locale non autorevole del token, il client dispone di un punto dedicato per ottenere il proprio profilo operativo essenziale. Ciò

migliora la robustezza del comportamento lato interfaccia e consente di mantenere più ordinata la logica di bootstrapping delle aree protette del sistema.

10.1.4. ATTIVAZIONE ACCOUNT E RESET PASSWORD

Il modulo Auth & Identity governa in modo esplicito anche i flussi di **attivazione account** e **reset password**, trattandoli come processi distinti ma strettamente collegati al dominio dell'identità. Entrambi i casi condividono una struttura comune: generazione di un token dedicato, persistenza del suo stato, invio di una comunicazione tramite e-mail e completamento del flusso da parte dell'utente mediante una procedura controllata.

L'**attivazione account** consente di trasformare una registrazione formalmente acquisita in un'utenza effettivamente abilitata all'uso della piattaforma. Il sistema emette un token di attivazione, ne governa la validità e veicola il collegamento necessario al completamento della procedura. Questa scelta evita che account appena creati diventino immediatamente operativi senza una fase di conferma esplicita.

Il **reset password** segue una logica simile, ma con finalità diverse: consentire il recupero dell'accesso in caso di smarrimento o inutilizzabilità delle credenziali. Anche in questo caso il sistema produce un token dedicato, lo collega all'account corretto, invia il messaggio all'indirizzo di riferimento e permette poi l'impostazione di una nuova password all'interno di un flusso verificato.

La presenza di interfacce dedicate per l'invio delle e-mail di attivazione e di reset indica una progettazione pulita del modulo: la logica applicativa rimane nel Core Service, mentre il canale tecnico di consegna è delegato a componenti infrastrutturali specializzati. Ciò mantiene distinta la responsabilità di business dalla modalità concreta con cui la comunicazione viene recapitata.

10.2. REGISTRY

Il modulo **Registry** governa la rappresentazione anagrafica e relazionale dei soggetti presenti nel sistema. La sua funzione non è limitata alla mera memorizzazione di dati personali, ma comprende la gestione strutturata di **pazienti, delegati, clinici, profili applicativi, deleghe e consensi**, costituendo così il punto di riferimento per l'identificazione operativa dei soggetti che prendono parte ai flussi della piattaforma.

Dal punto di vista progettuale, il Registry svolge una funzione di fondazione del resto del sistema. Scheduling, Clinical, Payments ed Events possono operare correttamente solo se il dominio anagrafico e relazionale è coerente: pazienti correttamente identificati, clinici associati al proprio ruolo, deleghe formalizzate, consensi disponibili e profili strutturati. Per questa ragione il modulo non è un archivio passivo, ma un vero dominio applicativo con servizi dedicati e regole di accesso specifiche.

Un aspetto particolarmente importante di questo modulo è la coesistenza di due prospettive operative differenti. Da un lato, esistono servizi di **amministrazione centralizzata**, in capo all'Admin, per la creazione e manutenzione di profili e relazioni. Dall'altro, esistono servizi di **self-profile management**, attraverso i quali Patient, Delegate e Clinician possono consultare e aggiornare il proprio profilo entro i limiti consentiti. Questa distinzione rende il Registry aderente a un contesto reale nel quale non tutta l'informazione anagrafica nasce o si modifica nello stesso modo.

10.2.1. GESTIONE PAZIENTI

La gestione dei **pazienti** comprende la creazione, consultazione e aggiornamento del profilo paziente, sia in modalità amministrata sia in modalità self-service. Il modulo mette a disposizione servizi che consentono all'Admin di creare o aggiornare il profilo del paziente e, parallelamente, al Patient autenticato di recuperare e modificare le informazioni di propria competenza.

Questa distinzione riflette una progettazione accurata del dominio. Il paziente non è soltanto un utente autenticato del sistema, ma anche un soggetto anagrafico con informazioni che devono poter essere governate in modo coerente dai processi amministrativi e dai flussi personali. La presenza di servizi differenti per il profilo amministrato e per il profilo self-service evita di confondere le responsabilità e rende più chiari i confini tra autonomia dell'utente finale e governo organizzativo della piattaforma.

Il modulo Registry collega, inoltre, il paziente ad altri elementi critici del sistema, come deleghe, consensi, appuntamenti, pre-triage, referti e pagamenti. La correttezza della sua rappresentazione anagrafica incide quindi su una parte molto ampia dei flussi della piattaforma.

10.2.2. GESTIONE DELEGATI

La gestione dei **delegati** introduce nel sistema un livello di complessità ulteriore rispetto alla semplice anagrafica del paziente. Il Delegate è infatti un soggetto dotato di propria identità, proprio account e proprio profilo, ma al tempo stesso collegato operativamente a uno o più assistiti attraverso relazioni di delega formalizzate.

Il modulo Registry gestisce questo ruolo sia sotto il profilo anagrafico, mediante servizi di creazione e aggiornamento del profilo delegato, sia sotto il profilo relazionale, tramite la consultazione delle deleghe associate. Dal punto di vista progettuale, il delegato non viene trattato come una variante del paziente, ma come un'entità con semantica distinta, le cui capacità operative dipendono da una relazione valida con l'assistito.

Questa scelta è importante perché consente di modellare in modo corretto la rappresentanza. Le operazioni che il Delegate può svolgere nei moduli Scheduling, Clinical, Payments ed Events sono infatti subordinate alla verifica della delega e del relativo scope; tale comportamento sarebbe molto più difficile da governare se il delegato non fosse rappresentato come attore autonomo nel Registry.

10.2.3. GESTIONE CLINICI

La gestione dei **clinici** comprende la creazione amministrata e la manutenzione del profilo professionale del soggetto appartenente al ruolo Clinician. In questo caso il dominio Registry deve rappresentare non soltanto l'identità dell'utente, ma anche gli attributi che lo rendono utilizzabile nei flussi di scheduling e clinical, come il profilo professionale e le informazioni necessarie alla contestualizzazione della sua attività nel sistema.

L'Admin dispone di servizi per creare il clinico come utenza di sistema e per valorizzarne o aggiornarne il profilo; parallelamente, il Clinician può consultare e aggiornare il proprio profilo tramite il relativo servizio self-service. Questa doppia prospettiva mantiene distinto il piano della **provisioning amministrativo** da quello della **manutenzione del profilo personale**, evitando ambiguità e preservando la coerenza del modello.

Il corretto governo del clinico nel Registry è essenziale per la qualità di altri moduli: la disponibilità degli slot, l'agenda del professionista, la corretta associazione degli encounter e la tracciabilità delle operazioni cliniche dipendono infatti dalla presenza di una rappresentazione affidabile del soggetto professionale.

10.2.4. PROFILI, DELEGHE E CONSENSI

Profili, deleghe e consensi costituiscono la parte più relazionale e sensibile del modulo Registry. I **profili** permettono di strutturare le informazioni operative dei diversi tipi di utente; le **deleghe** formalizzano i rapporti di rappresentanza tra paziente e delegato; i **consensi** registrano elementi necessari alla completezza e alla legittimità di alcuni flussi applicativi.

Dal punto di vista progettuale, la gestione delle deleghe è particolarmente significativa. Il modulo non si limita a creare un legame statico tra due soggetti, ma governa anche lo **stato** della delega, il suo **scope** operativo e la possibilità di aggiornarne i permessi o il ciclo di vita. Questa ricchezza semantica è necessaria perché molte operazioni delegate — prenotazioni, pagamenti, consultazione notifiche, accesso ai referti — devono essere permesse solo quando la relazione è valida e coerente con il tipo di azione richiesta.

La presenza di un **DelegationAccessService** rafforza ulteriormente questa impostazione, poiché introduce un punto applicativo dedicato alla verifica dell'effettiva legittimazione del delegato rispetto al paziente e allo scope richiesto. Il Registry diventa così non soltanto il luogo in cui le deleghe sono memorizzate, ma anche il dominio che rende possibile il loro uso sicuro nei moduli applicativi downstream.

I consensi completano questo quadro fornendo un ulteriore livello di formalizzazione. La loro gestione rende il Registry un modulo non solo anagrafico, ma anche **abilitativo**, nel senso che contribuisce a determinare quando il sistema possa considerare sufficientemente completo il contesto personale e relazionale di un soggetto.

10.3. SCHEDULING

Il modulo **Scheduling** governa la dimensione temporale e organizzativa della piattaforma. Esso rappresenta il dominio in cui vengono modellate disponibilità, slot, calendari clinici, appuntamenti e transizioni di stato associate al ciclo operativo della prenotazione. Il suo ruolo è centrale perché costituisce il punto di raccordo tra catalogo prestazioni, attori coinvolti, processi amministrativi e successivo percorso clinico.

Dal punto di vista progettuale, Scheduling non si limita a memorizzare orari, ma organizza una vera semantica di flusso. Disponibilità e slot definiscono il potenziale organizzativo; l'appuntamento rappresenta la sua concretizzazione in relazione a un paziente, a una prestazione e a un momento temporale preciso; gli stati dell'appuntamento governano l'evoluzione del processo dall'avvenuta prenotazione fino a eventi come check-in, completamento, annullamento o no-show.

Il modulo è costruito in modo da supportare prospettive diverse. L'Admin dispone di strumenti di governo generale su disponibilità e appuntamenti; il Patient può interagire con il proprio ciclo di prenotazione; il Clinician accede alla propria agenda; il Delegate opera per conto dell'assistito attraverso verifiche di delega. Questa pluralità di punti di vista è

coordinata da servizi che mantengono un'unica verità di dominio sul calendario e sullo stato delle prenotazioni.

10.3.1. DISPONIBILITA' E SLOT

La gestione di **disponibilità** e **slot** costituisce la base organizzativa del modulo Scheduling. Le disponibilità definiscono gli intervalli o le finestre operative in cui una risorsa clinica può erogare prestazioni; gli slot rappresentano la granularizzazione concreta di tale disponibilità in unità prenotabili dal sistema.

Dal punto di vista applicativo, il modulo permette all'Admin di creare slot per un clinico, consultarli e aggiornarne lo stato. Questa progettazione evidenzia che la disponibilità non è trattata come semplice configurazione statica, ma come entità operativa soggetta a ciclo di vita, controllo amministrativo e impatto diretto sulla prenotabilità.

La presenza di uno specifico servizio dedicato alla disponibilità conferma che il sistema considera questa parte come dominio autonomo e non come semplice dettaglio interno della prenotazione. Ciò permette di mantenere più chiaro il confine tra **offerta temporale** e **appuntamento effettivamente prenotato**.

10.3.2. AGENDA DEL CLINICO

L'**agenda del clinico** è la proiezione professionale del dominio Scheduling e permette al Clinician di visualizzare gli appuntamenti a lui pertinenti in una finestra temporale coerente con la propria attività. A differenza delle viste amministrative, l'agenda non ha funzione di supervisione globale, ma di supporto operativo diretto alla presa in carico del paziente.

Questo aspetto giustifica la presenza di un servizio dedicato al recupero dell'agenda del professionista. Il modulo Scheduling non espone infatti soltanto una visione centralizzata delle prenotazioni, ma anche una vista filtrata e contestualizzata in funzione del soggetto clinico autenticato. Tale distinzione è importante per mantenere separati il dominio organizzativo generale e il dominio operativo del professionista.

L'agenda è inoltre il punto di accesso naturale a flussi successivi del sistema, in particolare all'avvio dell'encounter clinico. La sua correttezza influisce quindi non solo sulla pianificazione, ma anche sulla coerenza del passaggio tra scheduling e clinical.

10.3.3. PRENOTAZIONI E STATI DEGLI APPUNTAMENTI

La **prenotazione** costituisce il cuore del modulo Scheduling. Il sistema supporta la creazione di appuntamenti da parte del paziente, dell'amministratore e del delegato per conto dell'assistito, nonché la consultazione del relativo elenco e dei dettagli associati. Questa pluralità di canali di booking è governata da una logica di dominio unica, che preserva la coerenza del modello degli appuntamenti indipendentemente dall'attore che avvia il flusso.

Un elemento centrale di questa progettazione è la presenza di **stati espliciti dell'appuntamento**. Il dominio distingue infatti momenti diversi del ciclo di vita, come prenotazione, check-in, completamento, annullamento e no-show. La presenza di tali stati evita che il sistema tratti l'appuntamento come semplice entità binaria esistente/non esistente, e consente invece di governarne il comportamento in modo più rigoroso e aderente ai processi reali.

La disponibilità di servizi di **cancel** e **reschedule** per attori differenti conferma che il modulo Scheduling non si limita a creare appuntamenti, ma governa anche le relative transizioni operative. Ogni modifica rilevante produce effetti sia sullo stato della prenotazione sia sui moduli Events e, indirettamente, sui flussi successivi della piattaforma.

10.3.4. CHECK-IN E NO-SHOW

Il **check-in** e il **no-show** rappresentano due passaggi cruciali del ciclo di vita dell'appuntamento, poiché collegano la semplice esistenza di una prenotazione alla sua effettiva esecuzione o mancata esecuzione. Il modulo Scheduling tratta entrambi come transizioni di stato esplicite, governate da servizi dedicati e attivabili nel contesto amministrativo appropriato.

Il check-in segnala che il paziente si è presentato e che il flusso può proseguire verso la presa in carico clinica; il no-show formalizza invece la mancata presentazione e consente al sistema di chiudere il ciclo della prenotazione in modo coerente. Questa modellazione è importante perché impedisce che il comportamento dell'appuntamento venga lasciato a convenzioni implicite o a interpretazioni manuali esterne al sistema.

Dal punto di vista architetturale, tali operazioni mostrano come Scheduling sia un modulo orientato non solo alla pianificazione, ma al **governo processuale** dell'evento prenotato lungo tutto il suo ciclo operativo.

10.4. CLINICAL

Il modulo **Clinical** rappresenta il dominio applicativo dedicato alla gestione del contenuto sanitario del sistema. Esso comprende il catalogo prestazioni dal punto di vista clinico-funzionale, il pre-triage, la gestione degli encounter, la registrazione di anamnesi e parametri vitali, la creazione di ordini, la documentazione delle esecuzioni e il workflow del referto. Si tratta del modulo più sensibile sotto il profilo della semantica di business, poiché in esso prende forma la documentazione clinica dell'episodio assistenziale.

La progettazione di questo modulo evidenzia una chiara distinzione tra **governo dell'episodio clinico** e **consultazione lato utente finale**. Da un lato esistono servizi dedicati al Clinician per l'intero percorso di presa in carico e documentazione; dall'altro, esistono servizi dedicati al Patient e al Delegate per la consultazione dei referti pubblicati e per la compilazione o lettura del pre-triage. Tale articolazione consente di mantenere una netta separazione tra produzione del contenuto clinico e sua successiva fruizione entro i limiti autorizzati.

10.4.1. CATALOGO PRESTAZIONI

Il **catalogo prestazioni** rappresenta l'interfaccia tra organizzazione dell'offerta sanitaria e moduli di scheduling e clinical. La presenza di un servizio dedicato sia lato amministrativo sia lato consultazione conferma che il catalogo non è un dato statico, ma un'entità governata applicativamente, con possibilità di creazione, aggiornamento, ricerca e controllo della visibilità.

Dal punto di vista progettuale, il catalogo ha una duplice funzione. In primo luogo, supporta i flussi di prenotazione, offrendo una base consistente per la scelta della prestazione da parte del paziente o dell'operatore. In secondo luogo, contribuisce alla

contestualizzazione del percorso clinico, poiché la prestazione selezionata partecipa alla definizione dell'appuntamento e dell'encounter successivo.

Il fatto che il sistema distingua un **AdminCatalogService** e un **CatalogService** mostra una separazione corretta tra amministrazione del dominio e consultazione operativa, preservando sia la governance del catalogo sia la sua fruibilità nei flussi applicativi.

10.4.2. PRE-TRIAGE

Il **pre-triage** è modellato come parte integrante del dominio Clinical e non come semplice raccolta documentale esterna. Questa scelta è coerente con il suo ruolo: raccogliere informazioni preliminari legate a uno specifico appuntamento, da rendere poi disponibili al Clinician nel contesto del percorso assistenziale.

La piattaforma distingue tra servizi rivolti al **Patient** e al **Delegate** per la compilazione o consultazione del questionario associato all'appuntamento, e servizi rivolti al **Clinician** per la lettura del pre-triage nel contesto professionale appropriato. Tale articolazione mantiene separata la fase di immissione dei dati dalla fase di utilizzo clinico, pur preservando la continuità del flusso.

Dal punto di vista architetturale, il pre-triage dimostra la capacità del modulo Clinical di integrare contenuti prodotti da attori diversi in un'unica catena di significato clinico, senza rinunciare ai necessari controlli di contesto e autorizzazione.

10.4.3. ENCOUNTER CLINICI

Gli **encounter clinici** costituiscono il centro operativo del modulo Clinical. Attraverso di essi il sistema struttura l'episodio assistenziale come contenitore logico delle informazioni prodotte dal Clinician: anamnesi, parametri vitali, ordini clinici, esecuzioni e referto. L'encounter non è quindi un semplice record di appoggio, ma l'unità semantica che rende coerente la documentazione del percorso sanitario.

Il servizio clinico rivolto al professionista permette di elencare gli encounter pertinenti, recuperarne il dettaglio, avviarne uno a partire dal contesto corretto e arricchirlo progressivamente con i diversi elementi della documentazione clinica. Questa progressione conferma una progettazione orientata al **workflow**, nella quale l'episodio assistenziale viene costruito passo dopo passo e non come inserimento monolitico di un unico documento finale.

La presenza di servizi dedicati alla registrazione di anamnesi, parametri vitali, ordini ed esecuzioni mostra inoltre che il modulo Clinical è organizzato per **componenti documentali specializzate**, ciascuna con significato proprio all'interno dell'encounter. Tale scelta migliora la granularità del modello e rende più leggibile la struttura della documentazione sanitaria.

10.4.4. WORKFLOW DEL REFERTO

Il **workflow del referto** è modellato in modo esplicito e distinto rispetto al resto dell'encounter. La piattaforma introduce infatti un servizio dedicato alla gestione del ciclo di vita del referto, distinguendo chiaramente tra **bozza**, **firma** e **pubblicazione**. Questa scelta è architetturealmente corretta perché il referto è il documento finale del percorso

clinico e richiede un trattamento più rigoroso rispetto a una semplice nota testuale o a un aggiornamento generico dell'episodio.

La possibilità di effettuare operazioni di **upsert**, **sign** e **publish** mostra che il sistema riconosce al referto uno stato evolutivo autonomo. Tale approccio consente di mantenere distinto il momento in cui il contenuto viene redatto, quello in cui viene consolidato dal professionista e quello in cui diventa effettivamente disponibile per la consultazione lato paziente o delegato.

Dal punto di vista semantico, il workflow del referto rappresenta il passaggio in cui il modulo Clinical trasforma il contenuto documentale interno all'encounter in informazione clinica formalizzata e distribuibile secondo le regole del sistema.

10.5. PAYMENTS

Il modulo **Payments** governa l'intero dominio economico della piattaforma, modellando **payment intent**, **transazioni**, **checkout workflow**, riconciliazione amministrativa ed esiti provenienti dal provider simulato. La sua progettazione risponde all'esigenza di trattare il pagamento non come evento contabile isolato, ma come parte integrante del ciclo operativo associato all'appuntamento e alla posizione del paziente.

Dal punto di vista architetturale, il modulo è costruito per supportare due prospettive differenti ma complementari: da un lato la visione **self-service** del paziente o del delegato, che può creare e processare payment intent nel contesto dell'appuntamento; dall'altro la visione **amministrativa**, che consente registrazione dei pagamenti in sede, consultazione degli intenti, riconciliazione e simulazione degli esiti provider. Tale struttura garantisce coerenza di dominio pur esponendo interfacce operative diverse ai vari attori.

10.5.1. PAYMENT INTENTS

Il **payment intent** rappresenta l'entità che formalizza l'intenzione di pagamento associata a un appuntamento. La sua introduzione è una scelta progettuale importante, perché separa il momento in cui il sistema decide che una determinata posizione economica deve essere pagata dal momento in cui la transazione viene effettivamente elaborata o conclusa.

Questa modellazione consente al sistema di governare in modo più preciso lo stato del flusso economico. Un payment intent può essere creato in modo idempotente per un appuntamento, consultato, associato a una o più transazioni e aggiornato sulla base degli esiti successivi. L'introduzione di un'entità autonoma evita che il pagamento venga ridotto a semplice flag di stato sull'appuntamento, migliorando la qualità del modello e la sua auditabilità.

10.5.2. TRANSAZIONI

Le **transazioni** rappresentano la concretizzazione operativa del dominio dei pagamenti. Se il payment intent formalizza l'intenzione, la transazione documenta l'evento tecnico o economico associato al tentativo di pagamento, al suo esito o alla sua evoluzione.

Dal punto di vista progettuale, separare transazioni e intenti permette di mantenere più leggibile il rapporto tra piano logico e piano operativo del pagamento. Una stessa intenzione economica può infatti generare eventi tecnici differenti, e il sistema deve poterli

rappresentare senza perdere il collegamento con il contesto dell'appuntamento e con il soggetto coinvolto.

La consultazione delle transazioni per un determinato intent, disponibile sul lato amministrativo, rafforza ulteriormente questa impostazione e mostra che il modulo Payments è progettato per offrire anche capacità di analisi e verifica, non soltanto di elaborazione.

10.5.3. CHECKOUT WORKFLOW

Il **checkout workflow** rappresenta il percorso applicativo attraverso cui un paziente o un delegato può creare un payment intent per un appuntamento e avviare il processo di pagamento. La presenza di un servizio dedicato a questo workflow evidenzia una scelta architetturale corretta: il pagamento non viene gestito come semplice operazione CRUD, ma come flusso coordinato che combina verifica del contesto, creazione dell'intento, processamento della richiesta e aggiornamento degli stati associati.

Questa orchestrazione è particolarmente importante perché il dominio economico interagisce con scheduling, notifiche ed eventi. Un checkout riuscito o fallito non ha valore soltanto all'interno del modulo Payments, ma produce effetti che devono essere interpretati e propagati in modo coerente dal resto della piattaforma.

La riusabilità di questo workflow anche nel contesto del Delegate, tramite verifica della delega e poi invocazione dei servizi di pagamento del paziente assistito, dimostra la buona separazione tra controllo della legittimazione e logica del pagamento vero e proprio.

10.5.4. PROVIDER SIMULATO E WEBHOOK

La presenza di un **provider simulato** e di un servizio dedicato alla gestione del **webhook** consente alla piattaforma di modellare il comportamento di un'integrazione di pagamento esterna senza dipendere da un sistema reale di terze parti. Questa scelta è tecnicamente molto significativa, perché permette di testare e documentare l'intero flusso economico — intent, transazioni, esiti, riconciliazione e notifiche — mantenendo il controllo completo dell'ambiente di esecuzione.

Il webhook simulato è trattato come un vero punto di ingresso applicativo per eventi di pagamento, e il relativo servizio si occupa di interpretare la richiesta ricevuta e aggiornare il dominio economico in modo coerente. Il modulo può così distinguere tra creazione dell'intento, processamento iniziale e conferma successiva dell'esito, mantenendo una semantica aderente a quella di un'integrazione reale.

Dal punto di vista architetturale, questa scelta rafforza la qualità del dominio Payments perché lo rende **event-driven** nella misura necessaria, senza introdurre dipendenze esterne incontrollate.

10.6. EVENTS

Il modulo **Events** governa i fenomeni applicativi trasversali legati a **notifiche**, **audit log**, **schedulazione delle comunicazioni** e **invio differito delle e-mail**. La sua presenza consente di raccogliere in un dominio dedicato tutto ciò che riguarda la produzione, la conservazione e la consegna di eventi informativi o di tracciamento, evitando che tali responsabilità vengano disperse nei moduli di business principali.

Dal punto di vista progettuale, Events svolge una funzione di **propagazione controllata** del comportamento del sistema. Quando un appuntamento viene prenotato, annullato o ripianificato, o quando un pagamento cambia stato, la piattaforma non si limita ad aggiornare il dominio primario coinvolto, ma produce anche notifiche, promemoria o eventi di audit. Modellare tali aspetti come dominio separato migliora la chiarezza dell'architettura e rende più leggibile il rapporto tra evento di business ed evento informativo.

10.6.1. NOTIFICHE APPLICATIVE

Le **notifiche applicative** rappresentano il meccanismo con cui il sistema informa gli utenti di eventi rilevanti, come prenotazioni, ripianificazioni, annullamenti o aggiornamenti di pagamento. Il modulo espone servizi sia per la consultazione amministrativa delle notifiche sia per la fruizione self-service da parte di Patient e Delegate, confermando che le notifiche sono trattate come vere entità applicative e non come semplice effetto collaterale non persistito.

La presenza di un **NotificationScheduler** è particolarmente significativa. Essa indica che le notifiche non vengono generate in modo estemporaneo e scollegato, ma attraverso un componente che incapsula la logica di scheduling rispetto agli eventi di business. Tale scelta migliora la coerenza del sistema e rende più semplice mantenere uniforme il comportamento notificativo tra moduli diversi.

Per il caso del Delegate, il sistema introduce inoltre un servizio specifico che combina consultazione delle notifiche con verifica della delega e dell'assistito selezionato. Ciò dimostra che il dominio Events è pienamente integrato con le regole di accesso del Registry e non opera come sottosistema disaccoppiato dal modello autorizzativo generale.

10.6.2. AUDIT LOG

L'**audit log** rappresenta la dimensione di tracciamento strutturato del sistema. Il modulo Events mette a disposizione sia un servizio di recupero delle registrazioni di audit in ottica amministrativa, sia un servizio applicativo dedicato alla produzione di tali eventi. Questa separazione tra generazione e consultazione è coerente con il fatto che l'audit deve essere sia persistito in modo rigoroso sia reso interrogabile dai soggetti autorizzati.

Dal punto di vista progettuale, l'audit non viene trattato come semplice logging tecnico. Esso è modellato come dominio applicativo, con proprie entità e proprie capacità di filtro e ricerca. Questo approccio è corretto perché le operazioni rilevanti del sistema — soprattutto in contesti amministrativi, clinici ed economici — richiedono una forma di tracciabilità che sia più forte e più semanticamente significativa rispetto al solo log infrastrutturale.

10.6.3. SCHEDULER DELLE NOTIFICHE

Lo **scheduler delle notifiche** è il componente che collega gli eventi di business ai messaggi che devono essere emessi o pianificati. Il fatto che esista un servizio dedicato a questa responsabilità mostra una progettazione pulita: la logica di “quando” e “come” una notifica debba essere creata non viene duplicata nei moduli Scheduling o Payments, ma viene centralizzata in un punto coerente del dominio Events.

Le operazioni previste dallo scheduler coprono scenari diversi, come notifica di prenotazione, annullamento, ripianificazione, pagamento riuscito o fallito, oltre alla

possibilità di programmare notifiche in-app generiche. Ciò conferma che il modulo Events non si limita a conservare notifiche, ma ne governa attivamente la produzione in relazione ai principali eventi della piattaforma.

10.6.4. INVIO E-MAIL TRAMITE BACKGROUND SERVICE

L'**invio e-mail tramite background service** rappresenta il passaggio dal piano applicativo della notifica al piano operativo della consegna tramite SMTP. Il modulo Events, insieme alle componenti infrastrutturali dedicate, supporta un modello in cui le notifiche e-mail vengono raccolte, pianificate e poi consegnate da un servizio in background separato dal ciclo sincrono della richiesta HTTP.

Questa scelta è architetturalmente corretta per diverse ragioni. In primo luogo, evita che l'invio della posta blocchi o appesantisca i flussi sincroni del sistema. In secondo luogo, permette di trattare la consegna come processo tecnico distinto dalla generazione dell'evento. In terzo luogo, rende più semplice governare stati come pending, sent o failed nel dominio delle notifiche.

Il background service dedicato al dispatch e-mail completa quindi il modulo Events trasformando le notifiche pianificate in comunicazioni effettivamente inoltrate, con piena integrazione con il dominio Registry per la risoluzione dei destinatari e con l'infrastruttura SMTP per la consegna.

11. PROGETTAZIONE DEL MODULO AI ASSISTANT

11.1. OBIETTIVO DEL SERVIZIO AI

Il modulo **AI Assistant** è stato progettato come un servizio applicativo separato, specializzato nel supporto informativo al personale clinico tramite funzionalità di *question answering* in linguaggio naturale. La sua finalità non è governare direttamente i processi clinici o sostituire i moduli centrali della piattaforma, ma offrire un canale consultivo integrato nell'ecosistema applicativo, coerente con il modello di sicurezza, con i vincoli di accesso e con l'architettura complessiva del sistema.

Dal punto di vista architetturale, il servizio AI risponde a un'esigenza precisa: introdurre una capacità di elaborazione linguistica avanzata senza contaminare il **Core Service** con dipendenze, modelli computazionali e responsabilità che appartengono a un dominio tecnico differente. La scelta di implementarlo come componente autonoma permette di mantenere una separazione netta tra gestione del business sanitario-amministrativo e supporto informativo basato su modello linguistico.

Il servizio è pensato per ricevere una domanda clinica formulata dal professionista, arricchita eventualmente da un contesto testuale e da allegati documentali, eseguire una serie di controlli preliminari di sicurezza e quindi produrre una risposta generata localmente dal modello. Il suo output è strutturato in forma JSON e contiene non soltanto la risposta, ma anche metadati relativi al modello utilizzato, ai controlli di sicurezza applicati e agli allegati processati. Questo approccio rende il servizio non solo utilizzabile dal punto di vista funzionale, ma anche **trasparente** e **tracciabile** sul piano del comportamento applicativo.

Un aspetto progettuale rilevante consiste nel fatto che il modulo AI non possiede alcuna responsabilità sul dato clinico di dominio. Esso non conserva encounter, referti, anagrafiche o flussi di pagamento, e non introduce una persistenza applicativa parallela rispetto al Core Service. La sua utilità risiede nella capacità di elaborare input in linguaggio naturale entro un perimetro fortemente controllato e con un ruolo chiaramente subordinato rispetto ai moduli centrali della piattaforma.

11.2. RUOLO CONSULTATIVO E LIMITI OPERATIVI

Il servizio AI è stato concepito come strumento **consultivo** e non come componente decisionale del sistema. Questo principio incide direttamente sia sulla progettazione del backend sia sul modo in cui il modulo viene integrato nell'esperienza del Clinician. Il servizio fornisce supporto informativo, spiegazioni strutturate e contenuti utili alla riflessione professionale, ma non assume il ruolo di autorità clinica e non sostituisce il giudizio del professionista.

La natura consultiva del modulo emerge già dalla costruzione del prompt di sistema, che indirizza il modello a fornire risposte chiare, strutturate e prudenti, richiamando buone pratiche e linee guida senza formulare diagnosi definitive né proporre terapie personalizzate. Questa scelta non è un semplice accorgimento di prompt engineering, ma un vero presidio di progettazione: il comportamento del modello viene orientato fin dall'origine verso una funzione di supporto, e non verso una delega impropria di responsabilità cliniche.

I limiti operativi del servizio sono altrettanto espliciti. Il modulo non è progettato per:

- effettuare diagnosi automatizzate;
- sostituire decisioni professionali;
- assumere ownership di dati clinici persistenti;
- costituire un canale alternativo rispetto al dominio autorizzativo del Core Service;
- elaborare liberamente contenuti identificativi del paziente senza filtri preliminari.

Questa delimitazione è fondamentale per preservare la coerenza della piattaforma. Un servizio AI inserito in un contesto sanitario deve essere progettato in modo da rimanere **tecnicamente utile** ma **funzionalmente contenuto**, evitando ambiguità sul proprio ruolo. Il modulo, pertanto, opera come componente specialistica, accessibile solo in un contesto professionale qualificato e integrata entro regole di sicurezza e validazione molto più restrittive rispetto a un semplice chatbot generalista.

11.3. ENDPOINT DI QUESTION ANSWERING

Il punto di accesso principale del servizio AI è l'endpoint di **question answering** esposto sul path `/clinicians/qa`. Tale endpoint è progettato per ricevere richieste HTTP POST provenienti dal gateway e rappresenta la superficie pubblica principale del modulo dal punto di vista applicativo.

La struttura dell'endpoint riflette una progettazione orientata alla chiarezza del contratto. L'input minimo richiesto è la **question**, cioè la domanda formulata dal Clinician, mentre il campo **context** è facoltativo e consente di arricchire la richiesta con informazioni testuali aggiuntive. Oltre a questi campi, il servizio può ricevere uno o più allegati, che vengono elaborati e trasformati in testo da integrare nel contesto del prompt.

La risposta è modellata come un payload JSON strutturato, contenente:

- il campo `answer`, che rappresenta il testo generato dal modello;
- il blocco `model`, con informazioni sul modello utilizzato e sui parametri principali di generazione;
- il blocco `safety`, che documenta il fatto che sia stato eseguito il controllo PII e che il chiamante sia associato a un certo ruolo;
- il blocco `attachments`, che restituisce metadati sugli allegati ricevuti;
- un `timestamp`, che qualifica temporalmente la risposta.

Questa struttura è importante perché evita che il servizio AI venga trattato come endpoint opaco che restituisce una semplice stringa. Al contrario, il modulo produce una risposta esplicativa e ricca di contesto tecnico, rendendo il suo comportamento più leggibile lato frontend e più controllabile in un'ottica di evoluzione futura.

È inoltre presente un endpoint di **health** separato, esposto sul path `/health`, la cui funzione è confermare la disponibilità base del servizio. Pur non rientrando direttamente nel question answering, tale endpoint completa la progettazione del modulo dal punto di vista operativo e della sua integrazione con il gateway.

11.4. AUTENTICAZIONE TRAMITE JWT DEL CORE SERVICE

Il servizio AI non possiede un sistema di autenticazione autonomo né una propria anagrafica utenti. La sua autenticazione si fonda interamente sui **JWT emessi dal Core Service**, che vengono considerati l'unica fonte di verità per identità e ruolo del soggetto chiamante. Questa scelta è coerente con il principio architetturale secondo cui l'ownership del dominio *Auth & Identity* appartiene esclusivamente al Core Service.

Dal punto di vista implementativo, il modulo utilizza una classe di autenticazione custom per Django REST Framework, incaricata di:

- estrarre il bearer token dall'header *Authorization*;
- validarne firma, issuer e scadenza;
- verificare che il token non sia stato revocato tramite introspection verso il Core Service;
- costruire un principal minimale da associare alla richiesta.

Il servizio non utilizza il modello *User* di Django e non introduce dipendenze verso sessioni server-side o anagrafiche locali. Il principal autenticato contiene esclusivamente le informazioni necessarie all'operatività del modulo, come identificativo utente, e-mail, ruolo e claim decodificate dal token. Questa impostazione è coerente con l'obiettivo di mantenere il servizio **stateless**, leggero e allineato al contesto identitario già definito a livello di piattaforma.

L'autenticazione tramite JWT del Core Service consente inoltre di mantenere perfettamente allineati gateway, Core Service e AI Assistant sul significato dell'accesso. Il modulo AI non introduce regole proprie sulla durata o sulla validità della sessione, ma consuma il medesimo contratto identitario adottato dal resto del backend, rafforzando così l'unità semantica dell'intero sistema.

11.5. VERIFICA DEL RUOLO CLINICIAN

L'accesso al servizio AI è riservato esclusivamente al ruolo **Clinician**. Questa restrizione non è lasciata a una convenzione applicativa esterna o a un controllo solo lato frontend, ma viene implementata esplicitamente nel backend del modulo attraverso una permission dedicata.

Dopo l'autenticazione, il servizio verifica che il principal associato alla richiesta sia effettivamente autenticato e che il valore del ruolo corrisponda a *Clinician*. Solo in presenza di tale condizione la richiesta può proseguire verso la validazione dell'input e la generazione della risposta. In caso contrario, il modulo restituisce un errore di autorizzazione coerente con la policy del sistema.

Questa scelta ha una forte valenza architetturale. Il servizio AI non è infatti pensato come componente generalista per tutti gli utenti della piattaforma, ma come supporto specialistico per chi opera nel dominio clinico. Limitare l'accesso a questo solo ruolo:

- riduce la superficie esposta del servizio;
- mantiene allineato il comportamento del modulo con la sua finalità professionale;

- evita un uso improprio da parte di attori per i quali il question answering clinico non è coerente con il modello di responsabilità del sistema.

La verifica del ruolo lato backend è anche un elemento di robustezza. Anche qualora il frontend tentasse di esporre impropriamente la funzionalità o un client esterno inviasse richieste dirette all'endpoint, il servizio continuerebbe a far valere il proprio perimetro autorizzativo in modo autonomo e coerente.

11.6. GESTIONE DI INPUT JSON E MULTIPART

Il servizio AI è progettato per accettare sia richieste **JSON** sia richieste **multipart/form-data**, così da supportare in modo naturale due scenari d'uso distinti: domande puramente testuali e domande accompagnate da allegati documentali. Tale scelta aumenta la flessibilità del modulo e ne migliora l'integrazione con il frontend clinico.

Dal punto di vista implementativo, il servizio registra parser espliciti per:

- `JSONParser`;
- `FormParser`;
- `MultiPartParser`.

Questo consente all'endpoint di accettare payload eterogenei mantenendo un unico contratto logico. I campi `question` e `context` vengono sempre validati tramite `serializer`, indipendentemente dal formato della richiesta, mentre gli eventuali allegati vengono recuperati dalla collezione dei file associati alla `request`.

La gestione congiunta di JSON e multipart è progettata con attenzione anche sotto il profilo della **robustezza semantica**. Il servizio non considera l'allegato come input separato o autonomo rispetto alla domanda, ma lo integra nel medesimo flusso applicativo: prima valida la struttura testuale minima della richiesta, poi processa gli allegati; quindi, costruisce un contesto complessivo e infine avvia i controlli di sicurezza e l'inferenza.

Questa scelta permette di mantenere il modulo AI semplice nell'esposizione esterna ma ricco nella capacità di ingestione del contenuto. Il frontend può quindi utilizzare un'unica funzionalità anche per scenari in cui il Clinician desidera accompagnare la domanda con documenti di supporto, senza dover introdurre API separate o pipeline differenziate.

11.7. ESTRAZIONE DEL TESTO DAGLI ALLEGATI

L'estrazione del testo dagli allegati è affidata a un servizio dedicato che trasforma documenti eterogenei in contenuto testuale da integrare nel prompt del modello. Questa componente svolge un ruolo cruciale perché consente al modulo AI di utilizzare non soltanto testo digitato manualmente, ma anche materiale documentale caricato dal Clinician.

I formati supportati sono:

- `.txt`
- `.md`

- .pdf
- .docx

La scelta di limitare l'insieme dei formati ammessi a un sottoinsieme preciso risponde a un criterio di **controllabilità**. Il modulo supporta soltanto documenti per cui esiste una pipeline di estrazione testuale semplice, sufficientemente affidabile e coerente con la natura del servizio. Non vengono accettati formati generici o binari non previsti, proprio per evitare ambiguità operative e aumenti incontrollati di complessità.

Per i file di testo semplice e markdown l'estrazione è diretta; per i PDF viene utilizzato un reader dedicato alla lettura del testo pagina per pagina; per i documenti Word viene letto il contenuto dei paragrafi. La progettazione segue un criterio di **semplicità pragmatica**: il servizio estrae il testo disponibile senza introdurre pipeline avanzate di normalizzazione o OCR, mantenendo così un comportamento prevedibile e coerente con il proprio ruolo consultivo.

È inoltre presente un limite di dimensione sul singolo allegato, configurabile ma con valore predefinito pari a **2 MiB**, e un limite complessivo sul testo estratto, con soglia predefinita di **8000 caratteri**. Quando il contenuto supera tale soglia, il testo viene troncato e la risposta restituisce metadati che informano il client dell'avvenuta truncation. Questa scelta protegge il modulo da prompt eccessivamente lunghi, preserva la stabilità del servizio e rende esplicito il comportamento applicato agli allegati.

11.8. RILEVAZIONE DI DATI IDENTIFICATIVI DIRETTI

Uno degli aspetti più importanti della progettazione del modulo AI è la presenza di una barriera esplicita di sicurezza contro l'invio al modello di **dati identificativi diretti del paziente**. Tale controllo viene applicato prima di qualunque tentativo di inferenza e riguarda sia il testo digitato nei campi `question` e `context`, sia l'eventuale testo estratto dagli allegati.

La rilevazione è implementata come controllo euristico su alcune categorie di PII diretta, in particolare:

- indirizzi e-mail;
- numeri di telefono;
- codice fiscale italiano nel formato standard.

Quando il servizio individua una o più di queste violazioni, interrompe il flusso e restituisce una risposta di errore applicativa con codice dedicato, invitando il chiamante a riformulare la richiesta o a rimuovere i riferimenti identificativi. Questo comportamento è progettato per agire come **barriera preventiva di primo livello**, riducendo il rischio che informazioni direttamente identificative vengano inoltrate al modello linguistico.

Dal punto di vista architetturale, la presenza di questo filtro è molto significativa. Il servizio AI non si limita a fidarsi del chiamante né delega completamente al frontend la responsabilità di sanitizzare l'input. Al contrario, introduce un controllo backend esplicito che fa parte integrante del contratto del modulo. In questo modo la sicurezza del trattamento del testo resta ancorata al servizio e non dipende da comportamenti corretti ma non verificabili del client.

È importante osservare che il controllo è deliberatamente euristico e mirato, non esaustivo. Ciò è coerente con il ruolo del modulo: non si tratta di una pipeline completa di data-loss prevention, ma di un presidio pragmatico e mirato, sufficiente a rafforzare la sicurezza del servizio all'interno del perimetro applicativo in cui esso opera.

11.9. INFERENZA LOCALE TRAMITE MODELLO HUGGING FACE

La generazione della risposta avviene tramite **inferenza locale** con un modello caricato attraverso l'ecosistema Hugging Face Transformers. Questo rappresenta uno degli aspetti più qualificanti del modulo AI, poiché consente di realizzare capacità di question answering senza dipendere da servizi esterni remoti o da API di inferenza di terze parti.

Il client AI è progettato per utilizzare, come impostazione predefinita, un modello identificato tramite la variabile `AI_MODEL_ID`, con valore di default `google/gemma-3-1b-it`, ma supporta anche la possibilità di caricare il modello da un percorso locale esplicito. Questa doppia modalità rende il servizio più flessibile senza rinunciare alla chiarezza della configurazione.

Il processo di generazione segue una sequenza ordinata:

- costruzione dei messaggi di sistema e utente;
- preparazione degli input per il tokenizer;
- applicazione del *chat template* quando disponibile;
- fallback a una costruzione prompt-based nei casi in cui il template non sia utilizzabile;
- generazione con parametri controllati;
- decodifica del testo prodotto.

La progettazione del client è orientata alla **robustezza**. Esso gestisce in modo tollerante differenti forme di output del tokenizer, supporta sia CPU sia GPU, sceglie il `torch_dtype` più appropriato in base al device disponibile e registra nei log le principali fasi del processo di caricamento e generazione. Questo approccio rende il servizio più resistente a differenze di runtime e più semplice da osservare dal punto di vista operativo.

I parametri principali di inferenza — come `max_new_tokens` e `temperature` — sono configurabili via ambiente, ma sono anche restituiti nel payload di risposta, in modo da rendere più trasparente il comportamento del modulo verso il client e verso il sistema nel suo complesso.

11.10. CARICAMENTO LAZY DEL MODELLO E FUNZIONAMENTO OFFLINE

Uno degli elementi più importanti della progettazione del modulo AI è il **caricamento lazy** del modello. Il servizio non forza il caricamento del tokenizer e del modello all'avvio del container, ma li inizializza solo nel momento in cui si presenta la prima richiesta che richiede effettivamente inferenza. Questa scelta riduce il costo di bootstrap del servizio e rende il runtime più efficiente nei contesti in cui l'assistente non venga utilizzato immediatamente.

Il caricamento è inoltre protetto da meccanismi di sincronizzazione che evitano race condition in presenza di richieste concorrenti iniziali. In questo modo il servizio mantiene una singola istanza del client di inferenza e garantisce che tokenizer e modello vengano caricati una sola volta, preservando sia correttezza sia efficienza.

Il funzionamento del modulo è progettato per essere **offline-oriented**. Il client cerca infatti di risolvere il modello da:

- un path locale esplicito, se configurato;
- una cache locale di Hugging Face, se disponibile;
- solo in ultima istanza, come riferimento logico di model ID, ma sempre con vincolo `local_files_only`.

Questo comportamento è coerente con la scelta architetturale di montare nel container una cache locale in sola lettura, dalla quale il servizio carica tokenizer e pesi del modello. In questo modo l'inferenza non dipende da download dinamici o da disponibilità di rete esterna al momento dell'uso. Tale approccio aumenta la riproducibilità del runtime, riduce l'imprevedibilità operativa e rende il comportamento del modulo più coerente con il principio di deployment controllato dell'intera piattaforma.

Anche la configurazione del container conferma questa impostazione: l'ambiente è predisposto per installare PyTorch in variante CPU o GPU in funzione del build argument `USE_GPU`, mentre Gunicorn viene configurato con un numero contenuto di worker e thread, compatibile con la natura CPU-bound o model-bound del servizio. Ne risulta un modulo progettato non come semplice API Python generica, ma come componente di inferenza locale consapevole delle proprie caratteristiche di runtime.

11.11. NATURA STATELESS DEL SERVIZIO

Il modulo AI è stato progettato come servizio **stateless** in senso forte. Questa proprietà non è solo dichiarata a livello architetturale, ma è anche concretamente realizzata nella configurazione del progetto Django: il backend database di default utilizza infatti `django.db.backends.dummy`, cioè un backend che fallisce immediatamente qualora qualcuno tenti di usare il database.

Questa scelta ha un significato architetturale molto preciso. Il servizio AI:

- non possiede anagrafiche locali;
- non conserva conversazioni;
- non salva richieste o risposte del question answering;
- non replica dati clinici o amministrativi;
- non diventa proprietario di alcun frammento del dominio informativo della piattaforma.

Ogni richiesta viene trattata come evento autonomo: il token viene validato, l'input viene analizzato, gli eventuali allegati vengono processati, il modello genera la risposta e il risultato viene restituito al chiamante senza lasciare persistenza applicativa interna. Questo comportamento è perfettamente coerente con la natura consultiva del modulo e

con il suo ruolo di componente specialistica subordinata ai domini principali della piattaforma.

La statelessness semplifica anche i confini di sicurezza e la governance del sistema. Poiché il modulo non introduce storage locale, risulta più chiaro che la verità applicativa su utenti, ruoli, appuntamenti, encounter e documentazione clinica resta interamente nel Core Service. L'AI Assistant si limita a consumare il contesto necessario alla singola richiesta, senza trasformarsi in un nodo parallelo di persistenza.

Questa proprietà rende infine il servizio più semplice da distribuire e più allineato a una topologia containerizzata. Il modulo può essere avviato, arrestato o sostituito senza dover preservare uno stato locale di dominio, a condizione che restino disponibili la configurazione, la cache del modello e il collegamento al Core Service per la token introspection.

12. MODELLO DEI DATI E STRATEGIA DI PERSISTENZA

12.1. SCELTA ARCHITETTURALE SCHEMA-PER-DOMAIN

La strategia di persistenza adottata dalla piattaforma si fonda su una singola istanza **MySQL 8.0** organizzata secondo il principio **schema-per-domain**. In termini operativi, ciò significa che il sistema non utilizza un unico database indistinto per tutti i dati, né frammenta la persistenza in più server eterogenei; sceglie invece una soluzione intermedia e rigorosa, nella quale ogni dominio funzionale dispone di un proprio spazio logico di memorizzazione all'interno della stessa istanza database.

Questa scelta risponde a esigenze architetture molto precise. La piattaforma governa domini differenti per natura, responsabilità e ciclo di vita: identità e profili utente, pianificazione temporale, documentazione clinica, pagamenti, notifiche e audit. Collocare tutto in uno schema unico avrebbe prodotto un modello meno leggibile, con maggiore rischio di accoppiamento semantico e minore chiarezza nell'ownership del dato. Utilizzare più server database distinti avrebbe invece introdotto una complessità infrastrutturale sproporzionata rispetto alle esigenze del sistema. La soluzione adottata consente quindi di mantenere **unità operativa** e **separazione logica** allo stesso tempo.

La strategia *schema-per-domain* offre diversi vantaggi progettuali. In primo luogo, rende immediatamente riconoscibile il rapporto tra **modulo applicativo** e **spazio persistente**, rafforzando la leggibilità dell'architettura. In secondo luogo, riduce la commistione tra dati che appartengono a processi differenti, facilitando la comprensione del modello e la manutenzione evolutiva. In terzo luogo, consente di implementare accessi cross-domain in modo esplicito e controllato, anziché lasciarli emergere implicitamente da una persistenza indistinta.

L'adozione di una singola istanza MySQL rimane comunque coerente con la necessità di semplicità operativa. I cinque database logici vengono creati all'avvio dell'ambiente tramite script di bootstrap e sono poi utilizzati da **DbContext** distinti all'interno del Core Service. Ne risulta una persistenza centralizzata dal punto di vista infrastrutturale, ma chiaramente separata dal punto di vista semantico.

I database logici previsti sono:

- healthcare_registry
- healthcare_scheduling
- healthcare_clinical
- healthcare_payments
- healthcare_events

Questa articolazione riflette in modo diretto la scomposizione del Core Service nei suoi moduli principali e consente di allineare la struttura del dato con la struttura del dominio applicativo.

12.2. DATABASE healthcare_registry

Il database healthcare_registry raccoglie i dati relativi all'identità degli utenti, ai profili anagrafici e alle relazioni di rappresentanza e consenso. Esso costituisce il nucleo persistente del dominio **Auth & Identity** e di una parte significativa del dominio **Registry**, ed è il punto di riferimento per tutto ciò che riguarda utenti, ruoli, profili, deleghe e validità del contesto di accesso.

Le entità principali persistenti in questo database sono:

- users
- patient_profiles
- delegate_profiles
- clinician_profiles
- delegations
- consents
- password_reset_tokens
- account_activation_tokens
- revoked_access_tokens

La tabella users rappresenta l'identità applicativa di base del soggetto autenticabile. Essa contiene l'e-mail come identificatore univoco, l'hash della password, il ruolo, lo stato di attivazione e i metadati temporali di creazione e aggiornamento. Questa struttura consente di distinguere chiaramente tra identità di accesso e profilo funzionale del soggetto.

Le tabelle patient_profiles, delegate_profiles e clinician_profiles modellano invece le tre principali tipologie di profilo applicativo, ciascuna con attributi specifici. Il **profilo paziente** include informazioni anagrafiche essenziali come nome, cognome, telefono, indirizzo e data di nascita. Il **profilo delegato** raccoglie le informazioni identificative necessarie alla gestione della rappresentanza operativa. Il **profilo clinico** aggiunge attributi professionali specifici, come specialità, numero di licenza e sede operativa. Tutte e tre le tabelle sono collegate in modo univoco alla tabella users, rafforzando la separazione tra account e profilo.

La tabella delegations formalizza il rapporto tra paziente e delegato. Essa non si limita a registrare l'esistenza di una delega, ma ne memorizza **scope**, **status**, intervallo temporale di validità e istante di creazione. Questo modello consente di trattare la rappresentanza come entità di dominio vera e propria, con ciclo di vita e capacità operative proprie, invece che come semplice associazione informativa.

La tabella consents consente di registrare i consensi associati al paziente, distinguendone il tipo, lo stato di concessione, l'istante di rilascio, un eventuale momento di revoca e note aggiuntive. Tale struttura permette di integrare il tema del consenso direttamente nel modello persistente del dominio registry, rendendolo interrogabile e governabile in modo coerente.

Le tabelle `password_reset_tokens`, `account_activation_tokens` e `revoked_access_tokens` completano il quadro del dominio dell'identità. Esse sono particolarmente importanti perché mostrano che il sistema tratta attivazione, reset password e revoca del token non come eventi effimeri, ma come elementi persistenti e verificabili del ciclo di vita dell'accesso.

Dal punto di vista dei vincoli, questo database adotta un modello rigoroso fatto di:

- indici univoci sull'e-mail dell'utente e sul numero di licenza del clinico;
- relazioni one-to-one tra `users` e profili;
- unicità sulla coppia paziente/delegato nella tabella `delegations`;
- indici dedicati per i token e per i consensi.

Ne deriva un dominio persistente fortemente strutturato, nel quale identità, profili e relazioni operative risultano chiaramente distinguibili ma semanticamente collegati.

12.3. DATABASE `healthcare_scheduling`

Il database `healthcare_scheduling` contiene tutte le entità necessarie a governare il calendario clinico, la disponibilità temporale e il ciclo di vita degli appuntamenti. Esso costituisce la persistenza del modulo **Scheduling** e rappresenta il punto di raccordo tra organizzazione delle risorse, flusso prenotativo e passaggio successivo verso il dominio clinico.

Le entità principali persistenti in questo database sono:

- `clinician_calendars`
- `availability_slots`
- `appointments`
- `appointment_status_changes`

La tabella `clinician_calendars` definisce il calendario di riferimento del professionista sanitario. Ogni clinico dispone di un calendario identificato univocamente e associato alla relativa time zone. Questa scelta consente di separare la nozione di "professionista" dal contenitore temporale in cui vengono inseriti gli slot, migliorando la chiarezza del modello.

La tabella `availability_slots` rappresenta la granularizzazione della disponibilità del clinico in unità prenotabili. Ogni slot è legato a un calendario, possiede un intervallo temporale definito da `StartUtc` ed `EndUtc`, uno stato e i metadati di creazione. L'unicità sulla coppia (`CalendarId`, `StartUtc`) garantisce che, all'interno dello stesso calendario, non possano esistere due slot con identico istante iniziale.

La tabella `appointments` rappresenta la concretizzazione della disponibilità in una prenotazione effettiva. Ogni appuntamento collega uno slot a un paziente, a un clinico e a una prestazione identificata tramite `ServiceId` e `ServiceCode`. Il modello include inoltre il prezzo quotato, la valuta, lo stato dell'appuntamento, eventuali note e i timestamp di creazione e aggiornamento. Questa struttura rende esplicito il fatto che l'appuntamento

non è soltanto una relazione tra slot e paziente, ma un oggetto di dominio completo, con contenuto economico e stato di processo.

Un elemento particolarmente interessante della progettazione è la scelta di **non imporre un vincolo di unicità assoluta su SlotId** nella tabella appointments. Ciò consente al sistema di mantenere traccia storica di appuntamenti riferiti allo stesso slot in scenari di cancellazione o rielaborazione, demandando l'unicità *funzionale* degli appuntamenti attivi alla logica applicativa. Questa è una scelta matura e coerente, perché distingue tra modello storico dei dati e regola operativa sugli appuntamenti effettivamente validi in un dato momento.

La tabella appointment_status_changes conserva la storia delle transizioni di stato degli appuntamenti. Per ciascun cambiamento vengono registrati stato di partenza, stato di arrivo, utente che ha effettuato la modifica, istante del cambiamento e motivazione. Questa struttura rafforza la tracciabilità interna del dominio scheduling e consente di ricostruire l'evoluzione del flusso prenotativo.

Dal punto di vista complessivo, healthcare_scheduling rappresenta un dominio temporale strutturato, in cui la disponibilità è distinta dalla prenotazione e la prenotazione è distinta dalla sua storia di stato. Questa articolazione migliora sia la qualità del modello sia la sua capacità di sostenere flussi complessi come reschedule, cancel, check-in e no-show.

12.4. DATABASE healthcare_clinical

Il database healthcare_clinical raccoglie il contenuto clinico e documentale della piattaforma. Esso supporta il modulo **Clinical** e contiene tutte le entità necessarie a modellare il catalogo delle prestazioni dal punto di vista sanitario, il pre-triage, l'encounter, le informazioni raccolte durante la visita, gli ordini clinici, le esecuzioni e il referto.

Le entità principali persistite in questo database sono:

- service_catalog
- clinical_encounters
- anamnesis_records
- vital_signs
- clinical_orders
- procedure_executions
- clinical_reports
- pretriage_questionnaires

La tabella service_catalog rappresenta il catalogo delle prestazioni disponibili, con codice univoco, nome, descrizione, prezzo base, valuta, stato di attivazione e metadati temporali. Questa tabella è particolarmente importante perché svolge una funzione trasversale: alimenta il flusso di prenotazione e, allo stesso tempo, partecipa alla contestualizzazione del dominio clinico.

La tabella `clinical_encounters` costituisce il fulcro dell'intero database. Ogni encounter collega logicamente un appuntamento a un paziente e a un clinico, e ne registra l'inizio, l'eventuale fine, eventuali note e il timestamp di creazione. L'indice univoco su `AppointmentId` evidenzia una scelta semantica chiara: a ogni appuntamento corrisponde al massimo un encounter clinico. Va osservato che `AppointmentId` non è modellato come foreign key fisica verso il database scheduling, ma come **riferimento logico cross-domain**, coerente con la separazione dei database.

Le tabelle `anamnesis_records` e `vital_signs` rappresentano due tipologie differenti di documentazione clinica collegata all'encounter. La prima raccoglie contenuto testuale esteso relativo all'anamnesi, insieme a data di creazione e autore della registrazione. La seconda struttura invece i parametri vitali in forma più granulare, distinguendo tipo di misura, valore numerico, unità, momento della rilevazione e operatore che l'ha effettuata.

La tabella `clinical_orders` consente di rappresentare ordini clinici interni al percorso assistenziale. Ogni ordine è collegato a un encounter e a un elemento del catalogo prestazioni, ha uno stato, eventuali note, un autore e un timestamp di creazione. Questa struttura permette di modellare la distinzione tra decisione clinica e sua eventuale esecuzione, migliorando la chiarezza del percorso documentale.

La tabella `procedure_executions` documenta le attività effettivamente eseguite a partire da un ordine clinico. Per ciascuna esecuzione vengono memorizzati l'ordine di riferimento, l'istante di esecuzione, il professionista che l'ha effettuata, l'esito e note opzionali. Tale scelta introduce una distinzione forte tra prescrizione e realizzazione concreta dell'attività.

La tabella `clinical_reports` formalizza il referto come entità autonoma con un proprio ciclo di vita. Ogni report è associato in modo univoco a un encounter, contiene il contenuto testuale, lo stato, eventuali metadati di firma, l'hash del contenuto, le informazioni di pubblicazione e il timestamp di creazione. L'unicità su `EncounterId` evidenzia che l'episodio clinico produce un solo referto logico, che può però evolvere internamente da bozza a documento firmato e pubblicato.

La tabella `pretriage_questionnaires` completa il database raccogliendo i questionari di pre-triage associati all'appuntamento e al paziente che li ha compilati. Anche qui il riferimento ad `AppointmentId` è logico e non realizzato come foreign key fisica. L'unicità sulla coppia (`AppointmentId`, `PatientUserId`) garantisce che non possano esistere più questionari per lo stesso paziente sullo stesso appuntamento, preservando la coerenza del flusso.

Nel complesso, `healthcare_clinical` è strutturato in modo da distinguere chiaramente tra:

- entità di contesto clinico generale;
- documentazione testuale;
- dati misurati;
- ordini ed esecuzioni;
- documento finale di refertazione.

Questa articolazione conferisce al dominio clinico una notevole granularità e lo rende adatto a rappresentare il percorso assistenziale come processo strutturato, non come semplice contenitore di testo.

12.5. DATABASE `healthcare_payments`

Il database `healthcare_payments` contiene la persistenza del modulo **Payments** ed è dedicato alla rappresentazione dell'intenzione di pagamento e degli eventi transazionali effettivamente associati al relativo workflow. Anche in questo caso il modello è costruito con una separazione semantica chiara tra piano logico del pagamento e sua concretizzazione operativa.

Le entità principali persistite in questo database sono:

- `payment_intents`
- `payment_transactions`

La tabella `payment_intents` formalizza l'intenzione di pagamento legata a un appuntamento. Ogni record contiene:

- l'identificativo logico dell'appuntamento;
- importo e valuta;
- stato del pagamento;
- provider utilizzato;
- identificativo dell'intento lato provider;
- chiave di idempotenza;
- timestamp di creazione e aggiornamento.

Questo modello è particolarmente significativo perché mostra che il sistema non riduce il pagamento a semplice "stato pagato/non pagato" sull'appuntamento. L'intento di pagamento è un oggetto di dominio autonomo, dotato di propria identità, propri metadati e proprio lifecycle. La presenza di una **chiave di idempotenza univoca** è inoltre un elemento di qualità notevole, poiché consente di proteggere il sistema da retry e duplicazioni involontarie nel processamento degli esiti.

La tabella `payment_transactions` documenta invece gli eventi transazionali associati a un dato intent. Ogni transazione memorizza l'identificativo del provider, lo stato, l'importo, l'istante di processamento e l'eventuale payload grezzo della risposta del provider in formato JSON. Questa scelta permette di preservare sia il livello strutturato del dominio sia una traccia del contenuto restituito dal sistema di pagamento simulato.

La relazione tra `payment_intents` e `payment_transactions` mostra una progettazione corretta del workflow economico. L'intento rappresenta il contesto logico di business; la transazione documenta il fenomeno operativo che lo attraversa. Tale distinzione facilita la riconciliazione amministrativa, la consultazione delle transazioni e la gestione di scenari in cui lo stesso intent possa essere coinvolto in più eventi tecnici successivi.

Anche in questo database il riferimento all'appuntamento è modellato come **collegamento logico cross-domain**, non come foreign key materiale verso healthcare_scheduling. Ciò mantiene coerente la separazione dei database pur consentendo al modulo di collegare in modo semanticamente corretto il pagamento al contesto prenotativo.

12.6. DATABASE healthcare_events

Il database healthcare_events raccoglie le entità del modulo **Events** e rappresenta la persistenza dei flussi informativi e di tracciamento del sistema. La sua funzione è duplice: da un lato conserva notifiche e promemoria destinati agli utenti; dall'altro memorizza gli eventi di audit necessari alla tracciabilità delle operazioni sensibili.

Le entità principali persistite in questo database sono:

- notifications
- audit_log

La tabella notifications modella le notifiche come entità di dominio vere e proprie. Ogni record contiene:

- l'utente destinatario;
- il canale di consegna;
- oggetto e corpo del messaggio;
- stato della notifica;
- istante previsto di schedulazione;
- eventuale istante di invio;
- eventuale errore;
- timestamp di creazione.

Questa struttura è particolarmente efficace perché consente di trattare le notifiche non come effetto collaterale effimero, ma come oggetti persistenti, schedulabili, consultabili e monitorabili. La presenza di un campo Status e di indici dedicati su destinatario e stato mostra che il sistema è progettato per gestire attivamente il ciclo di vita delle notifiche, incluse quelle da inviare tramite background service e SMTP.

La tabella audit_log raccoglie invece gli eventi di tracciamento amministrativo e operativo. Ogni entry registra:

- attore che ha compiuto l'azione;
- nome dell'azione;
- tipo dell'entità coinvolta;
- identificativo dell'entità;
- istante dell'evento;
- request ID associato;

- metadati aggiuntivi in formato JSON.

Il modello dell'audit log è particolarmente robusto perché separa in modo chiaro l'identità dell'attore, la semantica dell'azione e il contesto tecnico della richiesta. Questo consente al sistema di offrire un audit non riducibile al solo logging infrastrutturale, ma dotato di significato applicativo e di capacità di ricerca mirate.

Nel complesso, `healthcare_events` funge da archivio dei fenomeni trasversali della piattaforma: **chi è stato informato, quando, con quale stato di consegna, e quale azione rilevante è stata compiuta su quale risorsa**. Tale dominio è essenziale per collegare comportamento applicativo, comunicazione verso l'utente e tracciabilità amministrativa.

12.7. RELAZIONI LOGICHE TRA DOMINI

Sebbene i dati siano distribuiti in cinque database distinti, la piattaforma conserva una forte coerenza semantica attraverso un insieme di **relazioni logiche tra domini**. Tali relazioni non si manifestano sempre tramite foreign key fisiche, ma attraverso identificativi condivisi, convenzioni di dominio e servizi applicativi che interpretano in modo coerente il significato dei collegamenti tra entità appartenenti a database diversi.

Il dominio **Registry** è il principale punto di riferimento per l'identità dei soggetti. Gli identificativi utente presenti in Scheduling, Clinical, Payments ed Events fanno semanticamente riferimento agli utenti e ai profili definiti in `healthcare_registry`. Il paziente, il clinico, il delegato e l'amministratore non vengono duplicati negli altri database; i diversi moduli utilizzano il loro `UserId` come collegamento logico al dominio identitario.

Il dominio **Scheduling** collega in modo logico il soggetto utente e la prestazione sanitaria. Ogni appuntamento fa riferimento a:

- un paziente;
- un clinico;
- una prestazione del catalogo clinico;
- uno slot del calendario.

Questo rende Scheduling il punto di snodo tra identità, tempo e tipo di servizio. A sua volta, il dominio **Clinical** usa l'`AppointmentId` come riferimento logico per costruire l'encounter e associare il pre-triage all'episodio corretto.

Il dominio **Payments** si collega a Scheduling tramite `AppointmentId`, poiché il pagamento è concettualmente riferito a una prenotazione specifica. Ne deriva una relazione logica forte tra prenotazione e pagamento, pur senza introdurre foreign key cross-database che avrebbero aumentato l'accoppiamento fisico tra i moduli.

Il dominio **Events** si collega a Registry tramite `RecipientUserId` e `ActorUserId`, e si collega indirettamente agli altri moduli tramite `EntityType`, `EntityId`, contenuto delle notifiche e metadati JSON. Una notifica può nascere da un evento di scheduling o payments; un audit log può riferirsi a una prenotazione, a una delega o a una transazione. Il database eventi agisce quindi come archivio trasversale che conserva il riflesso informativo o tracciativo dei fenomeni generati altrove.

Queste relazioni logiche sono un elemento importante della strategia di persistenza. La piattaforma non persegue la totale indipendenza semantica dei database, ma una separazione **disciplinata**, nella quale i domini restano distinti pur cooperando attraverso riferimenti condivisi e controllati.

12.8. ACCESSI CROSS-DOMAIN CONTROLLATI

La separazione in database distinti non impedisce che alcuni flussi applicativi richiedano la consultazione di dati appartenenti a domini differenti. Per questa ragione la piattaforma adotta un modello di **accessi cross-domain controllati**, nel quale i moduli possono leggere o interpretare riferimenti esterni solo quando ciò è necessario al corretto svolgimento del processo.

Un primo esempio significativo è il dominio **Clinical**, che utilizza AppointmentId come collegamento logico verso Scheduling. La creazione o la gestione di un encounter presuppone infatti la verifica che esista un appuntamento coerente, associato al paziente e al clinico corretti. Clinical non possiede l'appuntamento, ma ne usa il riferimento per garantire che il percorso sanitario si appoggi a un contesto prenotativo valido.

Un secondo esempio è il dominio **Payments**, che usa anch'esso AppointmentId per collegare intenti e transazioni alla prenotazione di riferimento. Il pagamento non può essere compreso né gestito correttamente se non è rapportato all'appuntamento per il quale è stato generato. Anche in questo caso il riferimento è logico e l'interpretazione è delegata al livello applicativo, non alla foreign key cross-schema.

Il modulo **Events** effettua a sua volta accessi trasversali impliciti o espliciti. Le notifiche devono essere collegate agli utenti del Registry per determinare il destinatario effettivo; l'email dispatcher consulta registry per risolvere gli indirizzi di posta e consulta events per individuare le notifiche da inviare. Analogamente, l'audit può fare riferimento a entità appartenenti a qualsiasi modulo, purché il collegamento venga espresso in modo semanticamente chiaro tramite tipo, identificativo e metadati.

La presenza di questi accessi cross-domain è gestita in modo controllato perché:

- non vengono introdotte dipendenze fisiche indiscriminate tra tutti i database;
- i riferimenti interdominio sono espressi in forma esplicita;
- il significato del collegamento viene governato dal layer applicativo;
- l'ownership del dato resta nel dominio che lo possiede.

Questa scelta è particolarmente importante sotto il profilo architetturale. Il sistema evita sia l'isolamento eccessivo dei domini, che renderebbe impossibile la costruzione dei flussi reali, sia l'accoppiamento incontrollato, che annullerebbe i vantaggi della separazione *schema-per-domain*. Il risultato è una cooperazione selettiva e motivata, coerente con i casi d'uso effettivi della piattaforma.

12.9. SEED INIZIALE E DATI DIMOSTRATIVI

La piattaforma include un meccanismo di **seed iniziale** progettato per popolare i database con un dataset coerente e sufficiente a supportare verifiche end-to-end, test manuali e dimostrazione dei principali flussi applicativi. La presenza del seeder rappresenta una

scelta di qualità importante, poiché rende il sistema immediatamente fruibile dopo l'inizializzazione senza richiedere la creazione manuale di tutte le entità fondamentali.

Il seeder è costruito come processo **idempotente**. Esso verifica se nei domini principali siano già presenti utenti o catalogo prestazioni e, in caso affermativo, evita di reinserire i dati. Questa strategia permette di proteggere il sistema da duplicazioni involontarie e rende il bootstrap più sicuro nei riavvii dell'ambiente o nelle operazioni ripetute di inizializzazione.

Il dataset dimostrativo copre tutti e cinque i database logici e include elementi coerenti tra loro. In particolare, vengono creati:

- un **Admin**
- un **Clinician**
- un **Patient**
- un **Delegate**
- i relativi profili
- una delega attiva
- due consensi del paziente
- due prestazioni di catalogo
- un calendario clinico
- tre slot di disponibilità
- un appuntamento
- una transizione di stato iniziale dell'appuntamento
- un encounter clinico
- una anamnesi
- un parametro vitale
- un ordine clinico
- una esecuzione
- un referto in bozza
- un payment intent
- una transazione
- una notifica
- una voce di audit

Questa copertura mostra che il seed non è limitato a dati anagrafici elementari, ma costruisce un **microcosmo coerente** dell'intera piattaforma, collegando fra loro identità,

scheduling, clinical, payments ed events. Ciò rende possibile testare scenari realistici senza dover ricostruire manualmente tutte le dipendenze tra domini.

Un elemento particolarmente utile è la scelta di utilizzare **identificativi deterministici** per molte entità seeded. Questo facilita debug, test manuali, verifica dei flussi e comprensione del dataset, poiché rende più semplice riconoscere e correlare gli oggetti creati automaticamente nei diversi database.

Il seed include inoltre credenziali dimostrative già hashate secondo la stessa logica usata dal servizio di autenticazione, a conferma del fatto che il dataset non aggira il modello di sicurezza del sistema ma si integra con esso in modo coerente. Anche questo contribuisce a rendere il comportamento complessivo della piattaforma più ordinato e più vicino a un ambiente applicativo realmente utilizzabile.

13. SICUREZZA, PRIVACY E CONTROLLO DEGLI ACCESSI

13.1. AUTENTICAZIONE E GESTIONE DEGLI ACCESSI

L'autenticazione della piattaforma è progettata come un processo centralizzato e coerente, fondato sul modulo **Auth & Identity** del Core Service e sull'utilizzo di **JWT Bearer tokens** come meccanismo di trasporto del contesto di accesso tra client, gateway e servizi downstream. Questa scelta consente di mantenere separata la gestione dell'identità dalla logica di business dei moduli applicativi, preservando una chiara distinzione tra **chi è l'utente, quale ruolo possiede e quali operazioni può effettivamente eseguire**.

Il ciclo di autenticazione si articola in più fasi: registrazione, attivazione account, login, emissione del token, utilizzo del token nelle richieste protette, logout e revoca del token. L'utente non viene considerato immediatamente operativo al momento della sola creazione dell'account; l'abilitazione effettiva è subordinata al completamento del flusso di attivazione, che costituisce una misura di controllo ulteriore sul ciclo di vita dell'identità digitale.

Il **login** produce un token JWT che incorpora i claim necessari a identificare l'utente nel resto del sistema. Tra questi assumono particolare rilevanza l'identificativo del soggetto, il ruolo applicativo e le informazioni necessarie alla validazione del token da parte del gateway e dei servizi backend. La piattaforma adotta quindi un modello nel quale il token non rappresenta soltanto una credenziale tecnica, ma il **veicolo del contesto identitario** lungo l'intera catena applicativa.

La gestione della sessione è di tipo prevalentemente **stateless** lato API, ma non completamente priva di stato lato server. Il sistema, infatti, non si limita a emettere token firmati e a considerare valido ogni token fino alla sua scadenza naturale; mantiene anche informazioni server-side necessarie a verificarne la revoca e la legittimità corrente. Questa impostazione consente di combinare l'efficienza dell'approccio JWT con un livello di controllo più rigoroso sul ciclo di vita della sessione.

Dal punto di vista del frontend, la sessione viene gestita mediante conservazione del token lato client e suo utilizzo nelle chiamate API protette. Tuttavia, il browser non è considerato fonte autorevole per la determinazione dello stato di autenticazione. Il frontend si appoggia infatti a endpoint dedicati, come il profilo dell'utente corrente, per riallineare il proprio stato con la verità applicativa del backend, mantenendo così una separazione corretta tra **sessione locale e validità effettiva dell'accesso**.

Il **logout** non viene trattato come mera cancellazione locale del token, ma come operazione che produce anche un effetto sul lato server tramite registrazione della revoca. Questa scelta rafforza la sicurezza del sistema e impedisce che un token già emesso possa restare riutilizzabile per il solo fatto di non essere ancora formalmente scaduto.

13.2. AUTORIZZAZIONE BASATA SUI RUOLI

L'autorizzazione del sistema si fonda su un modello **role-based**, nel quale ogni utente autenticato opera in funzione del ruolo assegnato e del contesto applicativo di riferimento. I ruoli principali della piattaforma sono **Admin**, **Clinician**, **Patient** e **Delegate**, e la loro presenza non ha una finalità puramente descrittiva, ma determina in modo sostanziale le capacità operative, le interfacce disponibili e il perimetro informativo accessibile.

La presenza del ruolo all'interno del token JWT consente una prima qualificazione dell'utente lungo il percorso della richiesta. Il gateway può così verificare preliminarmente che il token sia valido, mentre il Core Service e il modulo AI possono applicare controlli più specifici sulla base del ruolo dichiarato. Questa progettazione realizza una distinzione corretta tra **autenticazione**, che risponde alla domanda "chi è l'utente?", e **autorizzazione**, che risponde alla domanda "cosa è legittimato a fare?".

L'autorizzazione non si esaurisce però nella sola appartenenza a una categoria astratta. Il ruolo costituisce il primo livello di filtro, ma viene poi integrato da controlli di contesto più specifici. Un Clinician, ad esempio, non può accedere indiscriminatamente a qualunque dato clinico solo in quanto appartenente al ruolo sanitario; la sua operatività è ulteriormente vincolata alla pertinenza dell'appuntamento, dell'encounter e della relazione con il paziente. Analogamente, un Delegate può operare soltanto in presenza di una delega valida e di uno scope coerente con l'azione richiesta.

La piattaforma traduce l'autorizzazione basata sui ruoli in più livelli coordinati:

- nel **frontend**, attraverso la separazione delle aree applicative e la segmentazione delle interfacce;
- nel **gateway**, tramite la verifica dei token e l'applicazione di policy uniformi sul traffico protetto;
- nel **Core Service**, mediante attributi di autorizzazione, controlli sui claim e verifiche di contesto;
- nel **modulo AI**, tramite permission dedicate che consentono l'accesso solo al ruolo Clinician.

Questa distribuzione dei controlli migliora la robustezza del sistema, perché impedisce che la protezione dell'accesso dipenda da un unico punto. Anche qualora un livello venga aggirato o usato impropriamente, gli altri continuano ad applicare i propri vincoli. L'autorizzazione diventa così una proprietà emergente dell'intera piattaforma, non una semplice annotazione applicata a singoli endpoint.

13.3. SEPARAZIONE TRA AMBITO AMMINISTRATIVO E AMBITO CLINICO

Uno dei principi più importanti della sicurezza della piattaforma è la netta separazione tra **ambito amministrativo** e **ambito clinico**. Questa distinzione è strutturale e attraversa tutti i livelli del sistema: modello dei dati, organizzazione delle interfacce, segmentazione dei controller, servizi applicativi, controlli di autorizzazione e tracciamento delle operazioni.

L'**Admin** dispone di capacità operative ampie sul piano organizzativo e amministrativo. Può gestire anagrafiche, profili, deleghe, disponibilità, appuntamenti, pagamenti, notifiche e audit. Tuttavia, non gli viene attribuito un ruolo clinico: non redige referti, non gestisce encounter come professionista sanitario e non utilizza l'assistente AI consultivo. Questa delimitazione è essenziale per impedire che il governo amministrativo del sistema si trasformi in accesso improprio a contenuti sanitari non pertinenti.

Il **Clinician**, dal canto suo, possiede accesso al dominio clinico e alla propria agenda, ma non dispone delle capacità di supervisione trasversale tipiche dell'amministrazione. La piattaforma evita quindi una commistione tra responsabilità gestionali e responsabilità

assistenziali. Il professionista può documentare il percorso clinico, consultare il pre-triage, creare ordini ed esecuzioni, redigere e pubblicare referti, ma non acquisisce automaticamente la possibilità di governare directory utenti, pagamenti amministrativi o audit globali.

Questa separazione non è realizzata soltanto tramite convenzioni organizzative, ma anche con meccanismi tecnici concreti:

- controller distinti per area amministrativa, clinica, patient e delegate;
- endpoint protetti tramite attributi di ruolo coerenti;
- controlli applicativi che limitano la visibilità alle sole risorse pertinenti;
- segmentazione del frontend in aree dedicate e non sovrapponibili;
- distinzione dei dati persistiti e delle viste di consultazione disponibili.

Dal punto di vista della privacy, tale separazione riduce il rischio di accessi impropri a informazioni sensibili. Dal punto di vista della sicurezza applicativa, rafforza il principio del **least privilege**, secondo cui ogni attore deve poter eseguire solo quanto strettamente necessario alla propria funzione. Dal punto di vista architetturale, essa contribuisce a mantenere coerente il rapporto tra ruolo, dominio operativo e responsabilità di business.

13.4. REVOKA DEI TOKEN E INTROSPECTION

La piattaforma adotta un modello di sicurezza nel quale il token JWT non viene considerato automaticamente valido fino alla sola scadenza temporale. A complemento della validazione crittografica della firma e dei claim, il sistema implementa una strategia di **revoca server-side** e di **introspection** verso il Core Service, così da verificare lo stato reale della sessione associata al token.

Il meccanismo di revoca è basato sulla persistenza dei token invalidati all'interno del dominio `healthcare_registry`, tramite la tabella dedicata ai **revoked access tokens**. Quando l'utente effettua il logout, il token corrente viene registrato come revocato e può quindi essere successivamente riconosciuto come non più valido, anche se la sua scadenza non è ancora stata raggiunta. Questa scelta introduce una forma di controllo dinamico sul ciclo di vita delle sessioni e consente di rendere il logout una vera operazione di invalidazione.

Il **Core Service** verifica la revoca anche in fase di autenticazione locale. Durante la validazione del JWT, oltre ai controlli standard su issuer, chiave di firma e lifetime, il servizio consulta il modulo Auth & Identity per determinare se il token sia presente nell'archivio delle revoche. In caso affermativo, l'accesso viene negato. Questo comportamento rende il Core Service il punto autorevole per la validità sostanziale del token.

Il **gateway** si integra con questo modello attraverso una vera **token introspection** verso il Core Service. Dopo aver effettuato il parsing e la verifica crittografica iniziale del JWT, il middleware di autenticazione del gateway invoca l'endpoint interno di introspection per sapere se il token sia ancora attivo o revocato. Solo se entrambe le condizioni risultano coerenti, la richiesta può essere inoltrata ai servizi downstream. In questo modo il gateway

non agisce come validatore completamente autonomo, ma si appoggia al modulo che detiene l'ownership del dominio identità.

Anche il **modulo AI** utilizza lo stesso principio. La sua autenticazione custom, infatti, valida il bearer token ricevuto e ne verifica lo stato tramite introspection verso il Core Service, mantenendo così piena coerenza con le politiche di sicurezza adottate dal resto della piattaforma.

Questa architettura offre vantaggi rilevanti:

- rende la revoca effettiva e non solo simbolica;
- unifica la nozione di token valido tra tutti i componenti del sistema;
- riduce il rischio di riutilizzo di sessioni invalidate;
- preserva la centralità del Core Service come autorità sul dominio dell'identità.

13.5. PROTEZIONE DEI DATI PERSONALI E CLINICI

La protezione dei dati personali e clinici è affrontata dalla piattaforma mediante una combinazione di **segmentazione del perimetro informativo**, **controlli di accesso contestuali**, **separazione dei domini** e **minimizzazione dell'esposizione**. Il sistema non si limita a autenticare gli utenti, ma struttura l'accesso ai dati in modo da ridurre al minimo la visibilità non necessaria.

Nel dominio **Registry** i dati personali sono separati dai dati clinici e collocati in un database dedicato. Questa distinzione migliora la chiarezza dell'ownership e impedisce che informazioni anagrafiche, credenziali, deleghe e consensi vengano fuse con la documentazione sanitaria. La strategia *schema-per-domain* contribuisce così direttamente anche alla privacy, poiché favorisce una gestione più disciplinata delle diverse classi di dato.

Nel dominio **Clinical**, i dati sanitari sono accessibili solo attraverso servizi e controller destinati al ruolo appropriato e, nel caso del Clinician, solo quando sussiste un contesto operativo coerente. L'accesso al pre-triage, all'encounter, alle anamnesi, ai parametri vitali, agli ordini, alle esecuzioni e al referto non è esposto come semplice consultazione libera, ma è mediato da servizi che verificano la pertinenza del soggetto autenticato rispetto alla risorsa.

Il modello **Delegate** introduce una protezione ulteriore basata sulla rappresentanza controllata. Il delegato non ottiene automaticamente la visibilità del paziente, ma può operare solo quando esiste una delega attiva, riferita all'assistito corretto e coerente con lo scope dell'operazione. Questo meccanismo limita l'accesso ai dati personali e clinici per rappresentanza ai soli casi autorizzati e tracciabili.

La piattaforma adotta inoltre una logica di **minimizzazione del dato esposto**. Ogni attore vede solo la porzione di informazione necessaria al proprio flusso:

- l'Admin governa anagrafiche, appuntamenti, pagamenti e audit, ma non si sostituisce al Clinician nella gestione clinica;
- il Clinician opera sul contenuto sanitario pertinente al proprio percorso professionale;

- il Patient vede il proprio profilo, i propri appuntamenti, i propri pagamenti e la documentazione a lui destinata;
- il Delegate accede esclusivamente ai dati dell'assistito e solo entro il perimetro della delega.

Anche la gestione delle notifiche e delle e-mail riflette questa attenzione. Le comunicazioni vengono generate a partire da eventi applicativi, ma il sistema conserva e risolve i destinatari tramite il dominio Registry, evitando esposizioni ridondanti o canali paralleli di gestione dell'identità.

Dal punto di vista complessivo, la privacy non viene trattata come semplice risultato della crittografia o della protezione del token, ma come proprietà che deriva dal **modo in cui i dati sono modellati, separati, resi visibili e correlati ai ruoli**.

13.6. AUDITABILITA' DELLE OPERAZIONI SENSIBILI

La piattaforma introduce un livello strutturato di **auditabilità** per le operazioni rilevanti, distinguendo chiaramente tra logging tecnico e audit applicativo. Il logging tecnico è orientato alla diagnosi del comportamento runtime dei servizi; l'audit, invece, documenta azioni sensibili compiute da utenti autenticati su risorse di dominio rilevanti.

Il meccanismo di audit è implementato nel Core Service attraverso una combinazione di:

- un **AuditActionFilter** applicato ai controller;
- un servizio applicativo dedicato alla creazione delle entry di audit;
- una persistenza specifica nel database `healthcare_events`;
- endpoint di consultazione amministrativa dei log.

L'**AuditActionFilter** intercetta l'esecuzione delle azioni sui controller e costruisce, quando opportuno, una rappresentazione strutturata dell'evento da registrare. Tra le informazioni rilevate figurano:

- l'attore che ha effettuato l'operazione;
- l'azione applicativa svolta;
- il tipo dell'entità coinvolta;
- l'identificativo della risorsa;
- il Request ID associato alla richiesta;
- eventuali metadati aggiuntivi.

Questo approccio è particolarmente efficace perché integra la tracciabilità direttamente nella pipeline applicativa, senza affidarla a chiamate manuali distribuite in modo disomogeneo all'interno del codice dei singoli moduli.

Le entry di audit vengono persistite nella tabella `audit_log` del dominio `healthcare_events`, rendendole:

- durevoli nel tempo;

- ricercabili;
- filtrabili;
- consultabili attraverso servizi amministrativi dedicati.

La consultazione dell'audit è esposta attraverso un controller riservato al ruolo amministrativo, il che conferma il fatto che i log di audit sono considerati risorsa operativa sensibile e non informazione liberamente accessibile a tutti gli utenti.

L'auditabilità delle operazioni sensibili migliora il sistema su più piani:

- rafforza la responsabilizzazione degli attori;
- permette di ricostruire il contesto di azioni rilevanti;
- consente verifiche ex post su attività amministrative o cliniche;
- integra la sicurezza preventiva con una capacità di osservazione successiva.

In una piattaforma che gestisce dati sanitari, profili utente, pagamenti e deleghe, questa capacità di tracciamento strutturato rappresenta un requisito di grande rilievo architettonico e operativo.

13.7. SALVAGUARDIE SPECIFICHE DEL MODULO AI

Il modulo **AI Assistant** introduce esigenze di sicurezza e privacy specifiche rispetto agli altri servizi della piattaforma. Per questa ragione non viene trattato come semplice endpoint applicativo aggiuntivo, ma come componente specialistica soggetta a salvaguardie dedicate su **accesso, input, trattamento dei contenuti e persistenza**.

La prima salvaguardia riguarda l'**accesso ristretto al solo ruolo Clinician**. Il servizio implementa una permission esplicita lato backend che permette l'uso dell'endpoint `/clinicians/qa` esclusivamente a utenti autenticati con ruolo `Clinician`. Questa scelta riduce la superficie di esposizione del modulo e mantiene il suo utilizzo confinato a un contesto professionale coerente con la natura consultiva del servizio.

La seconda salvaguardia riguarda l'**autenticazione tramite JWT del Core Service** e la relativa introspection. Il modulo non possiede un proprio sistema identitario e non si affida a sessioni locali o a utenti Django persistiti in un database. Convalida invece il token emesso dal Core Service, ne verifica firma, issuer, scadenza e stato di revoca. In questo modo, anche il servizio AI resta pienamente subordinato al modello di identità e sicurezza dell'intera piattaforma.

La terza salvaguardia concerne la **sanitizzazione del contenuto in ingresso**. Prima di avviare l'inferenza, il modulo esegue un controllo euristico su testo della domanda, contesto e allegati estratti per individuare alcune categorie di dati identificativi diretti del paziente, in particolare:

- indirizzi e-mail;
- numeri di telefono;
- codice fiscale italiano.

Se il sistema rileva una di queste violazioni, la richiesta viene bloccata e il client riceve un errore applicativo dedicato. Questo presidio ha un valore molto importante: impedisce che il servizio AI diventi un canale di elaborazione incontrollata di contenuti direttamente identificativi.

La quarta salvaguardia riguarda la **gestione limitata dei formati documentali**. Il modulo accetta solo un insieme ristretto di estensioni — testo semplice, markdown, PDF e DOCX — e applica limiti espliciti alla dimensione dei file e alla quantità di testo effettivamente utilizzata nel prompt. Questa scelta riduce il rischio di input non gestibili, contenuti eccessivi o comportamenti imprevedibili del servizio.

La quinta salvaguardia è la **natura stateless del modulo**. Il servizio non conserva richieste, risposte, allegati o dati clinici in un database locale. La configurazione Django utilizza deliberatamente un backend dummy, proprio per impedire l'uso accidentale di persistenza interna. Ciò riduce il rischio di duplicazione di dati sensibili e rafforza il principio secondo cui il dominio clinico resta di esclusiva competenza del Core Service.

Infine, il comportamento del modello è indirizzato anche a livello di **prompt di sistema**, che ne limita il ruolo a una funzione di supporto informativo prudente, strutturata e non sostitutiva del giudizio clinico. Questa scelta contribuisce a ridurre il rischio di interpretazioni improprie del servizio come motore decisionale, mantenendo il modulo entro un perimetro di utilizzo compatibile con la sicurezza complessiva della piattaforma.

14. INTERFACCE REST E ORGANIZZAZIONE DEGLI ENDPOINT

14.1. CONVENZIONI DI PROGETTAZIONE DELLE API

Le interfacce REST della piattaforma sono progettate secondo un insieme di convenzioni finalizzate a garantire **coerenza semantica, leggibilità degli URI, separazione per dominio e allineamento tra ruolo applicativo e perimetro funzionale**. L'obiettivo non è soltanto esporre operazioni HTTP funzionanti, ma costruire una superficie API che rifletta in modo disciplinato la struttura del sistema e renda immediatamente comprensibile la responsabilità di ciascun gruppo di endpoint.

La prima convenzione fondamentale riguarda l'esistenza di un **namespace API unificato**. Tutte le chiamate effettuate dal frontend transitano attraverso il prefisso `/api`, esposto dal gateway come punto di ingresso unico per il backend. Questo implica che, dal punto di vista del client, l'intero sistema si presenti come un unico perimetro logico, pur essendo internamente articolato in Core Service e AI Assistant. La topologia interna viene quindi nascosta dietro un contratto uniforme.

La seconda convenzione riguarda la **segmentazione per dominio**. Gli endpoint non sono organizzati in modo casuale o puramente tecnico, ma vengono raggruppati secondo i principali ambiti applicativi:

- `auth`
- `me`
- `registry`
- `scheduling`
- `catalog`
- `clinical`
- `payments`
- `notifications`
- `events`
- `health`
- `ai`

Questa suddivisione rende immediatamente esplicito il contesto a cui appartiene una risorsa e rafforza la corrispondenza tra architettura del Core Service e superficie esposta dalle API.

Una terza convenzione molto importante è la **specializzazione per ruolo** all'interno degli URI. Molti endpoint includono infatti segmenti come `admin`, `patients/me`, `delegates`, `clinicians/me`, in modo da rendere esplicito già nel path il punto di vista funzionale da cui l'operazione viene esposta. Questa scelta non sostituisce i controlli di autorizzazione lato backend, ma migliora la leggibilità del contratto API e rafforza il legame tra ruolo e responsabilità operativa.

Dal punto di vista dei metodi HTTP, la piattaforma adotta una convenzione pragmatica:

- GET per il recupero di risorse o collezioni;
- POST per creazione di nuove risorse o avvio di azioni di processo;
- PUT per aggiornamenti sostitutivi o consolidamento di contenuti;
- PATCH per modifiche parziali di stato o di permessi.

Questa distinzione è usata in modo coerente con la natura del dominio. Le semplici letture avvengono tramite GET, mentre i passaggi di workflow — ad esempio cancel, reschedule, check-in, publish, reconcile — sono modellati come operazioni esplicite tramite POST o PATCH, poiché rappresentano **transizioni di processo** più che aggiornamenti strutturali generici.

Un'ulteriore convenzione riguarda l'uso degli identificativi. Le risorse principali vengono generalmente identificate tramite **GUID**, esplicitati nel path con vincoli di routing che ne formalizzano il formato. Questa scelta è coerente con la struttura del modello dati e con la necessità di mantenere identificativi stabili e non ambigui lungo l'intera piattaforma.

La progettazione delle API adotta inoltre un approccio orientato alla **chiarezza operativa**. Molti endpoint non sono genericamente CRUD, ma rappresentano azioni dotate di significato di business, come:

- activate
- logout
- token/introspect
- report/sign
- report/publish
- simulate-provider-outcome
- read

Questo rende il contratto API più vicino al dominio reale e più espressivo rispetto a una modellazione eccessivamente astratta.

14.2. ENDPOINT DI AUTENTICAZIONE

Gli endpoint di autenticazione sono esposti dal Core Service sotto il namespace `/api/auth`, con l'aggiunta dell'endpoint dedicato `/api/me` per il recupero del contesto utente corrente. Questi endpoint costituiscono la superficie pubblica del modulo **Auth & Identity** e coprono l'intero ciclo di vita dell'account e della sessione.

Gli endpoint principali sono:

- POST `/api/auth/register/patient`
- POST `/api/auth/register/delegate`
- POST `/api/auth/login`

- POST /api/auth/activate
- GET /api/auth/activate/confirm
- POST /api/auth/activation/resend
- POST /api/auth/logout
- POST /api/auth/token/introspect
- POST /api/auth/password/forgot
- POST /api/auth/password/reset
- GET /api/me

La distinzione tra `register/patient` e `register/delegate` mostra che la piattaforma espone flussi di registrazione separati per le categorie self-service, mantenendo però un unico dominio autorevole per la gestione dell'identità. L'endpoint di `login` genera il contesto di autenticazione basato su JWT, mentre `logout` gestisce l'invalidazione della sessione mediante revoca del token.

Particolarmente importante è l'endpoint `token/introspect`, che non è rivolto al frontend come funzionalità utente, ma costituisce un servizio infrastrutturale di supporto per gateway e modulo AI. La sua presenza dimostra che la piattaforma non si limita a emettere token, ma offre anche un meccanismo centralizzato per verificarne lo stato effettivo.

Gli endpoint di attivazione e reset password completano il ciclo di vita dell'account, mentre `GET /api/me` fornisce al client il proprio profilo operativo essenziale e consente al frontend di riallinearsi con il contesto identitario effettivo riconosciuto dal backend.

14.3. ENDPOINT DI REGISTRY

Gli endpoint di **registry** sono organizzati sotto il namespace `/api/registry` e riflettono la distinzione tra gestione amministrativa delle anagrafiche e gestione self-service dei profili. La struttura del namespace rende molto chiara la pluralità dei punti di vista operativi presenti nella piattaforma.

L'area amministrativa è esposta principalmente tramite:

- GET /api/registry/admin/patients
- POST /api/registry/admin/patients
- GET /api/registry/admin/delegates
- POST /api/registry/admin/delegates
- GET /api/registry/admin/clinicians
- POST /api/registry/admin/clinicians
- GET /api/registry/admin/patients/{userId}/profile
- PUT /api/registry/admin/patients/{userId}/profile

- GET /api/registry/admin/delegates/{userId}/profile
- PUT /api/registry/admin/delegates/{userId}/profile
- GET /api/registry/admin/clinicians/{userId}/profile
- PUT /api/registry/admin/clinicians/{userId}/profile
- GET /api/registry/admin/patients/{userId}/delegations
- POST /api/registry/admin/patients/{userId}/delegations
- GET /api/registry/admin/delegates/{userId}/delegations
- PATCH /api/registry/admin/delegations/{delegationId}/status
- PATCH /api/registry/admin/delegations/{delegationId}/permissions
- GET /api/registry/admin/patients/{userId}/consents
- PUT /api/registry/admin/patients/{userId}/consents

Questi endpoint mostrano una progettazione molto chiara del dominio anagrafico: l'Admin governa utenti, profili, deleghe e consensi attraverso endpoint specializzati e semanticamente leggibili. La scelta di utilizzare PATCH per stato e permessi delle deleghe è corretta, poiché si tratta di aggiornamenti parziali di un'entità esistente.

Il lato self-service è invece articolato come segue:

- GET /api/registry/patients/me/profile
- PUT /api/registry/patients/me/profile
- GET /api/registry/patients/me/delegations
- PATCH /api/registry/patients/me/delegations/{delegationId}/permissions
- GET /api/registry/patients/me/consents
- PUT /api/registry/patients/me/consents
- GET /api/registry/delegates/me/profile
- PUT /api/registry/delegates/me/profile
- GET /api/registry/delegates/me/delegations
- GET /api/registry/clinicians/me/profile
- PUT /api/registry/clinicians/me/profile

Questa articolazione rende evidente che la piattaforma distingue tra:

- profili amministrati;
- profili personali;
- relazioni di delega;

- consensi.

Gli endpoint di registry rappresentano quindi un esempio molto forte di API disegnate secondo il principio **ruolo + dominio + responsabilità operativa**.

14.4. ENDPOINT DI SCHEDULING

Gli endpoint di **scheduling** sono esposti sotto il namespace `/api/scheduling` e modellano la dimensione temporale e organizzativa della piattaforma. La struttura distingue chiaramente tra prospettiva amministrativa, prospettiva del clinico, prospettiva del paziente e prospettiva del delegato.

L'area amministrativa comprende:

- GET `/api/scheduling/admin/availability`
- GET `/api/scheduling/admin/clinicians/{clinicianUserId}/slots`
- POST `/api/scheduling/admin/clinicians/{clinicianUserId}/slots`
- POST `/api/scheduling/admin/slots/{slotId}/status`
- GET `/api/scheduling/admin/clinicians/{clinicianUserId}/appointments`
- GET `/api/scheduling/admin/appointments`
- POST `/api/scheduling/admin/appointments`
- POST `/api/scheduling/admin/appointments/{appointmentId}/cancel`
- POST `/api/scheduling/admin/appointments/{appointmentId}/reschedule`
- POST `/api/scheduling/admin/appointments/{appointmentId}/check-in`
- POST `/api/scheduling/admin/appointments/{appointmentId}/no-show`

Questa sezione mostra una progettazione orientata al governo operativo dell'intero calendario e del ciclo di vita degli appuntamenti. Gli endpoint di creazione slot, cambio stato, check-in e no-show rappresentano bene la natura processuale del dominio scheduling.

La prospettiva del Clinician è più compatta e mirata:

- GET `/api/scheduling/clinicians/me/appointments`

Questo endpoint espone l'agenda del professionista in modo filtrato e contestualizzato, confermando che il ruolo clinico non ha visibilità amministrativa globale, ma solo accesso al proprio perimetro operativo.

La prospettiva del Patient è modellata come segue:

- GET `/api/scheduling/patients/me/availability`
- GET `/api/scheduling/patients/me/appointments`
- POST `/api/scheduling/patients/me/appointments`

- POST
/api/scheduling/patients/me/appointments/{appointmentId}/cancel
- POST
/api/scheduling/patients/me/appointments/{appointmentId}/reschedule

La prospettiva del Delegate è analoga ma distinta:

- GET /api/scheduling/delegates/me/availability
- GET /api/scheduling/delegates/me/appointments
- POST /api/scheduling/delegates/me/appointments
- POST
/api/scheduling/delegates/me/appointments/{appointmentId}/reschedule
- POST
/api/scheduling/delegates/me/appointments/{appointmentId}/cancel

È particolarmente significativa la scelta di esporre i flussi patient e delegate con namespace separati, pur mantenendo operazioni parallele. Questo rende l'API più leggibile e riflette correttamente la differenza semantica tra azione diretta e azione per rappresentanza.

14.5. ENDPOINT DI CLINICAL

Gli endpoint del dominio **clinical** sono distribuiti sotto due namespace principali:

- /api/catalog per il catalogo prestazioni;
- /api/clinical per pre-triage, encounter, referti e documentazione clinica.

Per il catalogo, la piattaforma distingue tra consultazione generale e amministrazione:

- GET /api/catalog/services
- GET /api/catalog/services/{id}
- GET /api/catalog/services/by-code/{code}
- GET /api/catalog/admin/services
- GET /api/catalog/admin/services/{id}
- POST /api/catalog/admin/services
- PUT /api/catalog/admin/services/{id}

Questa articolazione mostra chiaramente la distinzione tra fruizione del catalogo e sua gestione.

Il dominio clinico lato Clinician è esposto tramite:

- GET /api/clinical/clinicians/me/encounters
- GET /api/clinical/clinicians/me/encounters/{encounterId}

- POST /api/clinical/clinicians/me/encounters
- POST /api/clinical/clinicians/me/encounters/{encounterId}/anamneses
- POST /api/clinical/clinicians/me/encounters/{encounterId}/vital-signs
- POST /api/clinical/clinicians/me/encounters/{encounterId}/orders
- POST /api/clinical/clinicians/me/orders/{orderId}/executions
- PUT /api/clinical/clinicians/me/encounters/{encounterId}/report
- POST /api/clinical/clinicians/me/encounters/{encounterId}/report/sign
- POST /api/clinical/clinicians/me/encounters/{encounterId}/report/publish
- POST /api/clinical/clinicians/me/encounters/{encounterId}/complete

Questo gruppo di endpoint rappresenta uno dei migliori esempi di API orientate al dominio. L'encounter è il centro del workflow, e le operazioni sui suoi componenti — anamnesi, vital signs, orders, report — sono espresse in modo semanticamente forte e direttamente leggibile.

Il pre-triage è organizzato per ruolo:

- GET /api/clinical/clinicians/me/pretriage/appointments/{appointmentId}
- GET /api/clinical/patients/me/pretriage/appointments/{appointmentId}
- PUT /api/clinical/patients/me/pretriage/appointments/{appointmentId}
- GET /api/clinical/delegates/me/pretriage/appointments/{appointmentId}
- PUT /api/clinical/delegates/me/pretriage/appointments/{appointmentId}

La consultazione dei referti lato utente finale è distinta tra paziente e delegato:

- GET /api/clinical/patients/me/reports
- GET /api/clinical/delegates/me/reports
- GET /api/clinical/delegates/me/patients/{patientUserId}/reports

Questa progettazione rende molto chiaro che la produzione del contenuto clinico resta nel perimetro del Clinician, mentre la sua consultazione avviene in namespace distinti e coerenti con il soggetto che accede.

14.6. ENDPOINT DI PAYMENTS

Gli endpoint del dominio **payments** sono esposti sotto il namespace `/api/payments` e distinguono con precisione il punto di vista amministrativo, quello del paziente, quello del delegato e quello del provider simulato.

L'area amministrativa comprende:

- GET `/api/payments/admin/intents`
- GET `/api/payments/admin/intents/{intentId}/transactions`
- POST `/api/payments/admin/appointments/{appointmentId}/in-person`
- POST `/api/payments/admin/intents/{intentId}/reconcile`
- POST `/api/payments/admin/intents/{intentId}/simulate-provider-outcome`

Qui si coglie bene la distinzione tra consultazione generale, registrazione del pagamento in sede, riconciliazione amministrativa e simulazione dell'esito provider.

Il lato Patient espone:

- GET `/api/payments/patients/me/intents`
- POST `/api/payments/patients/me/appointments/{appointmentId}/intent`
- POST `/api/payments/patients/me/intents/{intentId}/process`

Il lato Delegate espone:

- GET `/api/payments/delegates/me/intents`
- POST `/api/payments/delegates/me/appointments/{appointmentId}/intent`
- POST `/api/payments/delegates/me/intents/{intentId}/process`

Anche in questo caso la duplicazione strutturale tra patient e delegate è una scelta corretta, perché mantiene separata la semantica dell'azione diretta da quella della rappresentanza.

Il provider simulato è esposto tramite:

- POST `/api/payments/provider/webhooks/simulated`

Questo endpoint non è destinato all'uso del frontend come operazione utente, ma rappresenta un punto di integrazione tecnico per modellare l'arrivo di esiti di pagamento da un sistema esterno simulato. La sua presenza rende il dominio payments più realistico e meglio strutturato dal punto di vista del workflow.

14.7. ENDPOINT DI NOTIFICATIONS E AUDIT

Il dominio **notifications** è esposto attraverso namespace distinti per area amministrativa, paziente e delegato, mentre l'audit è raccolto in un endpoint amministrativo dedicato.

Gli endpoint di notifica lato admin sono:

- GET /api/notifications/admin
- GET /api/notifications/admin/{notificationId}
- POST /api/notifications/admin

Questa sezione consente consultazione e creazione amministrativa di notifiche.

Lato patient:

- GET /api/notifications/patients/me
- GET /api/notifications/patients/me/{notificationId}
- POST /api/notifications/patients/me/{notificationId}/read

Lato delegate:

- GET /api/notifications/delegates/me
- GET /api/notifications/delegates/me/{notificationId}
- POST /api/notifications/delegates/me/{notificationId}/read

La presenza dell'azione read modellata come endpoint specifico indica che il sistema considera la lettura della notifica come **transizione di stato** e non come semplice aggiornamento informale. Questa è una scelta corretta e coerente con il dominio events.

L'audit è esposto tramite:

- GET /api/events/admin/audit

Il fatto che l'audit sia collocato sotto events e non sotto notifications è semanticamente corretto, perché il logging di audit appartiene a un dominio più ampio di eventi applicativi e di tracciamento, non al solo sottodominio delle notifiche.

14.8. ENDPOINT DI HEALTH

La piattaforma espone endpoint di **health** a più livelli, coerentemente con la sua architettura multi-servizio.

Il **Core Service** espone:

- GET /api/health dal punto di vista del client, che corrisponde internamente a GET /health sul servizio ASP.NET Core.

Il **modulo AI** espone:

- GET /health internamente, raggiungibile lato client tramite l'aggregazione del gateway e non come endpoint diretto separato.

Il **gateway** espone inoltre:

- GET /health per il proprio stato locale;
- GET /api/health come endpoint di **health aggregata** verso il frontend.

L'endpoint `/api/health` del gateway rappresenta quindi la vista più significativa dal punto di vista della piattaforma complessiva, perché sintetizza la disponibilità dei principali servizi backend. Questa distinzione tra health locale e health aggregata è importante anche sul piano documentale: non tutte le health esposte hanno lo stesso significato operativo.

14.9. ENDPOINT DEL MODULO AI

Il modulo AI è esposto al client attraverso il gateway sotto il namespace `/api/ai`, mentre internamente il servizio Django riceve il traffico su path privi di tale prefisso. Il gateway si occupa infatti di inoltrare le richieste al backend AI dopo avere rimosso il segmento `/ai`.

Dal punto di vista del frontend e del contratto esterno, gli endpoint rilevanti sono:

- `POST /api/ai/clinicians/qa`
- `GET /api/health` come health aggregata del sistema, che include anche la verifica del servizio AI

Dal punto di vista interno del servizio Django, gli endpoint corrispondenti sono:

- `POST /clinicians/qa`
- `GET /health`

L'endpoint di question answering è deliberatamente unico e altamente specializzato. Non esiste una molteplicità di rotte AI generiche, ma un solo punto di ingresso pensato per il ruolo Clinician e per un caso d'uso ben delimitato. Questa scelta è coerente con la natura consultiva e contenuta del modulo.

L'endpoint accetta sia payload JSON sia richieste multipart, così da poter supportare domande testuali semplici e domande arricchite da allegati. Dal punto di vista del contratto esterno, ciò significa che l'API AI non è solo semanticamente chiara, ma anche sufficientemente flessibile rispetto alle modalità di ingresso del contenuto.

14.10. FORMATO DELLE RISPOSTE ED ERROR HANDLING

Le API della piattaforma restituiscono risposte in formato **JSON** e adottano un modello di error handling coerente con la natura REST del sistema e con il ruolo del gateway come front door unificato. L'obiettivo non è soltanto produrre esiti tecnicamente corretti, ma offrire al frontend e agli altri client un comportamento prevedibile, interpretabile e sufficientemente uniforme.

Per le risposte di successo, il sistema adotta payload strutturati coerenti con il dominio coinvolto:

- DTO singoli per il recupero di una risorsa;
- collezioni o liste per endpoint di consultazione;
- oggetti con metadati aggiuntivi per casi particolari, come pagamenti, notifiche o AI;
- messaggi di esito quando l'operazione rappresenta una transizione di workflow più che una semplice lettura.

Per gli errori, la piattaforma utilizza prevalentemente:

- 400 Bad Request per input non validi o contesti non ammissibili;
- 401 Unauthorized per assenza o invalidità del contesto di autenticazione;
- 403 Forbidden o status equivalenti per mancanza di autorizzazione nel contesto corrente;
- 404 Not Found per risorse inesistenti o non accessibili nel perimetro richiesto;
- 500 Internal Server Error per errori inattesi lato servizio;
- 503 Service Unavailable per health negative o indisponibilità di componenti.

Un aspetto importante è che molti controller restituiscono **payload di errore espliciti**, spesso contenenti campi come message, code o metadati equivalenti, invece di limitarsi a uno status code privo di contenuto. Questa scelta è utile perché migliora la capacità del frontend di interpretare l'errore e di trasformarlo in feedback leggibile per l'utente.

Il gateway contribuisce in modo determinante alla coerenza dell'error handling. Tramite middleware di **Recovery JSON**, il sistema evita che errori imprevisti producano risposte HTML o formati non coerenti con la natura API della piattaforma. Anche in presenza di problemi tecnici lato routing o upstream, il client continua quindi a ricevere un output strutturato e gestibile.

Il modulo AI, a sua volta, adotta un modello di risposta arricchito che non si limita a restituire la sola answer, ma include anche blocchi come model, safety, attachments e timestamp. In caso di errore, utilizza payload con codice e dettaglio del problema, ad esempio per input non validi o contenenti dati identificativi diretti. Questo rende il servizio non solo più sicuro, ma anche più trasparente dal punto di vista del contratto.

Nel complesso, il formato delle risposte e la gestione degli errori riflettono una progettazione orientata a tre principi fondamentali: **coerenza strutturale**, **leggibilità per il client** e **allineamento tra semantica di dominio e semantica HTTP**.

15. DEPLOY, CONFIGURAZIONE ED ESECUZIONE DEL SISTEMA

15.1. PREREQUISITI DI ESECUZIONE

L'esecuzione della piattaforma richiede un ambiente in grado di supportare una topologia applicativa containerizzata composta da frontend statico, API Gateway, Core Service, AI Assistant, database relazionale e servizi di supporto. La scelta architetturale adottata, basata su **Docker** e **Docker Compose**, riduce in modo significativo la necessità di predisporre manualmente i singoli runtime applicativi, ma non elimina la presenza di alcuni prerequisiti fondamentali che devono essere soddisfatti affinché l'intero stack possa essere avviato in modo coerente.

Il primo prerequisito è la disponibilità di un'installazione funzionante di **Docker Engine** compatibile con l'esecuzione di container Linux e di **Docker Compose** o della relativa integrazione nativa nel comando `docker compose`. Poiché l'intera piattaforma è distribuita come insieme di servizi orchestrati, l'assenza di questo livello impedirebbe non soltanto l'avvio delle singole componenti, ma anche la corretta definizione della rete interna, dei volumi condivisi, delle dipendenze di startup e delle esposizioni di porta.

È inoltre necessario disporre di risorse hardware adeguate al carico combinato dei servizi. In particolare, il modulo AI introduce un fabbisogno superiore rispetto agli altri componenti, poiché il caricamento del modello Hugging Face e l'esecuzione dell'inferenza locale richiedono memoria e capacità computazionale non trascurabili. Anche in configurazione CPU-only, il servizio AI necessita di un ambiente sufficientemente dimensionato da sostenere il caricamento del modello, la tokenizzazione e la generazione del testo senza compromettere la stabilità complessiva del sistema.

Un ulteriore prerequisito riguarda la disponibilità del **modello AI in cache locale**. La progettazione del modulo AI è infatti esplicitamente orientata al funzionamento *offline-first* e non presuppone il download dinamico dei pesi del modello al primo avvio. È quindi necessario che la directory montata come cache Hugging Face contenga già i file richiesti dal modello configurato, oppure che venga predisposto un path locale equivalente opportunamente valorizzato nelle variabili d'ambiente del servizio AI.

Dal punto di vista del networking locale, l'ambiente di esecuzione deve consentire l'esposizione delle porte previste per i servizi pubblicamente raggiungibili o utili alla verifica operativa della piattaforma. In particolare, il frontend è reso disponibile tramite porta **8080**, il database MySQL tramite **3306** e Mailpit tramite **8025** per l'interfaccia web. La corretta disponibilità di tali porte è necessaria per accedere all'applicazione, ispezionare il traffico e-mail di sviluppo e, se richiesto, verificare la connettività verso il database.

È infine importante che l'ambiente consenta il montaggio dei volumi previsti dalla configurazione Compose. Questo aspetto è particolarmente rilevante sia per l'inizializzazione del database tramite script SQL sia per la disponibilità della cache locale del modello AI. La piattaforma, infatti, non si affida a una configurazione runtime completamente effimera, ma integra nel proprio deployment anche risorse persistenti e file di bootstrap che devono essere resi accessibili ai container nel modo previsto dalla topologia di esecuzione.

15.2. VARIABILI D'AMBIENTE

La configurazione del sistema è largamente governata tramite **variabili d'ambiente**, utilizzate per parametrizzare il comportamento dei servizi senza introdurre dipendenze rigide dal contesto di esecuzione. Questa scelta è coerente con una progettazione orientata alla containerizzazione, nella quale il codice applicativo deve rimanere separato dai parametri operativi, infrastrutturali e sensibili.

Nel **Core Service**, le variabili d'ambiente riguardano principalmente:

- la connessione al database MySQL;
- i parametri di emissione e validazione dei JWT;
- la configurazione SMTP per l'invio delle e-mail;
- i tempi di scheduling del dispatcher notifiche;
- eventuali informazioni di bootstrap del servizio.

La stringa di connessione ai diversi schemi logici MySQL rappresenta un elemento centrale, poiché il Core Service governa più domini persistenti separati e deve poter inizializzare i rispettivi DbContext in modo coerente. Allo stesso modo, issuer, audience, secret key e durata dei token costituiscono parametri fondamentali del dominio *Auth & Identity*, e devono essere definiti con precisione affinché gateway, Core Service e modulo AI interpretino in modo uniforme il contesto di autenticazione.

Per il sottosistema e-mail, le variabili d'ambiente definiscono host SMTP, porta, eventuali credenziali, mittente di default e parametri accessori di consegna. Anche se nel runtime locale il servizio di supporto è Mailpit, la presenza di questa configurazione mantiene separata la logica applicativa dalla destinazione concreta del canale SMTP.

Nel **gateway**, le variabili d'ambiente sono utilizzate per configurare:

- le URL interne dei servizi downstream;
- i parametri di verifica JWT;
- eventuali impostazioni di rate limiting;
- la configurazione della health aggregata e dei timeout di connessione verso gli upstream.

Questo assetto rende il gateway indipendente da valori hardcoded e ne facilita l'adattamento a diversi contesti di deployment, pur mantenendo invariata la sua responsabilità architetturale di punto di ingresso unico per il traffico API.

Nel **modulo AI**, le variabili d'ambiente assumono un ruolo ancora più significativo. Esse governano:

- l'identificativo del modello o il path locale del modello;
- la directory della cache Hugging Face;
- il numero massimo di token generabili;

- i parametri di temperatura;
- i limiti dimensionali per gli allegati;
- la configurazione JWT necessaria alla verifica dei token del Core Service;
- l'eventuale selezione del device o delle capacità GPU/CPU.

Questa configurabilità è essenziale perché il servizio AI, pur essendo integrato nell'architettura generale, rimane una componente specialistica con esigenze runtime proprie. La parametrizzazione tramite ambiente consente di adeguarne il comportamento senza intervenire sul codice, preservando portabilità e chiarezza del deployment.

Anche il database MySQL e i servizi di supporto si appoggiano a variabili d'ambiente per username, password, nome del database e parametri di bootstrap. La coerenza di tali valori è indispensabile per il corretto avvio dell'intero stack, poiché qualsiasi incongruenza nella configurazione comprometterebbe la capacità dei servizi applicativi di stabilire connessioni verso la persistenza o verso i sistemi ausiliari.

15.3. DOCKERFILE DEI COMPONENTI

Ogni componente runtime principale della piattaforma è accompagnato da un **Dockerfile** dedicato, progettato per produrre un'immagine coerente con il ruolo del servizio e con le dipendenze tecnologiche che esso richiede. La presenza di Dockerfile distinti per frontend, API Gateway, Core Service e AI Assistant riflette una scelta architetturale precisa: ogni componente deve essere impacchettato come unità autonoma, con il proprio ambiente di esecuzione e con un perimetro di responsabilità chiaramente definito.

Il **frontend** è distribuito come immagine Nginx che serve contenuti statici e applica una configurazione di proxy verso l'API Gateway. Il Dockerfile di questo componente è orientato alla semplicità: copia gli asset statici HTML, CSS e JavaScript, installa o eredita il web server necessario e include la configurazione Nginx che espone l'applicazione e inoltra il traffico `/api/*` al gateway. Questa scelta consente di mantenere il frontend estremamente leggero e di evitare la presenza di runtime superflui lato presentazione.

Il **gateway** utilizza un Dockerfile multi-stage tipico delle applicazioni Go, nel quale il servizio viene prima compilato in un'immagine di build e poi trasferito come binario in una runtime image più essenziale. Questa strategia riduce il peso finale dell'immagine, migliora la portabilità del servizio e rafforza la separazione tra fase di compilazione e fase di esecuzione. Dal punto di vista operativo, il risultato è un container snello, dedicato esclusivamente all'esecuzione del binario del gateway e delle sue dipendenze minime.

Il **Core Service** adotta anch'esso un Dockerfile multi-stage, coerente con le best practice del mondo .NET. Il progetto viene ripristinato, compilato e pubblicato in una fase di build, per poi essere distribuito in una immagine runtime ASP.NET Core più compatta. Questo approccio consente di ottenere immagini più pulite e meglio ottimizzate per l'esecuzione, evitando di trascinare nel container finale strumenti e artefatti non necessari alla sola fase runtime.

Il **modulo AI** presenta un Dockerfile più articolato, coerente con la natura specialistica del servizio. Oltre all'installazione delle dipendenze Python, esso gestisce l'installazione di **PyTorch** in modalità CPU o GPU a seconda dei parametri di build, prepara l'ambiente per

Django/DRF, include il codice del servizio e definisce il comando di avvio basato su Gunicorn. Questa costruzione evidenzia che il servizio AI non è un semplice microservizio HTTP, ma una componente che richiede attenzione particolare alla disponibilità delle librerie di inferenza e alla configurazione del runtime.

Nel loro insieme, i Dockerfile della piattaforma mostrano una progettazione coerente sotto tre profili: **separazione dei runtime, ottimizzazione del packaging e allineamento tra ruolo del servizio e struttura dell'immagine**. Ogni container è costruito per eseguire esattamente il compito per cui è stato introdotto, senza incorporare responsabilità estranee né dipendenze superflue.

15.4. ORCHESTRAZIONE CON DOCKER COMPOSE

L'orchestrazione dell'intero stack è affidata a **Docker Compose**, che rappresenta il livello di coordinamento runtime dell'architettura. Compose non è utilizzato come semplice strumento per avviare più container, ma come meccanismo dichiarativo con cui vengono formalizzati i servizi, la rete interna, i volumi, le porte esposte e le dipendenze reciproche dell'intero sistema.

La configurazione Compose definisce i principali servizi della piattaforma:

- frontend;
- API Gateway;
- Core Service;
- AI Assistant;
- MySQL;
- Mailpit.

Ciascun servizio viene descritto con la propria immagine o con il proprio contesto di build, le variabili d'ambiente, le porte, i volumi montati e gli eventuali vincoli di dipendenza. Questa struttura rende il deployment **riproducibile e trasparente**, poiché ogni relazione tra componenti è esplicitata nella configurazione di orchestrazione.

Un elemento particolarmente importante della progettazione Compose è la presenza di una **rete applicativa condivisa**, all'interno della quale i servizi possono raggiungersi tramite i rispettivi nomi logici. Il frontend comunica con il gateway, il gateway comunica con Core Service e AI Assistant, il Core Service comunica con MySQL e Mailpit, e l'AI Assistant accede alla cache del modello montata come volume. Tutti questi collegamenti avvengono entro una topologia di rete interna definita in modo esplicito e non lasciata a configurazioni estemporanee.

Compose governa anche l'esposizione verso l'esterno. Il frontend rende disponibile la piattaforma sul punto di ingresso principale, mentre Mailpit e MySQL espongono porte utili rispettivamente per ispezione SMTP locale e per connessioni verso la persistenza. Questa scelta è coerente con la natura del runtime: il browser deve interagire con il frontend, mentre gli altri componenti restano prevalentemente confinati nella rete interna, salvo i servizi che devono poter essere consultati o ispezionati dall'esterno per finalità operative.

Dal punto di vista architetturale, Docker Compose realizza nel concreto la topologia descritta nel diagramma del sistema. Ciò significa che la configurazione di deploy non è un elemento accessorio separato dall'architettura, ma il suo naturale completamento runtime. La relazione tra documentazione architetturale e orchestrazione operativa risulta quindi particolarmente forte e coerente.

15.5. SEQUENZA DI AVVIO DEI SERVIZI

La sequenza di avvio della piattaforma è organizzata in modo da rispettare le dipendenze funzionali e infrastrutturali tra i componenti. Sebbene Docker Compose non costituisca un sistema di orchestrazione con semantica di dependency management avanzata paragonabile a piattaforme più complesse, la configurazione della soluzione è comunque strutturata per favorire una startup ordinata e coerente.

Il primo elemento che deve risultare disponibile è il **database MySQL**, poiché rappresenta la base persistente necessaria al funzionamento del Core Service. Contestualmente, può essere avviato **Mailpit**, che non costituisce un prerequisito strutturale per l'inizializzazione del database ma è richiesto per i flussi di invio e-mail del Core Service e per la corretta disponibilità del sottosistema notifiche.

Una volta che la persistenza è raggiungibile, può avviarsi il **Core Service**, che necessita di connettersi ai diversi schemi MySQL, inizializzare eventuali processi di bootstrap, configurare i propri DbContext e predisporre i background services. Il Core Service svolge un ruolo centrale non solo per i flussi di business, ma anche per la token introspection richiesta sia dal gateway sia dal modulo AI.

Parallelamente, o immediatamente dopo, può avviarsi l'**AI Assistant**, il cui funzionamento dipende dal gateway solo in termini di esposizione esterna, ma dal Core Service in termini di validazione del token e del ruolo. Sebbene il servizio AI non richieda un database locale, esso necessita della disponibilità della cache del modello e della corretta configurazione delle variabili di ambiente che definiscono il comportamento del client Hugging Face e dell'autenticazione.

Il **gateway** deve poter raggiungere sia il Core Service sia l'AI Assistant per svolgere correttamente il proprio ruolo. Per questa ragione, dal punto di vista logico, il suo avvio risulta pienamente utile solo quando almeno i servizi downstream principali siano disponibili o in fase avanzata di startup. Il gateway espone inoltre la health aggregata, il che presuppone la possibilità di interrogare i servizi backend per restituire uno stato coerente.

Infine, il **frontend** può essere avviato e reso disponibile verso l'esterno come punto di accesso principale del sistema. Dal punto di vista strettamente tecnico, il frontend statico potrebbe essere servito anche indipendentemente dalla completa disponibilità del backend; tuttavia, dal punto di vista funzionale, la piattaforma è realmente operativa solo quando il frontend può inoltrare correttamente il traffico API al gateway e ricevere da esso risposte coerenti.

Questa sequenza mostra che l'avvio della piattaforma non dipende soltanto dall'ordine in cui vengono creati i container, ma dalla **raggiungibilità progressiva dei servizi di cui ogni componente ha bisogno per il proprio corretto comportamento**.

15.6. INIZIALIZZAZIONE DEL DATABASE

L'inizializzazione del livello dati è uno degli aspetti più importanti del deploy della piattaforma. Essa non si limita alla semplice creazione dell'istanza MySQL, ma comprende la predisposizione dei cinque database logici, la definizione delle tabelle, degli indici e dei vincoli, nonché il caricamento del **seed iniziale** necessario a rendere il sistema immediatamente utilizzabile.

La creazione dei database `healthcare_registry`, `healthcare_scheduling`, `healthcare_clinical`, `healthcare_payments` e `healthcare_events` avviene tramite script SQL montati nel container MySQL in fase di bootstrap. Questa scelta è coerente con una strategia di deployment che vuole rendere il livello di persistenza **riproducibile**, **dichiarativo** e **coerente con la struttura architetturale del sistema**.

Una volta creati gli schemi logici e le relative strutture, il Core Service esegue il proprio processo di **seed iniziale**. Tale operazione è progettata in modo idempotente e introduce:

- utenti e profili di base;
- deleghe e consensi;
- catalogo prestazioni;
- calendario e slot;
- appuntamenti;
- encounter, anamnesi, parametri vitali, ordini, esecuzioni e referto;
- payment intent e transazioni;
- notifiche e audit log.

Questo approccio produce un ambiente che non è soltanto tecnicamente avviabile, ma già **funzionalmente coerente**. La piattaforma può così essere verificata lungo i propri principali flussi end-to-end senza richiedere la creazione manuale di tutte le entità di base.

Dal punto di vista del deployment, la distinzione tra script di bootstrap SQL e seeding applicativo è molto corretta. Gli script SQL definiscono lo strato persistente strutturale, mentre il seeding del Core Service costruisce un dataset dimostrativo semanticamente coerente con il dominio. Questa separazione migliora la leggibilità del processo di inizializzazione e chiarisce il ruolo di ciascuna fase.

15.7. MAILPIT PER LA GESTIONE SMTP LOCALE

Mailpit è integrato nell'ambiente di esecuzione come servizio di supporto per la gestione **SMTP locale**. La sua funzione è ricevere le e-mail generate dal sistema senza inoltrarle verso un provider reale, permettendo così di mantenere il comportamento del sottosistema di comunicazione pienamente attivo ma all'interno di un ambiente controllato.

Questa scelta è particolarmente utile per tutti i flussi che dipendono da consegna e-mail, come:

- attivazione account;

- reset password;
- notifiche pianificate;
- comunicazioni applicative rilevanti.

Dal punto di vista della progettazione del deployment, Mailpit consente di trattare il canale e-mail come **parte integrante del runtime**, e non come simulazione astratta o funzionalità semplicemente disabilitata. Il Core Service può quindi utilizzare un host SMTP reale dal proprio punto di vista, pur sapendo che la destinazione concreta è un sink locale dedicato all'ispezione.

La presenza di una **web UI** esposta da Mailpit costituisce un vantaggio operativo notevole. Essa permette di verificare visivamente il corretto invio dei messaggi, di ispezionarne contenuto, destinatario e tempistiche, e di controllare il comportamento del background service di dispatch e-mail senza dover introdurre dipendenze da strumenti esterni.

Dal punto di vista architetturale, Mailpit svolge quindi una doppia funzione: da un lato simula il canale SMTP in modo credibile, dall'altro rende osservabile il sottosistema di notifica e-mail. Questa integrazione contribuisce alla qualità del deployment e rafforza la verificabilità dei flussi che dipendono dal canale di posta.

15.8. MONTAGGIO DELLA CACHE LOCALE DEL MODELLO AI

Il montaggio della **cache locale Hugging Face** costituisce un elemento essenziale del deploy del modulo AI. La piattaforma è progettata affinché il servizio di question answering operi in modalità **local-first** e non dipenda, al momento dell'esecuzione, dal download dinamico del modello da sorgenti remote. Per rendere possibile questo comportamento, il container dell'AI Assistant monta una directory locale contenente tokenizer, configurazione e pesi del modello.

Dal punto di vista tecnico, la directory viene montata nel container in modalità **read-only**. Questa scelta è importante per almeno tre ragioni. In primo luogo, chiarisce che il servizio AI deve consumare risorse già predisposte e non produrre o alterare contenuti persistenti in tale area. In secondo luogo, riduce il rischio di modifiche accidentali alla cache durante l'esecuzione. In terzo luogo, rafforza l'idea che il modello rappresenti una **dipendenza runtime controllata**, non uno spazio di lavoro mutevole del servizio.

Il client di inferenza del modulo AI è stato progettato per cercare il modello proprio all'interno di questa cache o, in alternativa, in un path locale esplicito definito a configurazione. La presenza del volume montato consente quindi di allineare perfettamente:

- configurazione del servizio;
- disponibilità del modello;
- comportamento *offline* dell'inferenza.

Questa progettazione migliora la **riproducibilità** del deployment. Ogni ambiente che disponga della stessa cache locale e della stessa configurazione può eseguire il modulo AI in modo sostanzialmente equivalente, senza dipendere da condizioni di rete o da tempi di download imprevedibili. Il montaggio della cache non è quindi un dettaglio implementativo

secondario, ma parte integrante della strategia di esecuzione del servizio AI e della sua coerenza con l'architettura generale della piattaforma.

16. VALIDAZIONE FUNZIONALE E SCENARI DI UTILIZZO

16.1. SCENARIO AMMINISTRATIVO END-TO-END

Lo scenario amministrativo end-to-end ha lo scopo di verificare che il sistema supporti in modo coerente l'intero ciclo operativo presidiato dal ruolo **Admin**, dalla gestione anagrafica fino al controllo dei pagamenti, delle notifiche e dell'audit. La validazione di questo scenario è particolarmente importante perché il profilo amministrativo attraversa domini diversi della piattaforma e rappresenta il principale punto di raccordo tra organizzazione interna, front-office operativo e controllo dei processi.

Il flusso si apre con l'autenticazione dell'Admin tramite il modulo **Auth & Identity**. Una volta ottenuto un token valido e verificata la corretta appartenenza al ruolo amministrativo, il frontend deve rendere disponibile l'area dedicata con dashboard, navigazione coerente e accesso ai moduli di registry, scheduling, catalogo, pagamenti, notifiche e audit. La validazione, in questa fase, riguarda sia la corretta emissione della sessione sia la capacità del frontend di esporre esclusivamente l'interfaccia compatibile con il ruolo.

Un primo blocco di verifica riguarda il dominio **Registry**. L'Admin deve poter:

- consultare l'elenco di pazienti, delegati e clinici;
- creare nuove anagrafiche amministrate;
- accedere al dettaglio del profilo;
- aggiornare le informazioni di profilo;
- gestire deleghe e consensi del paziente.

Lo scenario risulta correttamente validato quando la creazione di una nuova entità amministrata produce un record coerente nel dominio registry, la modifica del profilo è persistita correttamente e le viste di dettaglio riflettono in modo immediato l'aggiornamento effettuato. Nel caso delle deleghe, la validazione richiede anche la corretta gestione dello stato e dei permessi associati, poiché tali attributi influenzano direttamente i flussi lato delegate.

Un secondo blocco riguarda il dominio **Scheduling**. L'Admin deve poter:

- consultare la disponibilità complessiva;
- creare slot per un clinico;
- modificare lo stato di uno slot;
- creare un appuntamento per un paziente;
- consultare il dettaglio dell'appuntamento;
- ripianificare o annullare una prenotazione;
- eseguire il check-in;
- registrare il no-show.

La correttezza end-to-end di questo scenario emerge quando ogni operazione produce una transizione coerente dello stato dell'appuntamento, l'informazione è visibile nelle schermate successive e il sistema genera gli effetti collaterali previsti, in particolare notifiche e tracciamento di audit. La validazione è particolarmente significativa sul passaggio tra creazione appuntamento, check-in e no-show, poiché qui il dominio scheduling dimostra di non essere un semplice calendario statico ma un vero motore di processo.

Un terzo blocco di verifica riguarda il **catalogo prestazioni**. L'Admin deve poter consultare l'elenco delle prestazioni, crearne di nuove e aggiornarne i dati essenziali. Questo scenario risulta valido quando la prestazione appena creata o modificata diventa immediatamente disponibile nei flussi di consultazione e prenotazione compatibili, senza richiedere ulteriori passaggi di sincronizzazione.

Un quarto blocco riguarda il dominio **Payments**. L'Admin deve poter:

- consultare gli intenti di pagamento;
- consultare le transazioni associate;
- registrare un pagamento in sede;
- riconciliare un intento;
- simulare l'esito del provider.

La validazione è corretta quando il pagamento registrato o aggiornato modifica coerentemente lo stato della posizione economica e, se previsto, genera notifiche collegate. Particolarmente rilevante è il controllo sulla simulazione dell'esito provider, perché consente di verificare che il dominio economico reagisca correttamente a eventi asincroni o esterni.

Un quinto blocco riguarda il modulo **Events**, con particolare attenzione a:

- consultazione delle notifiche;
- creazione di notifiche amministrative;
- consultazione dell'audit log.

Lo scenario è validato quando le notifiche risultano visibili e coerenti con gli eventi di business che le hanno generate e quando le operazioni amministrative sensibili lasciano traccia consultabile nell'audit. La verifica dell'audit è particolarmente importante perché consente di controllare la presenza di attore, azione, entità coinvolta e request identifier.

Dal punto di vista end-to-end, lo scenario amministrativo può essere considerato correttamente validato quando un operatore Admin autentica la sessione, crea o aggiorna soggetti di dominio, governa gli slot, gestisce un appuntamento, registra o verifica un pagamento, consulta le notifiche e ritrova le operazioni sensibili nei log di audit, il tutto mantenendo coerenza tra frontend, API, persistenza e stato dei servizi.

16.2. SCENARIO CLINICO END-TO-END

Lo scenario clinico end-to-end verifica il comportamento della piattaforma dal punto di vista del **Clinician**, che rappresenta il soggetto responsabile della presa in carico del

paziente e della costruzione della documentazione sanitaria dell'episodio assistenziale. Questo scenario è essenziale perché attraversa il modulo **Scheduling**, il modulo **Clinical**, il dominio dell'identità e, facoltativamente, il modulo AI consultivo.

Il punto di partenza è l'autenticazione del Clinician e la corretta disponibilità della sua area applicativa dedicata. La validazione richiede che, a seguito del login, il frontend esponga:

- dashboard clinica;
- agenda personale;
- accesso agli encounter;
- accesso al profilo;
- accesso al modulo AI.

Il primo passo operativo consiste nella consultazione dell'**agenda del clinico**. Il sistema deve restituire solo gli appuntamenti pertinenti al professionista autenticato, senza esporre prenotazioni non assegnate o non coerenti con il suo perimetro operativo. Questo controllo è importante perché valida contemporaneamente:

- il filtraggio per ruolo;
- la coerenza tra scheduling e profilo professionale;
- la corretta segmentazione dell'interfaccia.

A partire da un appuntamento valido e nello stato corretto, il Clinician deve poter avviare o consultare l'**encounter clinico**. La validazione di questo passaggio richiede che l'apertura dell'encounter sia consentita solo quando sussistono i presupposti applicativi necessari, come la pertinenza dell'appuntamento e, ove previsto, il corretto avanzamento del workflow organizzativo. Una volta aperto l'encounter, il dettaglio deve mostrare un contesto coerente, comprensivo delle informazioni già disponibili e delle azioni compatibili con lo stato dell'episodio.

Lo scenario prosegue con la **registrazione anamnestica**. Il Clinician inserisce un contenuto anamnestico e il sistema lo associa correttamente all'encounter. La validazione è soddisfatta quando:

- il contenuto risulta persistito nel dominio clinical;
- il dettaglio encounter lo rende immediatamente visibile;
- la registrazione è riferita al professionista corretto.

Successivamente viene verificata la **registrazione dei parametri vitali**. Il sistema deve consentire l'inserimento di una rilevazione con tipo, valore e unità, collegandola all'encounter appropriato. Il comportamento è corretto quando il dato risulta recuperabile dal dettaglio dell'episodio clinico e mantiene una struttura coerente con il modello previsto.

Il passo successivo consiste nella **creazione di un ordine clinico**. Il Clinician deve poter formalizzare un ordine, associarlo all'encounter e a un riferimento di catalogo, e ritrovarlo nel quadro generale dell'episodio. La presenza di uno stato iniziale coerente e di eventuali

note dimostra la corretta modellazione dell'ordine come entità autonoma del percorso clinico.

Su un ordine valido deve poi essere possibile registrare una **esecuzione**. La validazione in questo caso richiede che il sistema colleghi l'esecuzione all'ordine corretto, la renda visibile nel contesto dell'encounter e preservi la distinzione semantica tra ordine e azione eseguita.

La parte conclusiva dello scenario riguarda il **workflow del referto**. Il Clinician deve poter:

- creare o aggiornare la bozza del referto;
- firmare il documento;
- pubblicarlo;
- completare l'encounter.

Questo passaggio è particolarmente importante perché verifica il comportamento più delicato del modulo clinical. La validazione è soddisfatta quando il referto evolve coerentemente da bozza a documento firmato e pubblicato, e quando la pubblicazione lo rende disponibile alla consultazione lato paziente o delegato secondo le regole del sistema.

Dal punto di vista end-to-end, lo scenario clinico risulta correttamente validato quando un professionista autentificato:

- accede solo ai propri appuntamenti;
- apre un encounter ammissibile;
- documenta anamnesi, parametri, ordini ed esecuzioni;
- produce e pubblica un referto;
- completa il percorso clinico mantenendo coerenza tra interfaccia, API, persistenza e stato del workflow.

16.3. SCENARIO PATIENT END-TO-END

Lo scenario **patient end-to-end** verifica la correttezza della piattaforma dal punto di vista dell'utente finale titolare del proprio percorso digitale. Si tratta di uno degli scenari più estesi, poiché coinvolge domini differenti: autenticazione, profilo, catalogo, scheduling, clinical, payments, notifications e consensi.

Il flusso si apre con la **registrazione** del paziente o, in alternativa, con l'utilizzo di un account già attivato. Nel caso della registrazione, la validazione richiede:

- corretta creazione dell'utente;
- generazione del token di attivazione;
- invio dell'e-mail di attivazione tramite il sottosistema notifiche/SMTP;
- impossibilità di effettuare il login prima dell'attivazione.

Una volta completata l'attivazione, il paziente deve poter eseguire il **login** e accedere all'area patient del frontend. In questa fase è importante verificare che la piattaforma esponga solo l'insieme di funzionalità pertinenti al ruolo: profilo, servizi, prenotazioni, pre-triage, deleghe, consensi, pagamenti, notifiche e referti.

Un primo blocco operativo riguarda la **gestione del profilo personale**. Il paziente deve poter recuperare il proprio profilo, modificarne i campi ammessi e verificarne il corretto aggiornamento lato interfaccia e lato persistenza. Parallelamente, deve poter consultare e aggiornare i **consensi** e visualizzare le **deleghe** che lo riguardano. Questo scenario conferma il corretto collegamento tra area patient e modulo registry.

Il secondo blocco riguarda il **catalogo prestazioni** e la **ricerca delle disponibilità**. Il paziente deve poter:

- consultare l'elenco dei servizi disponibili;
- selezionare una prestazione;
- interrogare la disponibilità;
- ottenere una lista di slot prenotabili.

La validazione è corretta quando gli slot restituiti riflettono la disponibilità reale del modulo scheduling e sono coerenti con il catalogo esposto.

Il terzo blocco riguarda la **prenotazione di un appuntamento**. Il paziente seleziona uno slot, conferma la prenotazione e il sistema crea un appuntamento nel proprio dominio scheduling. L'appuntamento deve poi essere visibile nell'elenco personale, consultabile nel dettaglio e identificabile tramite stato coerente. Su un appuntamento valido il paziente deve inoltre poter eseguire:

- **riplanificazione**, se ammessa dal contesto;
- **annullamento**, se ammesso dal contesto.

Questo scenario è particolarmente importante perché verifica l'integrazione tra scheduling, notifiche e interfaccia utente.

Il quarto blocco riguarda il **pre-triage**. Su un appuntamento appropriato, il paziente deve poter aprire il questionario, compilarlo e salvarlo. Il contenuto inserito deve poi risultare recuperabile sia dal lato paziente sia dal lato clinician, a conferma del corretto collegamento tra patient, clinical e scheduling.

Il quinto blocco riguarda il **pagamento in app**. Il paziente deve poter:

- creare un payment intent per un appuntamento;
- processare il relativo intento;
- consultare l'elenco dei propri intenti;
- verificare il risultato della transazione.

Lo scenario è valido quando il pagamento genera gli oggetti economici corretti, li collega all'appuntamento giusto e aggiorna gli stati in modo coerente rispetto agli esiti ricevuti dal provider simulato.

Il sesto blocco riguarda la **consultazione dei referti**. Una volta che il referto sia stato pubblicato dal Clinician, il paziente deve poterlo visualizzare nella propria area dedicata. Ciò consente di verificare il corretto passaggio dal workflow clinico interno alla fruizione controllata del documento da parte dell'utente finale.

Infine, il paziente deve poter consultare le **notifiche** a lui indirizzate e marcare come lette quelle già visualizzate. Questo passaggio conferma il corretto comportamento del dominio events dal punto di vista dell'utente finale.

16.4. SCENARIO DELEGATE END-TO-END

Lo scenario **delegate end-to-end** verifica il comportamento della piattaforma quando un soggetto opera per rappresentanza in forza di una **delega valida**. Questo scenario è particolarmente rilevante perché testa non solo i flussi funzionali, ma anche i controlli autorizzativi e relazionali che distinguono il delegato dal paziente titolare del dato.

Il punto di partenza è la presenza di un account delegate attivato e di almeno una delega valida associata a un assistito. Una volta eseguito il login, il frontend deve esporre l'area dedicata al ruolo Delegate e rendere chiaro il contesto di rappresentanza, evitando ambiguità tra soggetto autenticato e soggetto per conto del quale si opera.

Il primo blocco di verifica riguarda il **profilo personale del delegato**. L'utente deve poter consultare e aggiornare il proprio profilo e, separatamente, visualizzare l'elenco delle deleghe a lui associate. Questo conferma che il sistema non confonde il profilo del delegato con quello del paziente, pur consentendogli di operare nel contesto dell'assistito.

Il secondo blocco riguarda la **ricerca disponibilità** e la **prenotazione per conto dell'assistito**. Il delegato deve poter interrogare la disponibilità, selezionare una prestazione e creare un appuntamento riferito al paziente rappresentato. La validazione è corretta quando:

- il sistema richiede o assume il corretto contesto paziente;
- la delega è verificata;
- l'appuntamento viene creato come prenotazione dell'assistito e non del delegato stesso.

Successivamente deve essere possibile:

- consultare gli appuntamenti dell'assistito;
- accedere al dettaglio;
- annullare o ripianificare, se ammesso.

Il terzo blocco riguarda il **pre-triage**. Il delegato deve poter recuperare il questionario associato a un appuntamento dell'assistito e compilarlo per suo conto. Anche in questo caso la validazione richiede che il collegamento alla delega sia sempre presente e che il contenuto venga poi reso disponibile nel dominio clinical come informazione riferita al paziente, non al delegato.

Il quarto blocco riguarda il **pagamento per rappresentanza**. Il delegate deve poter:

- creare un payment intent per l'appuntamento dell'assistito;

- processarlo;
- consultare l'elenco dei propri intenti riferiti ai pazienti rappresentati.

Questo scenario è particolarmente significativo perché combina:

- controllo della delega;
- dominio payments;
- collegamento all'appuntamento del paziente;
- visibilità delle transazioni in ottica di rappresentanza.

Il quinto blocco riguarda la **consultazione dei referti** dell'assistito. Il delegato deve poter vedere soltanto i referti riferiti ai pazienti per i quali esiste una delega valida e solo entro il perimetro di accesso consentito. La correttezza di questo scenario dimostra la buona integrazione tra registry, clinical e controlli di autorizzazione contestuale.

Infine, il delegato deve poter consultare e marcare come lette le **notifiche** a lui indirizzate, con eventuale distinzione rispetto all'assistito selezionato. Questa funzionalità completa il quadro della rappresentanza digitale, mostrando che la piattaforma governa in modo coerente non solo le azioni, ma anche il flusso informativo verso il delegato.

16.5. SCENARIO DI CONSULTAZIONE AI

Lo scenario di consultazione AI verifica il comportamento del modulo **AI Assistant** nel contesto per cui è stato progettato: utilizzo da parte del **Clinician** come strumento di supporto informativo, integrato nella piattaforma ma sottoposto a vincoli di accesso e sicurezza specifici.

Il punto di partenza è l'autenticazione del Clinician e l'accesso all'area dedicata dell'interfaccia. Il frontend deve rendere disponibile la pagina dell'assistente AI solo a fronte di un contesto di sessione coerente con il ruolo clinico. Questa è già una prima validazione importante, perché conferma l'allineamento tra segmentazione dell'interfaccia e modello autorizzativo del sistema.

Il primo blocco di verifica riguarda una richiesta **puramente testuale**. Il Clinician inserisce una domanda nel form dedicato e il frontend invia la richiesta al path `/api/ai/clinicians/qa`. Il gateway inoltra il traffico al modulo AI, il servizio:

- valida il token;
- ne verifica l'introspection;
- controlla il ruolo Clinician;
- valida la richiesta;
- avvia la generazione della risposta.

La validazione è soddisfatta quando il sistema restituisce un payload coerente, contenente la risposta generata e i metadati previsti sul modello e sulla sicurezza.

Il secondo blocco riguarda l'uso di **input contestualizzato** con campo context valorizzato. In questo caso si verifica che il servizio utilizzi in modo combinato domanda e contesto, mantenendo la stessa pipeline di sicurezza e lo stesso formato di risposta.

Il terzo blocco riguarda il caricamento di **allegati documentali**. Il Clinician invia una richiesta multipart con uno o più file supportati. Il servizio deve:

- accettare soltanto formati ammessi;
- estrarre il testo dagli allegati;
- applicare i limiti dimensionali previsti;
- includere il contenuto nel prompt;
- restituire metadati sugli allegati effettivamente processati.

Questo scenario è particolarmente importante perché verifica il comportamento più ricco del modulo AI e la sua capacità di integrare input eterogenei senza perdere coerenza.

Il quarto blocco riguarda le **salvaguardie sui contenuti**. La validazione è corretta quando una richiesta che contiene dati identificativi diretti del paziente viene bloccata dal servizio con risposta di errore coerente, senza avvio dell'inferenza. Questo passaggio dimostra che il modulo non si limita a generare testo, ma applica una barriera di sicurezza specifica prima del processamento.

Il quinto blocco riguarda il **funzionamento locale del modello**. Lo scenario è da considerarsi corretto quando il modulo è in grado di produrre la risposta utilizzando il modello caricato dalla cache locale, senza dipendenze da chiamate remote di inferenza. Questa verifica rafforza il carattere controllato e riproducibile dell'AI Assistant nel contesto architetturale della piattaforma.

16.6. VERIFICA DEI CONTROLLI DI ACCESSO

La verifica dei controlli di accesso ha lo scopo di accertare che la piattaforma applichi in modo coerente le regole di autenticazione, autorizzazione basata sui ruoli e autorizzazione contestuale. Non si tratta di uno scenario utente singolo, ma di una validazione trasversale che attraversa tutti i domini del sistema.

Un primo gruppo di verifiche riguarda la **protezione delle aree applicative** lato frontend e lato backend. Un utente non autenticato non deve poter accedere alle aree protette; un token mancante o invalido deve produrre risposta coerente e reindirizzamento al login; un utente autenticato con ruolo non compatibile non deve poter utilizzare né visualizzare interfacce di altri ruoli.

Un secondo gruppo riguarda l'**autorizzazione per ruolo**. Le verifiche più significative includono:

- impossibilità per un Patient di accedere alle API amministrative;
- impossibilità per un Admin di utilizzare l'assistente AI;
- impossibilità per un Delegate di accedere ai flussi del paziente senza delega;

- impossibilità per un Clinician di utilizzare endpoint amministrativi non coerenti con il proprio ruolo.

Un terzo gruppo riguarda l'**autorizzazione contestuale**. Questa è particolarmente importante perché molti endpoint non si proteggono solo tramite ruolo, ma tramite pertinenza della risorsa. Alcuni esempi di validazione rilevanti sono:

- un Clinician può visualizzare solo i propri appuntamenti e encounter;
- un Patient può vedere solo i propri referti, pagamenti e notifiche;
- un Delegate può operare solo per gli assistiti collegati da una delega valida;
- un referto o un pre-triage non devono essere accessibili fuori dal contesto corretto.

Un quarto gruppo riguarda la **revoca e l'introspection del token**. Dopo il logout, il token deve risultare inutilizzabile sia verso il Core Service sia verso il modulo AI, a conferma del fatto che la piattaforma non considera valida una sessione soltanto in base alla scadenza temporale del JWT.

Nel complesso, la verifica dei controlli di accesso è soddisfatta quando il sistema dimostra tre proprietà simultanee:

- riconosce correttamente chi sta chiamando;
- applica il ruolo corretto;
- verifica che la risorsa richiesta sia pertinente al soggetto e al processo in corso.

16.7. VERIFICA DI HEALTH E DISPONIBILITA' DEI SERVIZI

La verifica di **health** e disponibilità dei servizi ha lo scopo di confermare che la piattaforma sia non solo funzionalmente corretta, ma anche operativamente osservabile e coerente dal punto di vista del runtime. In un'architettura composta da più container e servizi cooperanti, la disponibilità del sistema non coincide con il solo avvio del frontend, ma richiede il corretto funzionamento coordinato di più componenti.

Il primo livello di verifica riguarda il **frontend**. La piattaforma deve risultare raggiungibile sul punto di esposizione principale e deve essere in grado di servire correttamente le pagine statiche, gli asset e le chiamate verso il proxy `/api/*`. Questo verifica la disponibilità del container Nginx e la correttezza della configurazione iniziale del web server.

Il secondo livello riguarda il **gateway**. Il suo stato locale deve risultare disponibile tramite l'endpoint di health dedicato, mentre il suo comportamento applicativo deve dimostrare la capacità di ricevere traffico dal frontend, applicare i middleware e instradare correttamente le richieste verso i servizi downstream.

Il terzo livello riguarda la **health aggregata** esposta dal gateway sul path `/api/health`. Questa rappresenta la vista più significativa dal punto di vista sistemico, perché verifica con una singola chiamata:

- la raggiungibilità del Core Service;
- la raggiungibilità dell'AI Assistant;

- la capacità del gateway di aggregare i risultati in modo coerente.

Il quarto livello riguarda il **Core Service**. La sua health è valida quando il servizio non solo è avviato, ma è anche in grado di interrogare correttamente i cinque schemi MySQL di riferimento e di restituire uno stato positivo sulla propria capacità operativa minima. La disponibilità del Core Service è infatti strettamente legata a quella della persistenza.

Il quinto livello riguarda il **modulo AI**. La sua health deve confermare che il servizio Django/Gunicorn sia avviato e pronto a ricevere richieste. In combinazione con gli scenari di consultazione AI, questa verifica conferma che il modulo non è soltanto raggiungibile, ma effettivamente in grado di elaborare richieste coerenti con la configurazione runtime.

Il sesto livello riguarda i **servizi di supporto**, in particolare:

- **MySQL**, che deve essere raggiungibile e coerente con gli schemi previsti;
- **Mailpit**, che deve essere disponibile per la ricezione delle comunicazioni SMTP e per la loro ispezione tramite interfaccia web.

La validazione della disponibilità del sistema è quindi completa quando:

- il frontend è raggiungibile;
- il gateway risponde e instrada correttamente;
- `/api/health` restituisce uno stato aggregato coerente;
- il Core Service risulta in grado di operare sui propri database;
- il modulo AI è raggiungibile e pronto all'uso;
- i servizi di supporto sono attivi e integrati nel runtime.

17. STRUTTURA DELLA REPOSITORY

17.1. ORGANIZZAZIONE DELLE DIRECTORY PRINCIPALI

La repository è organizzata secondo una struttura che riflette in modo diretto la scomposizione architetturale della piattaforma e ne rende immediatamente leggibili i principali blocchi funzionali e infrastrutturali. La directory radice contiene gli elementi necessari all'avvio dell'intero stack, alla configurazione dell'ambiente, alla definizione della persistenza e all'implementazione dei singoli servizi applicativi.

A livello superiore, la struttura è articolata intorno ai seguenti elementi principali:

- `.env`
- `.gitignore`
- `.vscode/`
- `db/`
- `docker-compose.yml`
- `docs/`
- `frontend/`
- `hf_cache/`
- `README.md`
- `services/`

La presenza del file `.env` indica che la piattaforma adotta una configurazione fortemente parametrizzata tramite variabili d'ambiente, coerentemente con una strategia di deploy containerizzata e con la necessità di separare i parametri operativi dal codice sorgente. Il file `.gitignore` formalizza invece le esclusioni dal versionamento, contribuendo a mantenere la repository pulita e coerente con le pratiche di sviluppo del progetto.

La directory `.vscode/` contiene impostazioni locali dell'ambiente di sviluppo. Pur non incidendo direttamente sull'architettura applicativa, essa testimonia una strutturazione della repository che tiene conto anche del supporto agli strumenti di lavoro utilizzati durante lo sviluppo e la manutenzione.

La directory `db/` raccoglie gli artefatti relativi al bootstrap del livello dati. Al suo interno, il sottopercorso `db/init/` contiene lo script SQL di creazione degli schemi logici, elemento essenziale per la realizzazione della strategia *schema-per-domain* adottata dal sistema. Questa collocazione è corretta e leggibile, perché isola la parte di inizializzazione strutturale del database dal resto del codice applicativo.

Il file `docker-compose.yml` occupa una posizione centrale nella root della repository, coerentemente con il suo ruolo di manifesto runtime dell'intera piattaforma. Esso rappresenta il punto di raccordo tra servizi, rete, volumi, porte e configurazioni, e la sua collocazione in radice rafforza l'idea che il deployment orchestrato sia una proprietà primaria del progetto, non un elemento accessorio.

Il file `README.md` in root ha la funzione di fornire una documentazione operativa sintetica e immediatamente accessibile, dedicata in particolare alle istruzioni di **build & deploy** della piattaforma. La sua presenza nella directory principale è coerente con le best practice di organizzazione della repository, poiché rende disponibili fin dal primo livello le informazioni essenziali per comprendere come avviare, configurare ed eseguire il sistema.

La directory `docs/` è destinata a raccogliere la documentazione tecnica di progetto in forma di artefatti finali. Al suo interno sono previsti, in particolare, la **relazione in formato .pdf** e l'**immagine del diagramma architetturale in formato .png**. Questa collocazione è particolarmente appropriata, poiché separa in modo ordinato il codice eseguibile dagli elaborati documentali che descrivono architettura, scelte progettuali e modalità di presentazione della soluzione.

La directory `frontend/` contiene l'intera applicazione web lato presentazione, organizzata a sua volta in sottosezioni dedicate ad asset, componenti, pagine, script, stili e configurazione Nginx. La directory `services/` raccoglie invece i servizi backend della piattaforma, separati secondo le loro responsabilità architeturali: gateway, Core Service e AI Assistant. La directory `hf_cache/`, infine, è dedicata alla disponibilità locale del modello AI e rappresenta una risorsa runtime specialistica, coerentemente separata dal codice del servizio che la utilizza.

Nel complesso, l'organizzazione delle directory principali riflette un criterio preciso: distinguere in modo chiaro tra **configurazione generale**, **frontend**, **servizi backend**, **persistenza** e **risorse di supporto**, mantenendo la repository ordinata e pienamente allineata alla struttura logica del sistema.

17.2. SEPARAZIONE TRA FRONTEND, GATEWAY, CORE SERVICE E AI ASSISTANT

La repository implementa in modo molto netto la separazione tra i quattro macro-blocchi applicativi principali della piattaforma: **frontend**, **API Gateway**, **Core Service** e **AI Assistant**. Questa suddivisione non è soltanto coerente con il diagramma architetturale, ma emerge chiaramente già dalla struttura fisica del codice.

Il **frontend** è interamente collocato nella directory `frontend/`. Al suo interno si distinguono:

- `assets/` per risorse statiche come favicon;
- `components/` per frammenti HTML riutilizzabili, come navbar, footer e modale;
- `css/` per gli stili condivisi;
- `js/` per la logica client-side, ulteriormente segmentata in `admin/`, `auth/`, `clinician/`, `common/`, `delegate/` e `patient/`;
- `pages/` per le pagine HTML organizzate per ruolo;
- `_errors/` per le pagine statiche di errore o fallback;
- `Dockerfile` e `nginx.conf` per il packaging e l'esposizione del frontend.

Questa struttura evidenzia una progettazione frontend ordinata e coerente con il modello multi-page. La distinzione tra pagine e script per ruolo conferma la volontà di mantenere

separate le aree applicative e di rendere leggibile il rapporto tra interfaccia, comportamento lato client e responsabilità funzionale.

L'**API Gateway** è collocato in `services/api-gateway/`. La struttura interna riflette in modo molto chiaro la natura infrastrutturale del componente:

- `cmd/gateway/` per il punto di ingresso dell'applicazione;
- `internal/config/` per la configurazione del servizio;
- `internal/handlers/` per la gestione delle richieste, incluse health e proxy;
- `internal/http/` per il router;
- `internal/middleware/` per request ID, logging, recovery, CORS, rate limiting e verifica JWT;
- `internal/proxy/` per il reverse proxy;
- `internal/response/` per la normalizzazione delle risposte di errore;
- `Dockerfile`, `go.mod` e `go.sum` per build e dipendenze del modulo.

Questa articolazione è particolarmente corretta perché isola nel gateway tutto ciò che appartiene al controllo del traffico, al dispatching e alle responsabilità trasversali, senza mescolarlo con logica di business.

Il **Core Service** è collocato in `services/core-service/` e costituisce il blocco strutturalmente più ricco della repository. Oltre al file di `solution CoreService.sln` e al relativo `Dockerfile`, esso contiene la directory `src/`, al cui interno sono presenti i quattro progetti che implementano l'architettura a layer:

- `CoreService.Api/`
- `CoreService.Application/`
- `CoreService.Domain/`
- `CoreService.Infrastructure/`

Questa organizzazione riflette perfettamente la progettazione già descritta sul piano architetturale. Ogni progetto ha un ruolo specifico e ben riconoscibile, e la repository ne rende evidente la separazione già a livello di cartelle e namespace.

L'**AI Assistant** è collocato in `services/ai-assistant/` ed è strutturato secondo una logica tipica del mondo Django, ma adattata a una funzione molto specializzata. Al suo interno si trovano:

- `ai_assistant/` per la configurazione principale del progetto Django;
- `assistant/api/` per autenticazione, permission, serializers, URL e viste REST;
- `assistant/services/` per client AI, estrazione del testo dai file, autenticazione JWT e sanitizzazione del testo;

- `manage.py`, `requirements.txt` e `Dockerfile` per gestione del servizio, dipendenze e packaging.

Anche in questo caso la struttura fisica della repository riflette una separazione molto chiara tra configurazione del framework, esposizione API e logica specialistica del servizio AI. L'isolamento del modulo rispetto al Core Service emerge dunque non solo a livello concettuale, ma anche nella sua collocazione concreta all'interno del progetto.

17.3. CARTELLE INFRASTRUTTURALI E DI SUPPORTO

Oltre ai componenti applicativi principali, la repository contiene alcune cartelle e file dedicati agli aspetti **infrastrutturali** e **di supporto**, che contribuiscono in modo significativo alla completezza del sistema e alla sua capacità di essere eseguito in modo coerente.

La directory `db/init/` è il primo esempio rilevante. La presenza dello script `create_schemas.sql` dimostra che la repository incorpora la definizione iniziale della persistenza e non delega a documentazione esterna o a procedure manuali la creazione degli schemi di database. Questa scelta migliora la riproducibilità del deploy e rafforza il legame tra modello dei dati e runtime effettivo.

La directory `hf_cache/` è particolarmente importante per il modulo AI. Essa non contiene codice applicativo, ma una risorsa di supporto essenziale per il funzionamento locale del modello Hugging Face. La sua collocazione separata dal servizio AI è corretta, perché ne evidenzia la natura di **dipendenza runtime esterna al codice**, pur restando parte integrante dell'ambiente di esecuzione del sistema.

All'interno del frontend, file come `nginx.conf` e `Dockerfile` rappresentano a loro volta elementi infrastrutturali rilevanti. Il primo definisce il comportamento del web server e del proxy iniziale verso il gateway; il secondo formalizza il packaging del frontend statico. Il fatto che tali file siano collocati direttamente nella directory del frontend è coerente con l'idea che ogni macro-componente debba possedere al proprio interno gli artefatti necessari alla propria esecuzione.

In modo analogo, ogni servizio backend include il proprio **Dockerfile** e i propri file di configurazione o gestione delle dipendenze:

- `go.mod` e `go.sum` per il gateway;
- `requirements.txt` per l'AI Assistant;
- `appsettings.json` e file `.csproj` per il Core Service.

Questa distribuzione rafforza l'autonomia dei singoli componenti e rende più leggibile il rapporto tra codice e runtime di riferimento.

Le cartelle `frontend/_errors/` e `frontend/components/` possono anch'esse essere considerate parti di supporto dell'esperienza applicativa. La prima raccoglie pagine statiche di errore o di sessione scaduta, contribuendo alla robustezza del frontend; la seconda ospita frammenti riutilizzabili che supportano la coerenza del layout e dell'interazione.

Nel Core Service, hanno una particolare rilevanza infrastrutturale percorsi come:

- `CoreService.Infrastructure/Persistence/`
- `CoreService.Infrastructure/Email/`

La prima area contiene DbContext, repository, seed e inizializzazione del database; la seconda gestisce la consegna SMTP e il background service di dispatch delle e-mail. Questi percorsi mostrano che la repository non separa solo i macro-servizi, ma organizza con ordine anche le **capacità di supporto interne** al servizio principale.

Nel complesso, le cartelle infrastrutturali e di supporto della repository confermano che il progetto non si limita a contenere il codice di business, ma incorpora tutti gli artefatti necessari a rendere il sistema:

- avviabile;
- configurabile;
- osservabile;
- dimostrabile;
- coerente con il deployment dichiarato.

17.4. COERENZA TRA REPOSITORY E ARCHITETTURA IMPLEMENTATA

Uno degli aspetti più significativi della repository è la sua elevata **coerenza con l'architettura implementata**. La struttura delle directory, la collocazione dei file di configurazione, la separazione dei servizi e l'organizzazione interna dei moduli riflettono in modo molto fedele la scomposizione concettuale descritta dal diagramma architetturale e dalla progettazione del sistema.

Il primo elemento di coerenza riguarda la corrispondenza diretta tra i **runtime componenti** della piattaforma e le directory principali della repository. Il frontend statico, il gateway Go/Gin, il Core Service in ASP.NET Core e l'AI Assistant in Django/DRF sono tutti rappresentati come blocchi fisicamente separati. Questa scelta rende immediatamente leggibile la topologia del sistema già a partire dall'albero dei file e riduce il rischio di accoppiamenti strutturali non desiderati.

Il secondo elemento riguarda il rapporto tra **architettura logica** e **organizzazione interna del codice**. Nel Core Service, ad esempio, la separazione tra API, Application, Domain e Infrastructure non resta una semplice dichiarazione progettuale, ma si concretizza in quattro progetti distinti della solution. Analogamente, nel gateway la distinzione tra router, handlers, middleware, proxy e gestione degli errori emerge in directory dedicate. Nel modulo AI, la distinzione tra configurazione Django, API REST e servizi specialistici è altrettanto evidente.

Il terzo elemento riguarda il rapporto tra **repository** e **strategia di deployment**. La presenza del file `docker-compose.yml` in radice, dei Dockerfile dedicati per ciascun componente, del file `README.md` per le istruzioni di build & deploy e delle risorse di supporto come `db/init/`, `docs/` e `hf_cache/` mostra che la repository è progettata per essere non solo un contenitore di codice, ma anche il **manifesto operativo e documentale** dell'intero sistema. Il deploy dichiarato nell'architettura trova infatti una

corrispondenza concreta e immediata nei file presenti nel progetto, mentre la documentazione finale risulta collocata in uno spazio coerente e dedicato.

Il quarto elemento riguarda la coerenza tra **segmentazione dei ruoli applicativi** e organizzazione del frontend. Le directory `pages/admin`, `pages/auth`, `pages/clinician`, `pages/delegate`, `pages/patient` e le corrispondenti sottodirectory JavaScript confermano che la separazione dei ruoli, descritta a livello funzionale e di sicurezza, è stata tradotta fedelmente anche nella struttura della repository. Questo rafforza la leggibilità complessiva del sistema e rende più agevole il mantenimento delle singole aree applicative.

Anche la strategia *schema-per-domain* trova una chiara espressione nella repository. Essa non è rappresentata soltanto nello script SQL di creazione dei database, ma anche nella presenza di:

- DbContext distinti;
- repository separati per dominio;
- seed coerente con i cinque database logici;
- moduli applicativi allineati a registry, scheduling, clinical, payments ed events.

Questa convergenza tra struttura del codice, struttura del dato e architettura documentata è un indicatore molto positivo della qualità del progetto. Una repository realmente coerente non obbliga il lettore a ricostruire mentalmente un'architettura non visibile nei file; al contrario, rende l'architettura **ispezionabile**, **riconoscibile** e **confermata** dalla forma concreta del codice e delle configurazioni presenti.

18. CONSIDERAZIONI FINALI

18.1. SINTESI DEI RISULTATI OTTENUTI

La piattaforma realizzata consente di governare in modo integrato un insieme articolato di processi amministrativi, organizzativi, clinici ed economici, mantenendo una chiara distinzione tra ruoli, responsabilità operative e confini applicativi. Il risultato ottenuto non consiste in una semplice aggregazione di funzionalità isolate, ma nella costruzione di un sistema coerente, nel quale i principali flussi del dominio risultano collegati tra loro da un modello architetturale leggibile e da una persistenza strutturata secondo criteri di separazione semantica.

Dal punto di vista funzionale, la soluzione copre in modo esteso l'intero ciclo applicativo: gestione del ciclo di vita dell'account, autenticazione, amministrazione di profili e deleghe, consultazione del catalogo, disponibilità e prenotazioni, check-in, pre-triage, presa in carico clinica, redazione e pubblicazione del referto, pagamenti, notifiche, audit e supporto consultivo tramite modulo AI. La piattaforma rende quindi possibile una continuità di processo che attraversa più domini senza perdere chiarezza nella distribuzione delle responsabilità.

Sul piano tecnico, il sistema realizza un'integrazione efficace tra un frontend web multi-page, un **API Gateway** dedicato alla gestione del perimetro esposto, un **Core Service** organizzato in layer e moduli di dominio, un **AI Assistant** separato e stateless, una persistenza relazionale *schema-per-domain* e un ambiente di esecuzione completamente containerizzato. Tale combinazione consente di ottenere una soluzione strutturata, portabile e aderente a criteri di qualità architetturale generalmente richiesti in contesti software professionali.

Un risultato particolarmente rilevante è la capacità del sistema di mantenere allineati tre livelli che, in molti progetti, tendono a divergere: **modello funzionale**, **architettura software** e **organizzazione concreta del codice**. Nella soluzione realizzata, questi tre elementi si presentano invece come reciprocamente coerenti: i ruoli applicativi trovano corrispondenza nelle interfacce, i domini di business nei moduli del Core Service, i confini della persistenza negli schemi di database e i servizi runtime nella topologia definita da Docker Compose.

18.2. COERENZA TRA ARCHITETTURA PROGETTATA E IMPLEMENTAZIONE

Uno degli aspetti più significativi della soluzione è l'elevata coerenza tra l'architettura progettata e la sua implementazione effettiva. Il disegno architetturale non è rimasto confinato a una rappresentazione teorica o documentale, ma è stato tradotto in una struttura concreta di servizi, directory, moduli, endpoint, database e configurazioni runtime che riflettono in modo diretto le scelte progettuali definite.

La distinzione tra **Presentation Tier**, **Integration & Security Tier**, **Application Services Tier** e **Data & Supporting Services** trova una realizzazione chiara nella piattaforma. Il frontend è distribuito come applicazione statica servita da Nginx; il gateway in Go/Gin opera come punto di ingresso unico per le API; il Core Service in ASP.NET Core costituisce il centro della logica di business; l'AI Assistant in Django/DRF è confinato in un

servizio separato; MySQL, Mailpit e la cache locale del modello completano il runtime come servizi di supporto coerenti con il diagramma architetturale.

Anche la progettazione interna del Core Service si dimostra allineata all'impostazione definita. La separazione in **Api**, **Application**, **Domain** e **Infrastructure** non rappresenta una classificazione nominale, ma una concreta organizzazione del codice che contribuisce a mantenere distinto il livello di esposizione HTTP, il coordinamento dei casi d'uso, la semantica del dominio e i dettagli infrastrutturali. Questa corrispondenza rafforza la manutenibilità della soluzione e testimonia un'implementazione disciplinata, non opportunistica.

La strategia di persistenza *schema-per-domain* costituisce un ulteriore punto di forte coerenza. I domini principali della piattaforma — registry, scheduling, clinical, payments ed events — sono rappresentati sia a livello concettuale sia a livello di schema database, con una distribuzione dei dati che rispecchia i confini funzionali del sistema. Ciò riduce l'ambiguità del modello informativo e migliora la tracciabilità tra moduli applicativi e ownership del dato.

Anche il modulo AI risulta pienamente coerente con l'impostazione architetturale prevista. Esso non è stato incorporato nel Core Service né trattato come estensione generica del backend, ma realizzato come servizio specialistico autonomo, autenticato tramite JWT del Core Service, accessibile solo al ruolo Clinician, dotato di salvaguardie specifiche sugli input e privo di persistenza locale. Questo rappresenta un segnale di particolare maturità progettuale, perché dimostra la volontà di integrare una componente avanzata senza compromettere la leggibilità e la sicurezza dell'ecosistema complessivo.

18.3. PUNTI DI FORZA DELLA SOLUZIONE

Uno dei principali punti di forza della piattaforma è la **chiarezza della separazione delle responsabilità**. Ogni componente, ogni modulo e ogni ruolo applicativo operano entro un perimetro definito: il frontend gestisce la presentazione, il gateway presidia il traffico API, il Core Service governa il business, il modulo AI fornisce supporto consultivo, i database sono separati per dominio e gli attori della piattaforma interagiscono secondo ruoli distinti e coerenti. Questa impostazione riduce l'accoppiamento, migliora la leggibilità del sistema e rafforza la sicurezza complessiva.

Un secondo punto di forza è la **coerenza end-to-end dei flussi operativi**. La soluzione non si limita a esporre endpoint o schermate scollegate, ma supporta scenari completi che attraversano il sistema in modo ordinato: registrazione e attivazione dell'account, creazione o gestione dei profili, prenotazione, check-in, pre-triage, presa in carico clinica, refertazione, pagamento, notifica e tracciamento. Questa continuità di processo attribuisce alla piattaforma una qualità sostanzialmente superiore rispetto a sistemi costruiti per funzionalità isolate e non per catene operative integrate.

Un terzo elemento di forza è rappresentato dal **modello di sicurezza**. L'uso di JWT, la revoca dei token, la token introspection, la segmentazione delle interfacce per ruolo, la separazione tra ambito amministrativo e ambito clinico, il controllo delle deleghe e l'audit strutturato costituiscono un insieme coerente di misure che migliorano l'affidabilità della piattaforma e ne limitano la superficie di rischio. Particolarmente positiva è anche

l'attenzione dedicata al modulo AI, che viene protetto da controlli di ruolo e da filtri sui dati identificativi in ingresso.

Un ulteriore punto di forza è la **qualità dell'organizzazione del codice e della repository**. La struttura del progetto rende immediatamente leggibile la corrispondenza tra architettura e implementazione, facilita la navigazione tra i componenti e riduce il costo cognitivo necessario per comprendere la piattaforma. Anche dal punto di vista del deploy, la presenza di Dockerfile distinti, di una configurazione Compose centralizzata, degli script di bootstrap del database e della cache locale del modello AI contribuisce a una gestione ordinata del runtime.

Merita inoltre particolare attenzione la scelta di introdurre un **modulo AI consultivo locale**, integrato nell'architettura senza renderlo invasivo. L'assistente non altera i confini di business del sistema, ma aggiunge una capacità specialistica che arricchisce la piattaforma mantenendo un profilo di rischio e complessità controllato. Questa integrazione rappresenta un punto di forza non solo dal punto di vista funzionale, ma anche dal punto di vista della qualità del disegno architetturale.

18.4. POSSIBILI EVOLUZIONI FUTURE

La piattaforma presenta una base architetture sufficientemente solida da consentire evoluzioni future significative senza richiedere una ridefinizione radicale del modello generale. Le principali direttrici di crescita riguardano l'approfondimento dei singoli domini, il rafforzamento delle capacità operative e l'estensione dell'osservabilità e dell'integrazione con servizi esterni.

Una prima area di evoluzione riguarda il dominio **clinical**. Il sistema potrebbe essere esteso introducendo una gestione più ricca della storia clinica longitudinale, la modellazione di ulteriori tipologie documentali, la gestione di allegati clinici strutturati, workflow più avanzati di firma e pubblicazione, oppure funzionalità di consultazione comparativa tra episodi assistenziali diversi. Questa evoluzione sarebbe coerente con l'attuale struttura del modulo, che già distingue in modo chiaro encounter, ordini, esecuzioni e referto.

Una seconda area di crescita riguarda il dominio **payments**. L'introduzione di un'integrazione con un provider di pagamento reale, la gestione di notifiche asincrone più articolate, il supporto a rimborsi o storni, nonché l'aggiunta di funzionalità di reportistica amministrativa, costituirebbero sviluppi naturali di un modulo che possiede già una buona separazione tra payment intents, transazioni, checkout workflow e webhook.

Un'evoluzione significativa potrebbe interessare anche il dominio **events**, ad esempio mediante l'introduzione di canali di notifica aggiuntivi, meccanismi di recapito più sofisticati, regole configurabili di scheduling o strumenti di supervisione più ricchi per la consultazione dell'audit e delle notifiche. La presenza di un database dedicato e di un dispatcher in background rende il modulo particolarmente adatto a una crescita in questa direzione.

Sul piano dell'**osservabilità**, la soluzione potrebbe beneficiare dell'introduzione di metriche più avanzate, dashboard operative, tracing distribuito e monitoraggio strutturato dei tempi di risposta e degli errori per dominio. L'architettura attuale, con gateway

dedicato, health aggregata e logging strutturato, fornisce già una base utile, ma lascia spazio a un'evoluzione verso strumenti più completi di monitoraggio e diagnosi.

Anche il **modulo AI** si presta a sviluppi ulteriori. Potrebbero essere introdotti modelli diversi selezionabili a configurazione, pipeline di ranking o retrieval locale, filtri di sicurezza più sofisticati, controlli di conformità più avanzati sugli allegati o strumenti di tracciamento delle richieste AI in ottica operativa. Sarebbe inoltre possibile estendere il supporto documentale, purché ciò avvenga senza compromettere la natura stateless e il perimetro consultivo del servizio.

Dal punto di vista infrastrutturale, la piattaforma potrebbe evolvere verso scenari di deploy più articolati, con separazione tra ambienti, gestione più strutturata dei secret, pipeline CI/CD, orchestrazione su piattaforme container-native e politiche di scaling differenziate per i componenti più sensibili al carico, in particolare il modulo AI. La chiarezza della separazione attuale tra servizi costituisce un presupposto favorevole anche per questo tipo di crescita.

Infine, una linea evolutiva particolarmente interessante riguarda l'introduzione di ulteriori meccanismi di **configurabilità dei flussi**. Alcune regole attualmente incorporate nei comportamenti applicativi potrebbero essere estese tramite parametri di business configurabili, rendendo la piattaforma più adattabile a contesti organizzativi differenti pur mantenendo intatti i propri principi architetturali di base.